Artificial Intelligence Review (2025) 58:23
https://doi.org/10.1007/s10462-024-10989-8

Deep learning-based time series forecasting

Xiaobao Song[1,2] · Liwei Deng[2] · Hao Wang[1,2] · Yaoan Zhang[2] · Yuxin He[3] · Wenming Cao[1,2]

Accepted: 4 October 2024 / Published online: 25 November 2024

Abstract
With the advancement of deep learning algorithms and the growing availability of computational power, deep learning-based forecasting methods have gained significant importance in the domain of time series forecasting. In the past decade, there has been a rapid rise in time series forecasting approaches. This paper comprehensively reviews the advancements in deep learning-based forecasting models spanning 2014 to 2024. We provide a comprehensive examination of the capabilities of these models in capturing correlations among time steps and time series variables. Additionally, we explore methods to enhance the efficiency of long-term time series forecasting and summarize the diverse loss functions employed in these models. Moreover, this study systematically evaluates the effectiveness of these approaches in both univariate and multivariate time series forecasting tasks across diverse domains. We comprehensively discuss the strengths and limitations of various algorithms from multiple perspectives, analyze their capacity to capture different types of time series information, including trend and season patterns, and compare methods for enhancing the computational efficiency of these models. Finally, we summarize the experimental results and discuss the future directions in time series forecasting. Codes and datasets are available at https://github.com/TCCofWANG/Deep-Learning-based-Time-Series-Forecasting.

Keywords Time series forecasting · Deep learning · Time series decomposition

1 Introduction

1.1 Objective

Time series forecasting plays a crucial role in numerous applications, such as energy consumption (Wu et al. 2021; Guo et al. 2023; Son and Van Cuong 2023; Dinh et al. 2023), transportation planning (Venkateshwari et al. 2023; Hu and Xiong 2023; Chen et al. 2023) and weather forecasting (Ma et al. 2023; Mung and Phyu 2023; Chen et al. 2023). In these practical application scenarios, forecasting future time series with the help of historical data

Extended author information available on the last page of the article

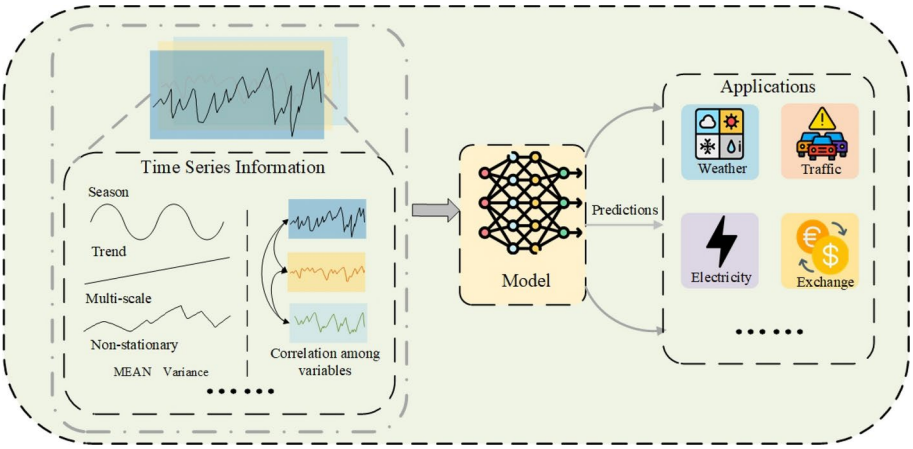

Fig. 1 Process of time series forecasting

is of great significance for long-term planning and early warning in related fields (Wang et al. 2023; Miller et al. 2024). This process can be shown in Fig. 1. Therefore, this paper aims to explore deep learning-based time series forecasting models from multiple perspectives, offering a comprehensive evaluation of current mainstream models and encouraging readers to consider future directions for development in this field.

1.2 Review of existing approaches

Next, we will summarize the existing methods from four main perspectives: time-step dependencies, correlations between temporal variables, the trade-off between expanding the model's receptive field and reducing computational costs, and loss functions.

Time series information usually consists of *time-step dependencies* and *correlations between temporal variables* (Parzen 1961; Lacasa et al. 2015; Hsieh 2004; Orang et al. 2023). Fully exploiting these two types of information plays a crucial role in improving the model's capability. Traditional models like the autoregressive integrated moving average (ARIMA) (Zhang 2003; Ariyo et al. 2014; Contreras et al. 2003) rely on statistical properties to extract information. However, they often fall short in fully capturing the time-step dependencies within complex time series. This is because traditional models primarily focus on linear features, while real-time series data usually contains intricate nonlinear correlations. As a result, traditional models struggle to adequately leverage these dependencies. In recent years, with the development of algorithms related to deep lCR121earning and the improvement of computational power, deep learning-based methods have become increasingly crucial in time series forecasting. Autoencoder and Stacked Autoencoders (SAE) (Lv et al. 2014) are utilized to extract the time-step features of the time series and obtain the prediction results directly. (CNNs) (Gudelek et al. 2017; Markova 2022; Lu et al. 2020; Zhao et al. 2017; Liu et al. 2018; Hatami et al. 2018) are often used to extract time-series features in the short-term range because of their ability to aggregate time-step data in the receptive field. To capture a broader range of temporal dependencies, the Temporal Convolutional Network (TCN) (Bai et al. 2018) expands the receptive field of its convolutional kernel. Specifically, TCN introduces dilated causal convolutions to time-series forecasting tasks. In

contrast, Recurrent Neural Networks (RNNs) (Shi et al. 2015; Dey and Salem 2017; Salinas et al. 2020; Hajirahimi and Khashei 2023; Lai et al. 2018) are specialized for temporal sequences and, theoretically, do not suffer from the limited receptive fields characteristic of CNNs. However, related studies have shown that the recurrent structure of RNNs can lead to issues such as vanishing gradient, limiting their ability to leverage long-term dependencies in time series. To address this, researchers have developed models like Long Short-Term Memory (LSTM) (Shi et al. 2015; Fischer and Krauss 2018; Zheng et al. 2017) and Gated Recurrent Units (GRU) (Dey and Salem 2017), which employ gating mechanisms to better capture long-term temporal correlations. Despite these advancements, LSTM and GRU models still face challenges, such as vanishing gradient (Pascanu et al. 2013; Noh 2021; Le and Zuidema 2016) and error accumulation during training (Tang et al. 2021; Fan et al. 2019; Liao et al. 20185), which are common in RNN architectures. The effectiveness (Hewamalage et al. 2021; Pavlov-Kagadejev et al. 2024) of RNN-based forecasting models declines with longer forecasting time steps.

Given the exceptional performance of Transformer models in both natural language processing (Vaswani et al. 2017; Devlin et al. 2018) and image processing (Dosovitskiy et al. 2020) domains, these models are now being introduced to time series forecasting (Wu et al. 2021; Li et al. 2019; Zhou et al. 2021). Compared with RNN-based models, Transformer based models adopt an encoder-decoder structure (Wang et al. 2022; Woo et al. 2022; Lee et al. 2024) and the attention mechanism (Liu et al. 2023; Niu et al. 2021; Young et al. 2022). It can dramatically alleviate the error accumulation (Li et al. 2019; Zhou et al. 2021) in long-term time-series forecasting tasks. However, the authors of DLinear (Yun et al. 2019) point out that the attention mechanism is permutation invariance. The Transformer model does not make good use of time series order information. To solve this problem, DLinear uses a linear layer to implement time series forecasting.

Generally speaking, the correlation between time steps in a time series comprises multiple patterns (Cleveland et al. 1990; Hyndman and Athanasopoulos 2018; Dagum 2010), including trend (Verbesselt et al. 2010) [62] (Qi and Zhang 2008; Woodward and Gray 1993), seasonal patterns (Bell and Hillmer 1984; De Livera et al. 2011), etc. To reduce the complexity of time series forecasting and capture these temporal patterns, some models have introduced time series decomposition techniques (Wu et al. 2021; Zhou et al. 2022; Oreshkin et al. 2019; Woo et al. 2022). These approaches initially decompose the time series into several components, usually containing trend information, seasonal information, time-scale information (Taylor and Letham 2018; Jiang et al. 2021; Murray et al. 2000), and other time-series information. Subsequently, the model analyzes these distinct elements using specialized modules. For instance, Autoformer (Wu et al. 2021) utilizes a mean filter to convolve the input sequence, extracting trend terms that represent the time series' trend patterns. Similarly, Fedformer (Zhou et al. 2022) employs multiple mean filters of varying sizes to derive trend terms, effectively addressing the limited receptive field issue. LSTnet (Lai et al. 2018) employs linear mapping to extract trend information from sequences and incorporates a predefined window to minimize the impact of distant information on trend forecasting. N-BEATS (Oreshkin et al. 2019) adopts a polynomial fitting method to model the time series' trend terms. Similarly, both DLinear (Yun et al. 2019) and TDformer (Zhang et al. 2022) utilize shallow linear layers for this purpose. For seasonal information, LSTnet introduces a skip-connection architecture based on LSTM, enabling the model to account for information from time steps preceding a fixed interval, thereby capturing seasonal infor-

mation in the data. Autoformer enhances the ability of the attention mechanism to discern seasonal patterns by using time-delayed similarities of the input. Fedformer, on the other hand, transforms the input time series into the frequency domain and employs an attention mechanism to analyze similarities between frequency components, effectively capturing the periodicity of the time series. ETSformer (Woo et al. 2022) filters frequency components based on their magnitude, reducing noise interference in seasonal information capturing. Time-scale information (Zhai et al. 2023), referring to the correlation of time steps at different scales, is addressed by models like Scaleformer (Shabani et al. 2022) and Pyraformer (Liu et al. 2021). The former applies downsampling to acquire inputs at various temporal scales and uses a forecasting model for multi-scale predictions. The latter processes time series at different scales and employs an attention mechanism to derive cross-scale attention correlations.

In addition to time-step dependencies, the essence of time series information also lies in the cross-time-step correlations among variables (Chakraborty et al. 1992; Yin et al. 2019). Thus, feature extraction of time series variables and exploring the inter-variable correlations are crucial for accurate forecasting. The TFT (Lim et al. 2021) employs RNN for feature extraction from time series variables and incorporates a feature filtering module to identify and utilize valuable information within these features. Aliformer (Qi et al. 2021) categorizes input time-series variables and incorporates future information about these variables, enhancing the model's forecasting capabilities. Crossformer (Zhang and Yan 2022) leverages the attention mechanism to analyze attention-based correlations between variables, offering a nuanced approach to understanding their interplay. It is worth noting that in recent years, with the development of large language models(LLMS) (Zhao et al. 2023) like ChatGPT (Achiam et al. 2023), LLMs can directly or indirectly generate prediction sequences through prompt engineering (Liang et al. 2024; Zhang et al. 2024).

CNN models are constrained by their limited receptive field, hindering their ability to effectively capture long-term time series data (Tang et al. 2020; Gál et al. 2004; Luo et al. 2016). The RNN model, due to the problems of error accumulation, vanishing gradient, and exploding gradient, also cannot handle the long-term time series prediction task well. While Transformer models circumvent these issues, the attention mechanism's computational cost grows quadratically with the series length, rendering the traditional Transformer impractical for long-term forecasting (Li et al. 2019). Therefore, some research is devoted to reducing the computational cost of the attention mechanism, primarily through sparse attention mechanisms or shortening time series length. Related studies based on the sparse attention mechanism include: LogTrans (Li et al. 2019) filters the target of attention computation based on the distance between time steps, reducing computation. Reformer (Kitaev et al. 2020) maps the data into a hash space and filters the target of attention computation based on the distance among time steps in the hash space. Informer (Zhou et al. 2021) filters the target of attention computation based on computational output distribution. Related studies on reducing series length include: Informer utilizes convolutional aggregation of time steps in the receptive field. Convolutional aggregation reduces the input series length of the attention mechanism. PatchTST (Nie et al. 2022) slices long sequences into multiple fixed-size patches, reducing the attention mechanism's computational cost.

Regarding loss function, the above deep learning-based models commonly use Mean Absolute Error (MAE) or Mean Squared Error (MSE) to evaluate the gap between predictions and actual outcomes (Li et al. 2019; Zhou et al. 2021; Kitaev et al. 2020; Zhou et

al. 2022; Oreshkin et al. 2019). However, using MAE or MSE as the sole optimization objective has some limitations (Goodfellow et al. 2014; Mogren 2016; Lyu et al. 2019). In response, several models have innovated upon the traditional loss function to address this issue. For instance, DeepAR (Salinas et al. 2020) converts the model output from specific values to probability distributions, using negative log-likelihood as its loss function. Similarly, SSDNet (Lin et al. 2021) integrates negative log-likelihood with MAE to form a hybrid loss function, allowing for the assessment of discrepancies in both value and probability terms. AST (Wu et al. 2020) employs a strategy inspired by the training mechanism of generative adversarial networks (GANs) (Goodfellow et al. 2014), comprising both discriminant loss and generative loss components. This approach facilitates the alignment of the predicted distribution with the actual data distribution.

Deep learning architectures usually require large-scale labeled datasets for achieving good performance on forecasting time series. Recent techniques of self-supervised learning (Pöppelbaum et al. 2022; Jaiswal et al. 2020) have opened up new a research frontier where deep learning architectures can learn general features from unlabeled time series. The task of self-supervised learning is usually accomplished with some sort of time-series augmentation such as flipping, random noise, time warping, and random smoothing. Based on these data augmentation methods (Wen et al. 2020; Cui 2016) and the strategy of contrastive training (Pöppelbaum et al. 2022; Al-Tahan and Mohsenzadeh 2021), deep learning-based time-series prediction models can capture temporal features during the pre-training phase. Subsequently, with a small amount of labeled data, they train the predictor to carry out predictions. At the same time, recent technology has also leveraged Few-Shot Learning to address the challenge of limited large-scale labeled datasets. This approach involves training the model on various public datasets, enabling it to extract time-series features effectively and exhibit strong generalization capabilities. Such a strategy leads to enhanced performance when applied to new datasets. Methods employing Few-Shot Learning can be categorized into two distinct directions. In Direction 1, the Memory network (Weston et al. 2014) approach is utilized. This involves encoding historical time-series data during training and retrieving these encodings when presented with new time-series datasets. This allows the model to identify similar patterns and make accurate predictions (Iwata and Kumagai 2020). Direction 2 involves using pre-trained LLMs known for their robust generalization abilities to generate precise predictions for time series data (Jin et al. 2023).

1.3 Highlights

The contributions of this paper are as follows:

- This paper provides a systematic review of the evolution of deep learning-based time series forecasting models from multiple perspectives. It thoroughly examines the ability of various models to capture correlations both between time steps and among variables, explores the trade-offs between expanding the model's receptive field and reducing computational costs, and analyzes commonly used loss functions.
- This paper conducts a comprehensive evaluation of the effectiveness of various deep learning-based time series forecasting models in both univariate and multivariate tasks across different domains. Through experiments, we uncover the strengths and weaknesses of different algorithms, assess their ability to capture various types of time series

patterns (including trends and seasonal variations), and discuss strategies to enhance the computational efficiency of these models.
– Finally, this paper discusses future directions in the field of time series forecasting based on the analysis results.The organization of this paper is as follows. Section 2 introduces the background knowledge used in this survey. Section 3 traces the progress of such models from 2014 to 2024, focusing on the logic behind time series information mining, including correlation among the time step and correlation among variables. Section 4 discusses the strategies for reducing computational costs in long-term forecasting. Section 5 summarizes the loss functions utilized in deep learning-based time series forecasting. In Sect. 6, we conduct experiments on the discussed models and methods across several datasets. We demonstrate the advantages and disadvantages of various algorithms from different perspectives, such as prediction accuracy, information extraction capabilities, the ability to identify trend and seasonal patterns, and the impact of different attention mechanisms. In Sect. 7, we summarize the experimental results and discuss the prospective directions for the future development of time series forecasting.

2 Background

In this section, we will briefly introduce the background information essential for this survey.

2.1 Time series forecasting

Given a historical time series $\boldsymbol{X}_{t-L:t} = [\boldsymbol{x}_{t-L}, \cdots, \boldsymbol{x}_{t-1}] \in \mathbb{R}^{D\times L}$, where D is the number of dimensions and L is the length of the historical time series. The future time series to be predicted is $\boldsymbol{X}_{t:t+O} = [\boldsymbol{x}_t, \cdots, \boldsymbol{x}_{t+O-1}] \in \mathbb{R}^{D\times O}$, with O being the forecast horizon. A time series forecasting model can predict the future sequence $\boldsymbol{X}_{t:t+O}$ from the historical sequence $\boldsymbol{X}_{t-L:t}$. Assuming that the predicted sequence is denoted as $\tilde{\boldsymbol{X}}_{t:t+O} \in \mathbb{R}^{D\times O}$ and the prediction model is denoted as $\varphi(\cdot)$, the task of the time series forecasting can be expressed as:

$$\tilde{\boldsymbol{X}}_{t:t+O} = \varphi(\boldsymbol{X}_{t-L:t}).$$

Note that $\boldsymbol{x}_t \in \mathbb{R}^D$ denotes the time step at time t, and the time step correlation refers to the interrelations among these vectors. The time series can also be represented as $\boldsymbol{X} = [\boldsymbol{v}_1^\top; \cdots; \boldsymbol{v}_D^\top]$, where $\boldsymbol{v}_i^\top \in \mathbb{R}^L$ denotes the i-th time-series variable. The correlation of the time-series variable is the vectors among different $\boldsymbol{v}$ vectors.

2.2 Time series decomposition

Time-step dependencies usually consist of various correlations, such as trend and season correlations. To more thoroughly explore these dependencies and simplify the forecasting process, some forecasting models incorporate the concept of time series decomposition, an approach rooted in traditional time series analysis (West 1997; Soltani 2002; Dagum 2010). This analysis posits that any given time series $\boldsymbol{X}$ is composed of distinct components: a trend

term ($\boldsymbol{T}$), indicative of the series' long-term progression; a seasonal term ($\boldsymbol{S}$), representing systematic and predictable fluctuations tied to seasonal effects; a recurrent fluctuation term ($\boldsymbol{C}$), denoting periodic variations within a specific period of time; and an irregular fluctuation term ($\boldsymbol{N}$), which accounts for random variability in the data. Decomposition models fall into two primary categories: additive and multiplicative. The additive model represents the time series as the sum of the four components, i.e., $\boldsymbol{X} = \boldsymbol{T} + \boldsymbol{S} + \boldsymbol{C} + \boldsymbol{N}$. While, for the multiplicative model, an arbitrary time series can be expressed as $\boldsymbol{X} = \boldsymbol{T} * \boldsymbol{S} * \boldsymbol{C} * \boldsymbol{N}$.

3 The logic of time series information mining

In this section, we will review the development of models over the past decade, with the processing logic of time series as the main storyline. Generally, the processing logic of models for time series can be categorized into two types: mining correlations among time steps and mining correlations among variables.

3.1 Mining correlations among time steps

In this subsection, we introduce the models and methods used to extract correlations among time steps. Specifically, these approaches can be categorized into two main types: holistic mining and targeted information mining, such as extracting trend or seasonal patterns.

3.1.1 Holistic mining

Autoencoders (Lv et al. 2014) are effective in uncovering time-step dependencies in time series. They typically consist of three components: an encoder, a decoder, and a prediction module, with the workflow divided into pretraining and prediction phases. During the pretraining phase, the encoder extracts features that are fed into the decoder, which then reconstructs the input sequence to train the model parameters. In the prediction phase, the extracted features are input into the predictor, generating the forecasted sequence. In contrast, DLinear (Yun et al. 2019) uses a single linear layer to map the input vector's dimension directly to the desired output dimension. Due to the simplicity of autoencoders and linear layers, they are generally considered inadequate for fully capturing time-step dependencies in time series.

Given the success of CNNs in the image processing domain (He et al. 2016; Krizhevsky et al. 2017; Szegedy et al. 2015), they are also frequently utilized to identify temporal features in time series (Markova 2022; Lu et al. 2020; Zhao et al. 2017). Through convolution and pooling operations, CNNs aggregate temporal information within their receptive field to uncover internal time-step dependencies. However, due to the inherently limited receptive field, CNNs primarily extract short-term and discontinuous dependencies. To broaden the receptive field of CNNs and uncover dependencies across a wider range, Temporal Convolutional Networks (TCN) incorporate Dilated Causal Convolution (DCC) into the CNN architecture. Dilated convolution extends the receptive field without altering the convolution kernel's size (Yu and Koltun 2015; Yazdanbakhsh and Dick 2019), allowing CNNs to capture more extensive time-step relationships. Furthermore, TCN enhances dilated convolution with temporal causality, ensuring that each time step's output is only influ-

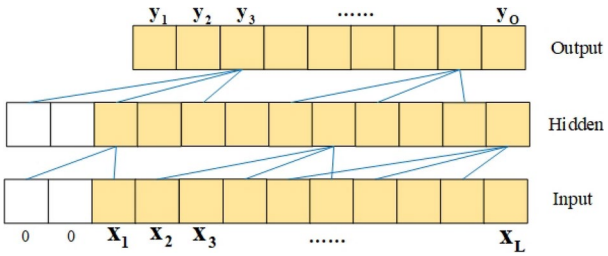

Fig. 2 Dilated casual convolution

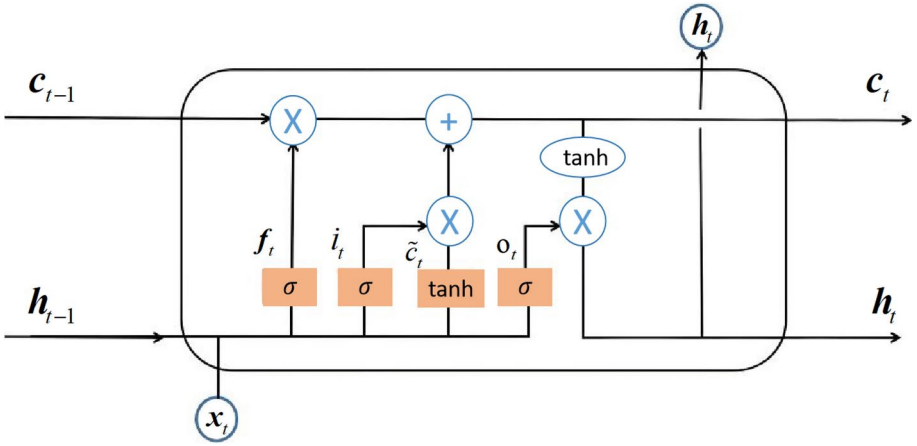

Fig. 3 The structure of LSTM

enced by preceding inputs. The operational intricacies of DCC, TCN's central component, are depicted in Fig. 2, showcasing how dilated causal convolution effectively satisfies the demands of temporal sequence analysis.

Meanwhile, in order to capture time-step dependencies, researchers have proposed Recurrent Neural Networks (RNNs) for time series forecasting, drawing upon the distinctive characteristics of time-series data (Shelatkar et al. 2020; Amalou et al. 2022; Tang et al. 2021). RNNs, utilizing their recurrent connections, enable the features of a preceding time step to act as inputs for the current step, thereby encapsulating the temporal attributes of each step with information from its antecedents. Unconstrained by the receptive field, RNNs theoretically can capture more extensive time-step correlations. However, the increased span of these recurrent connections may lead to problems like gradient explosion and vanishing. To counter these issues, Long Short-Term Memory (LSTM) (Shi et al. 2015; Fischer and Krauss 2018) units introduce a gate mechanism and cell states within the RNN framework. Cell states hold crucial information for long durations, while the gate mechanism-comprising input, output, and forget gates-regulates the flow of information. The input gate controls the incorporation of current input into the cell state, the forget gate determines the retention of cell memory, and the output gate dictates the utilization of cell information for current output, as depicted in Fig. 3. Thus, for a given input time step $\boldsymbol{x}_t$, LSTM's output can be formulated as follows:

$$\boldsymbol{i}_t = \sigma(\boldsymbol{W}_{xi}\boldsymbol{x}_t + \boldsymbol{W}_{hi}\boldsymbol{h}_{t-1} + \boldsymbol{b}_i), \quad \boldsymbol{f}_t = \sigma(\boldsymbol{W}_{xf}\boldsymbol{x}_t + \boldsymbol{W}_{hf}\boldsymbol{h}_{t-1} + \boldsymbol{b}_f), \tag{1}$$

$$\widetilde{\boldsymbol{c}}_t = \tanh(\boldsymbol{W}_{xc}\boldsymbol{x}_t + \boldsymbol{W}_{hc}\boldsymbol{h}_{t-1} + \boldsymbol{b}_c), \quad \boldsymbol{o}_t = \sigma(\boldsymbol{W}_{xo}\boldsymbol{x}_t + \boldsymbol{W}_{ho}\boldsymbol{h}_{t-1} + \boldsymbol{b}_o), \tag{2}$$

$$\boldsymbol{c}_t = \boldsymbol{f}_t \odot \boldsymbol{c}_{t-1} + \boldsymbol{i}_t \odot \widetilde{\boldsymbol{c}}_t, \quad \boldsymbol{h}_t = \boldsymbol{o}_t \odot \tanh(\boldsymbol{c}_t), \tag{3}$$

where $\boldsymbol{i_t}$, $\boldsymbol{f_t}$, $\boldsymbol{o}_t$ respectively denote the input gate, the forget gate, and the output gate. $\widetilde{\boldsymbol{c}}_t$ is the cell state candidates, $\boldsymbol{c}_t$ is the cell state at the t-th time step, and $\boldsymbol{h}_t$ is the time-step feature extracted at the t-th time step. $\boldsymbol{W}_{xi}$, $\boldsymbol{W}_{hi}$, and $\boldsymbol{b}_i$ denote the weight matrices and bias vectors for the input gates. Similarly, $\boldsymbol{W}_{xf}$, $\boldsymbol{W}_{hf}$, and $\boldsymbol{b}_f$ correspond to the weight matrices and bias vectors for the forget gates. For computing the cell states, $\boldsymbol{W}_{xc}$, $\boldsymbol{W}_{hc}$, and $\boldsymbol{b}_c$ are used. Lastly, $\boldsymbol{W}_{xo}$, $\boldsymbol{W}_{ho}$, and $\boldsymbol{b}_o$ represent the weight matrices and bias vectors for the output gates.

The gated structure allows models to selectively retain information from past time steps along with current features (Weerakody et al. 2021; Lin et al. 2022). This helps mitigate issues like gradient explosion and vanishing gradients and also reduces the impact of noise in historical data. This capability allows LSTM units to extract long-term temporal information more efficiently than traditional RNN models. Gated Recurrent Unit (GRU) simplifies LSTM's gate mechanism, which can be regarded as a simplified version of LSTM and will not be introduced in detail here. For multi-step time series forecasting, RNN-based models typically rely on autoregression for sequential prediction (Maggiolo and Spanakis 2019; Binkowski et al. 2018), using outputs from one step as inputs for the next. However, this autoregressive approach may lead to error accumulation. Furthermore, as the sequence length processed by the model increases, RNNs face amplified risks of gradient explosion and error accumulation, challenges that gate mechanisms do not fully resolve.

Given its exceptional performance in both natural language and image processing domains (Devlin et al. 2018; Vaswani et al. 2017), the Transformer model is frequently employed to analyze general time-series data (Wen et al. 2022; Cai et al. 2020). As depicted in Fig. 4, the Transformer mitigates error accumulation in long-term forecasting by utilizing an encoder-decoder architecture. It encodes the temporal information of historical time series via the encoder and then decodes this information with the decoder to produce predictions. At the heart of the Transformer is the self-attention mechanism, which initially maps the input time series $\boldsymbol{X}$ into query $\boldsymbol{Q}$, key $\boldsymbol{K}$, and value $\boldsymbol{V}$ matrices. Subsequently, it computes a similarity matrix between $\boldsymbol{Q}$ and $\boldsymbol{K}$ and adjusts $\boldsymbol{V}$ accordingly. Therefore, the Attention mechanism's output, $\boldsymbol{O} = \mathrm{Atten}(\boldsymbol{X}, \boldsymbol{X}, \boldsymbol{X})$, is determined by the following process:

$$\boldsymbol{Q} = \boldsymbol{W}_Q^\top \boldsymbol{X}, \quad \boldsymbol{K} = \boldsymbol{W}_K^\top \boldsymbol{X}, \quad \boldsymbol{V} = \boldsymbol{W}_V^\top \boldsymbol{X}, \tag{4}$$

$$\boldsymbol{A} = \boldsymbol{Q}^\top \boldsymbol{K}, \quad \boldsymbol{O} = \boldsymbol{V}\mathrm{Softmax}(\frac{\boldsymbol{A}}{\sqrt{D_k}}), \tag{5}$$

where $\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V} \in \mathbb{R}^{D_k \times L}$. The $\boldsymbol{A} \in \mathbb{R}^{L \times L}$ represents the similarity between the L time steps of the input time series, and $\mathrm{Softmax}(\cdot)$ transforms the similarity into an attention weight distribution. The value matrix $\boldsymbol{V}$ is then weighted and summed based on the atten-

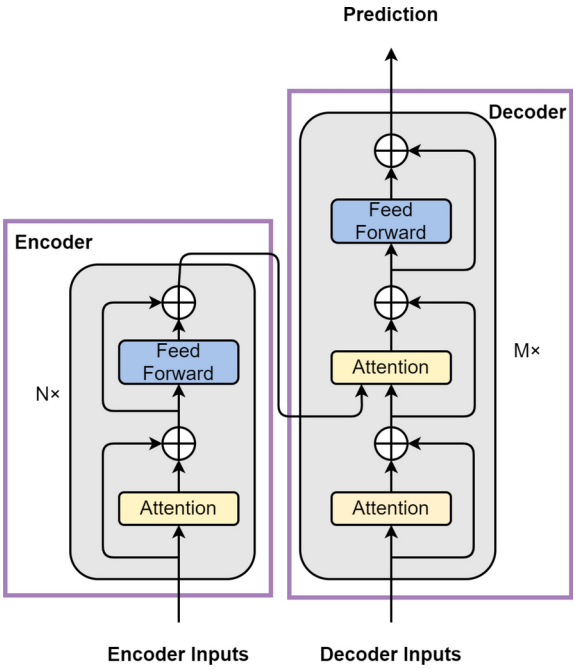

Fig. 4 The structure of Transformer

Table 1 Deep learning time series prediction models using time series decomposition

Time series information	Implications of the information	Key methods
Trend Information	Information on apparent persistence and regularity of change between time steps	Autoformer (Wu et al. 2021), Fedformer (Zhou et al. 2022), LSTnet (Lai et al. 2018), TDformer (Zhang et al. 2022), DLinear (Yun et al. 2019), N-BEATS (Oreshkin et al. 2019), ETSformer (Woo et al. 2022), PatchTST (Nie et al. 2022)
Seasonal information	Periodic information with fixed time intervals between time steps	LSTnet (Lai et al. 2018), Autoformer (Wu et al. 2021), Fedformer (Zhou et al. 2022), ETSformer (Woo et al. 2022), TDformer (Zhang et al. 2022), N-BEATS (Oreshkin et al. 2019)
Multi time scale information	Time-series information at different time scales	Scaleformer (Shabani et al. 2022), Pyraformer (Liu et al. 2021)
Non-stationary information	Information on the statistical properties of the time series over time	RevIN (Kim et al. 2021), NS-Transformer (Liu et al. 2022)

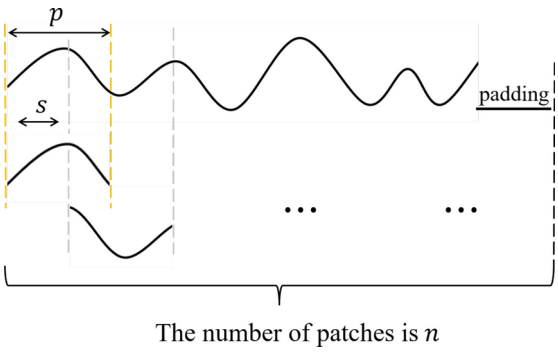

Fig. 5 The slice of the patch, where p is the length of the sliding window, and s is the step size of the window

tion weight distribution to obtain the time-series feature O . In this way, O incorporates the similarity correlations between time steps.

In Fig. 4, the encoder layer of the Transformer encodes the temporal dependencies within the input time series, subsequently feeding the encoded data to the corresponding decoder layer. This decoder layer then generates the forecasted sequence, leveraging the identified temporal dependencies. Besides, the feed-forward module within both the encoder and decoder consists of several fully connected layers. With multiple layers of encoders and decoders stacked, the Transformer architecture can capture extensive long-term temporal correlations. The core of Transformer in time series forecasting lies in storing the similarity relationships between historical time steps and predicting future time steps based on these relationships. It is worth noting that existing large language models(LLM) (Zhao et al. 2023) like ChatGPT (Achiam et al. 2023) can be directly used for time series forecasting. Currently, large language models are commonly involved in time series forecasting in two ways: (1) as a predictive model (Liang et al. 2024): directly achieving sequential prediction based on prompt engineering (Mao et al. 2023; Shin et al. 2020), and (2) not as a predictive model (Zhang et al. 2024): generating embeddings by taking in relevant text information and using these embeddings as auxiliary variables input into the predictive model.

3.1.2 Targeted information mining

To further explore the correlations among time steps, some models leverage the concept of time series decomposition by developing custom modules designed to extract targeted information, such as trend or seasonal information. The final predictive output is then obtained by integrating the forecasting results of various types of target information. Current deep learning models that employ time series decomposition are listed in Table 1.

Trend Information Trend information represents the trend correlations exhibited by the series across time steps, where the term "trend correlations" denotes the discernible persistence and regularity of fluctuations between consecutive time steps within a defined scope (Taylor and Letham 2018; Cleveland et al. 1990; Asadi and Regan 2020). In addressing the attributes of trend information, Autoformer (Wu et al. 2021) employs a mean filter to extract the trend features from the time series. This involves convolving the input time series with a mean filter using a sliding window approach. However, due to the limited receptive field of the mean filter, the trend information acquired through the sliding window method tends to be discontinuous and localized. To mitigate this limitation, Fedformer (Zhou et al. 2022)

devises a set of mean filters with varying receptive fields, yielding trend terms corresponding to different forecasting horizons. Subsequently, the model assigns learnable weights to aggregate the trend terms through weighted averaging. This approach enables Fedformer to incorporate trend information spanning diverse forecasting horizons.

Since trend information is usually more stable on a local scale, the time step to be predicted is more correlated with trend information from closer historical time steps, while data from distant steps are deemed less favorable for trend prediction (Li et al. 2019; Lai et al. 2018). To mitigate the influence of distant data on trend predictions, LSTnet (Lai et al. 2018) incorporates a predefined window, confining trend predictions to temporal information within the window nearest to the target prediction. Within this preset window, which includes k time steps, LSTnet employs linear mapping to capture the linear relationships within the sequence, subsequently leveraging autoregression to forecast the trend term. Thus, LSTnet's prediction of the trend term at time t is expressed as:

$$\boldsymbol{y}_t = \sum_{i=0}^{k-1} \boldsymbol{W}_i \boldsymbol{x}_{t-k+i} + \boldsymbol{b}, \tag{6}$$

where $\boldsymbol{y}_t \in \mathbb{R}^D$ denotes the prediction of the trend term at time t, $\boldsymbol{x}_{t-k+i} \in \mathbb{R}^D$ denotes historical value of the time series at the $(t-k+i)$th time step, $\boldsymbol{W}_i \in \mathbb{R}^{D\times D}$ is the weight matrix and $\boldsymbol{b} \in \mathbb{R}^D$ is the bias. LSTnet employs autoregression to generate trend term predictions for all target locations. Similarly, both TDformer (Zhang et al. 2022) and DLinear (Yun et al. 2019) adopt linear mapping techniques for trend term prediction.However, linear relationships inadequately characterize the trend information of time steps, leading to weak interpretability of trend terms obtained through linear mapping. To address these issues, N-BEATS (Oreshkin et al. 2019) introduces the Trend Stack module, aimed at capturing the trend information within time series data. This module primarily employs polynomial fitting techniques to model the trend relationships between time steps. For the input sequence $\mathbf{X}$, the Trend Stack module initially employs a fully connected layer to extract time-series features, obtaining parameters for polynomial fitting. Subsequently, it introduces the time vector $\boldsymbol{t} = \frac{[0,1,2,\cdots,(O-1)]}{O} \in \mathbb{R}^O$ as the independent variable for the fitting process. The resulting trend term generated by the Trend Stack can be represented as:

$$\boldsymbol{\theta} = \boldsymbol{X}\boldsymbol{W} + \boldsymbol{B}, \quad \boldsymbol{X}_T = \sum_{i=0}^{h} \boldsymbol{\theta}_i \boldsymbol{t}^i, \tag{7}$$

where $\boldsymbol{\theta} \in \mathbb{R}^{D\times h}$ is the fit factor for polynomial fitting, $\boldsymbol{W} \in \mathbb{R}^{L\times h}$ is the weight matrix, and $\boldsymbol{B} \in \mathbb{R}^{D\times h}$ is the bias. $\boldsymbol{X}_T \in \mathbb{R}^{D\times O}$ is the trend term generated by the Trend Stack. h is the highest power in the polynomial fit, which is usually set to a small value $(0 < h < 5)$ by N-BEATS to prevent overfitting.While LSTnet mitigates the influence of distant information on trend term prediction by implementing a preset window, it maintains uniform attention across time steps within this window. In contrast, ETSformer (Woo et al. 2022) posits that data proximate to the prediction position holds greater significance in encoding trend terms, prioritizing time steps in close proximity to the prediction location.

Exponentially weighted average enables the prediction of trend terms to pay more attention to those closer historical time steps (Hyndman et al. 2008; Ensafi et al. 2022), strengthening correlations among short-term time steps. ETSformer introduces the Exponential Smoothing Attention (ESA) module, integrating the concept of exponential weighted averaging into the attention mechanism. For the output $\boldsymbol{O}$ of the attention mechanism derived from Equation (5), the operation of ESA can be expressed as:

$$\hat{\boldsymbol{O}}_t = \alpha \boldsymbol{O}_t + (1-\alpha)\hat{\boldsymbol{O}}_{t-1} = \sum_{j=0}^{t-1} \alpha(1-\alpha)^j \boldsymbol{O}_{t-j} + (1-\alpha)^t v_0, \tag{8}$$

where $0 < \alpha < 1$ is the learnable smoothing parameter and v_0 denotes the learnable initial state.The aforementioned models primarily capture the short-term trends within the time series, inadvertently overlooking the influence of long-term trend information. To address this limitation, PatchTST (Nie et al. 2022) draws inspiration from patch processing methodologies prevalent in computer vision (Dosovitskiy et al. 2020; Liu et al. 2021), integrating the concept of patches into the attention mechanism. As depicted in Fig. 5, PatchTST initially segments the input time series into fixed-length patches using a sliding window approach. Subsequently, these patches serve as inputs to the attention mechanism. Unlike traditional attention mechanisms, PatchTST does not focus on correlations among time steps within the individual patch; instead, it treats each patch as a unit and employs the attention mechanism to assess the similarity between patches. These patches contain aggregated information from short-term time steps, facilitating the exploration of long-term dependencies within the time series. For the input sequence $\boldsymbol{X}$, this process can be represented as:

$$\boldsymbol{X}_p = \text{Patch}(\boldsymbol{X}), \boldsymbol{Q}_p = \boldsymbol{X}_p \boldsymbol{W}_Q + \boldsymbol{b}_Q, \boldsymbol{K}_p = \boldsymbol{X}_p \boldsymbol{W}_K + \boldsymbol{b}_K, \boldsymbol{V}_p = \boldsymbol{X}_p \boldsymbol{W}_V + \boldsymbol{b}_V, \tag{9}$$

$$\boldsymbol{A}_p = \boldsymbol{Q}_p \boldsymbol{K}_p^\top, \quad \boldsymbol{O}_p = \boldsymbol{V}_p \text{Softmax}(\frac{\boldsymbol{A}_p}{\sqrt{D_k}}), \tag{10}$$

where $\boldsymbol{X}_p \in \mathbb{R}^{D \times n \times p}$ is the set of sliced patches, n denotes the number of sliced patches, and p denotes the length of each individual patch. $\boldsymbol{W}_Q, \boldsymbol{W}_K, \boldsymbol{W}_V \in \mathbb{R}^{D \times p \times h}$ is the weight matrix, and h is the dimension of hidden layer. $\boldsymbol{A}_p \in \mathbb{R}^{D \times n \times n}$ is the attention matrix, which represents the similarity relationship between n patches.*Seasonal Information*: Seasonal information is very important in time series analysis (Edition et al. 2002; Rawat et al. 2019), as it indicates the periodic behavior within the data, elucidating correlations between time steps separated by fixed intervals. However, extracting seasonal information is often more challenging than trend information due to the intricate periodicity inherent in time series data. Approaches for predicting seasonal terms can be broadly categorized into two groups: time domain-based methods and frequency domain-based methods. Time domain-based methods directly model the complex periodicity within the time domain (Lai et al. 2018; Wu et al. 2021). In contrast, frequency domain-based methods first transform the input time series into the frequency domain using a Fourier transform (Zhou et al. 2022; Woo et al. 2022; Zhang et al. 2022), then identify complex periodicities based on frequency components. In the following, we will delve into the seasonal terms mining approaches in both the time and frequency domains.

To enable RNNs to capture seasonal information within time series data, LSTnet (Lai et al. 2018) introduces a recurrent-skip network that contains skip connections. Incorporating skip connections facilitates direct interaction between time steps separated by a predefined interval. As depicted in Fig. 6, LSTnet initially assumes a potential period p for the input sequence. Subsequently, connections are established between the tth time step and the $(t-p)$th time step within the RNN. The update of the hidden layer state at the tth time step encompasses information from the $(t-p)$th time step, the current input, and the hidden layer state from the previous time step. The skip-connected RNN allows the state update at each time step to fully consider the information of the time steps before a fixed time interval.

The skip-connected RNN necessitates preset timing periods p and can only be utilized to extract a single periodic relation, rendering LSTnet inadequate for analyzing time series exhibiting intricate periodic patterns. To address this limitation and capture the complex periodic relations inherent in time series data, Autoformer (Wu et al. 2021) introduces the Auto-Correction module. This module aims to assess the time-delay similarity among input sequences to characterize the periodicity of the original sequences, where time-delay similarity denotes the likeness between time-delayed sequences and the original ones. Time-delayed sequences are shifted by τ time steps. Autoformer posits that if a time-delayed sequence with a τ-step delay exhibits significant similarity to the original sequence, τ can be considered a period of the original sequence. To capture these complex period relations, as depicted in Fig. 7, the Auto-Correction module initially employs $\mathrm{Roll}(\cdot,\tau)$ to shift the input sequence by a length of τ, subsequently evaluating the similarity between the delayed sequence and the original one. This similarity is then utilized as a weighting factor to average and aggregate the time-delayed sequence, thereby generating the seasonal terms of the sequence. Thus, for the input time series $\mathbf{X}$, the Auto-Correction process can be delineated as:

$$R_{\boldsymbol{X}}(\tau) = \lim_{L\to\infty} \frac{1}{L} \sum_{t=1}^{L} \boldsymbol{x}_t \boldsymbol{x}_{t-\tau}, \quad \tau_1, \cdots, \tau_k = \arg \underset{\tau\in\{1,\cdots,L\}}{TopK}(R_{\boldsymbol{X}}(\tau)), \tag{11}$$

$$\hat{R}_{\boldsymbol{X}}(\tau_1), \cdots, \hat{R}_{\boldsymbol{X}}(\tau_k) = \mathrm{Softmax}(R_{\boldsymbol{X}}(\tau_1), \cdots, R_{\boldsymbol{X}}(\tau_k)), \tag{12}$$

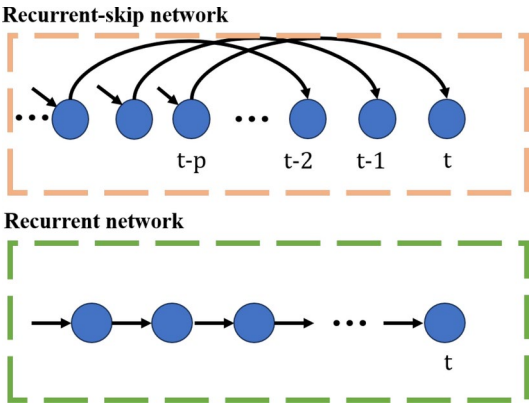

Fig. 6 Recurrent-skip network

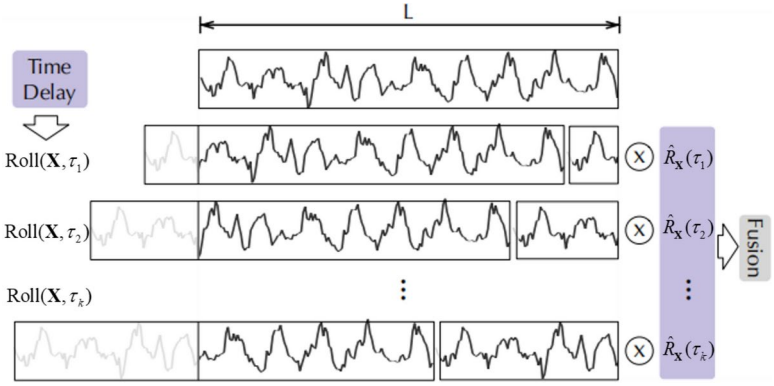

Fig. 7 Auto-Correction in Autoformer

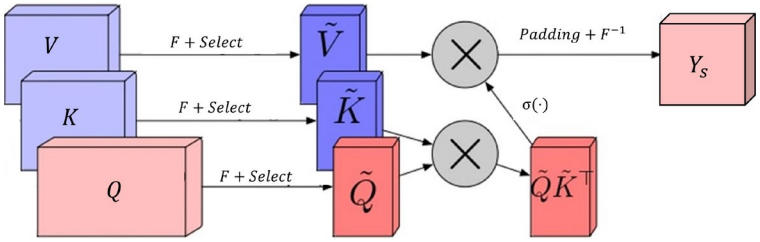

Fig. 8 FEA in Fedformer

$$\text{Auto-Correlation}(\boldsymbol{X}) = \sum_{i=1}^{k} \text{Roll}(\boldsymbol{X}, \tau_i)\hat{R}_{\boldsymbol{X}}(\tau_i), \tag{13}$$

where $\boldsymbol{x}_t$ denotes the tth time step of the input time series, $R_{\boldsymbol{X}}(\tau)$ denotes the similarity between the original sequence $\mathbf{X}$ and its delayed sequence by a time delay of τ time steps. $\tau_1, \cdots, \tau_k$ represent k time delays of the delayed sequence that has the highest similarity to the original sequence. Autoformer leverages the similarity of time delays to extract the periodicity inherent in the input time series. It approximates the complexity of the time series' periods by utilizing multiple time delays of different degrees.

According to the Fourier series (Mathieu et al. 2013; Celeghini et al. 2021), any periodic signal can be expressed as a linear combination of sinusoids with different frequencies. Consequently, frequency domain-based processing involves transforming the time series into the frequency domain via Fourier transform and subsequently analyzing the series in this domain. Frequency domain-based seasonal term mining algorithms are able to analyze the frequency components directly, thereby extracting complex period information from the time series. Subsequently, we will delve into the seasonal term mining algorithm in the frequency domain.

Fedformer (Zhou et al. 2022) adopts an approach by directly assessing the similarity between frequency components within the frequency domain space. Fedformer introduces the FEA (Frequency Enhanced Attention) module, which employs an attention mechanism

to scrutinize frequency components in the frequency domain, thereby delving into the periodic characteristics of the input time series.

In Fig. 8, the FEA module first transforms the input time series from the time domain to the frequency domain using the Fourier transform $F(\cdot)$. To mitigate computational costs in the subsequent Attention mechanism and counteract the effects of noise, FEA employs $\mathrm{Select}(\cdot)$ to randomly sample frequency components in the frequency domain. Subsequently, FEA applies the Attention mechanism to these selected frequency components, deriving similarity relations among them and processing the components accordingly. Finally, FEA returns the processed results to the time domain using the inverse Fourier transform $F^{-1}(\cdot)$. Consequently, the FEA processing, based on the provided description and Equation (5), can be expressed as:

$$\tilde{\boldsymbol{Q}} = \mathrm{Select}(F(\boldsymbol{Q})), \quad \tilde{\boldsymbol{K}} = \mathrm{Select}(F(\boldsymbol{K})), \quad \tilde{\boldsymbol{V}} = \mathrm{Select}(F(\boldsymbol{V})), \tag{14}$$

$$\tilde{\boldsymbol{O}} = \mathrm{Atten}(\tilde{\boldsymbol{Q}}, \tilde{\boldsymbol{K}}, \tilde{\boldsymbol{V}}), \quad \boldsymbol{Y}_S = F^{-1}(\mathrm{Padding}(\tilde{\boldsymbol{O}})), \tag{15}$$

where $\tilde{\boldsymbol{O}} \in \mathbb{C}^{I \times D_k}$, I denotes the number of frequency components in the frequency domain. To fulfill the forecasting task, FEA requires padding the processing results in the frequency domain using $\mathrm{Padding}(\cdot)$ to ensure that the sequence meets length requirements.Since FEA performs a random selection of frequency components in the frequency domain, there's a risk of losing crucial season information. To address this, ETSformer (Woo et al. 2022) filters frequency components based on their magnitudes, retaining the k components with the highest magnitude. For seasonal term processing, ETSformer introduces the FA (Frequency Attention) module. This module first transforms the input time series into the frequency domain, preserving the k frequency components with the greatest magnitude. Subsequently, the FA module returns the retained frequency components to the time domain via Fourier inverse transform and outputs the predicted seasonal terms.

In contrast to Fedformer's FEA module, ETSformer operates under the premise that noise in time series data typically resides in frequency components with lower energy. Hence, it prioritizes frequency components with higher magnitudes. Conversely, Fedformer assumes noise may be present across all frequency components and thus employs random selection for screening. TDformer's (Zhang et al. 2022) approach to seasonal term processing is the same as that of Fedformer. Meanwhile, DLinear (Yun et al. 2019) utilizes a single linear layer to handle the seasonal terms.

To streamline the module for extracting seasonal terms, N-BEATS (Oreshkin et al. 2019) simplifies its approach by exclusively processing specific frequencies within the frequency domain. Specifically, N-BEATS introduces a Season Stack, which utilizes Fourier period coding to analyze the seasonality of the time series. As depicted in Fig. 9, N-BEATS feeds the input time series into the Season Stack. The stack utilizes a fully connected network (FC) to derive Fourier period coding, representing the amplitudes of specific frequencies. Subsequently, leveraging the Fourier period coding alongside the input time vector $\boldsymbol{t} = \frac{[0,1,2,\cdots,(O-1)]}{O} \in \mathbb{R}^O$, the Season Stack employs Fourier series to predict the seasonal terms. Thus, the processing of seasonal terms can be expressed as:

$$\boldsymbol{\theta} = \boldsymbol{X}\boldsymbol{W} + \boldsymbol{B}, \tag{16}$$

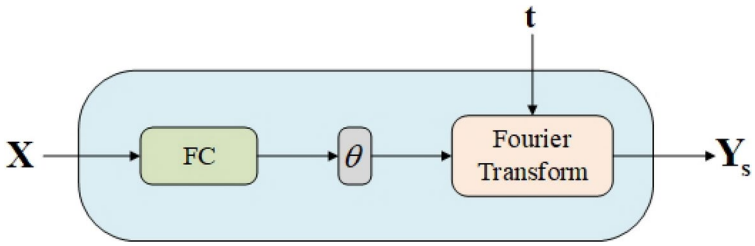

Fig. 9 Season Stack in N-BEATS

$$\boldsymbol{Y}_S = \sum_{i=0}^{\lfloor \frac{h}{2}+1 \rfloor} \boldsymbol{\theta}_i cos(2\pi i \boldsymbol{t}) + \boldsymbol{\theta}_{i+\lfloor \frac{h}{2} \rfloor} sin(2\pi i \boldsymbol{t}), \tag{17}$$

where $\boldsymbol{W} \in \mathbb{R}^{L \times h}$ denotes the weight matrix, $\boldsymbol{\theta} \in \mathbb{R}^{D \times h}$ denotes the Fourier coefficients predicted by an FC network, and h denotes the number of frequency components processed by the season module in the frequency domain. $\boldsymbol{Y}_S \in \mathbb{R}^{D \times O}$ denotes the seasonal terms predicted by the season module. From the equation provided, it's evident that, unlike Fedformer, N-BEATS chooses to process specific frequency components, aiming to simplify the module's task of extracting seasonal information.

Multi Scale Information: In addition to trend and seasonal information, models have begun to target other time-series features such as multi-scale and non-stationary information. Multi-scale information pertains to the temporal dependencies of a time series at different scales (Shabani et al. 2022; Challu et al. 2023). For instance, considering hourly observations as the finest scale, coarser scales could represent daily, weekly, or even monthly patterns within the time series. Integrating multi-scale temporal information enables models to capture dependencies across various durations, crucial for accurate time series modeling and forecasting.

To facilitate the capture of information across diverse time scales, Scaleformer (Shabani et al. 2022) introduces a multi-scale iterative framework for time series forecasting. This framework employs existing models (e.g., Autoformer, Fedformer) as the primary forecasting model and applies filtering operations to transform the input time series into different time scales. As illustrated in Fig. 10, Scaleformer comprises multiple processing layers, with each layer's inputs derived from downsampling the original inputs and upsampling the outputs of the preceding layer. At the ith processing layer, Scaleformer first upsamples the output of the preceding layer $\boldsymbol{X}_{out,i-1}$ to obtain $\boldsymbol{U}_i$. Simultaneously, it downsamples the original input $\boldsymbol{X}$ using a scale factor s_i to obtain $\boldsymbol{P}_i$. These $\boldsymbol{U}_i$ and $\boldsymbol{P}_i$ are then normalized across scales and fed into the forecasting model. The primary forecasting model produces the processing result $\boldsymbol{X}_{out,i}$ for the current layer, and the output of the final processing layer yields the ultimate prediction $\boldsymbol{Y}$.

Since Scaleformer needs to get the prediction in different time scale iterations, the distribution of the model's intermediate variables can change dramatically, which may lead to the accumulation and propagation of erroneous distributional information. In order to eliminate the effect of time scale changes, Scaleformer will perform cross-scale normalization operations on inputs $\boldsymbol{U}_i$ and $\boldsymbol{P}_i$ at each processing layer. For $\boldsymbol{U}_i \in \mathbb{R}^{D \times L_n}$ and $\boldsymbol{P}_i \in \mathbb{R}^{D \times L_m}$, the cross-scale normalization operation can be expressed as:

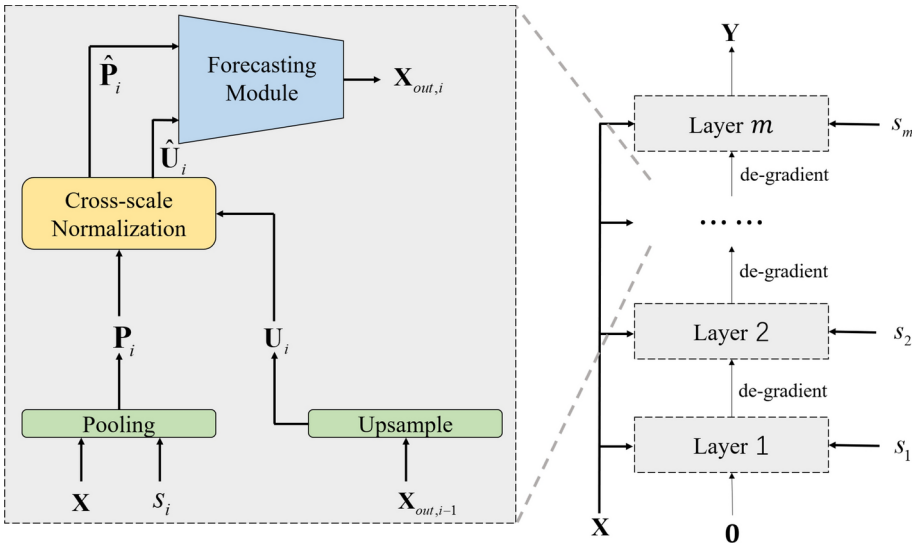

Fig. 10 The structure of Scaleformer

$$\boldsymbol{\mu}_i = \frac{1}{n+m}\left(\sum_{t=1}^{m} \boldsymbol{P}_{i,t} + \sum_{t=1}^{n} \boldsymbol{U}_{i,t}\right), \quad \hat{\boldsymbol{P}}_i = \boldsymbol{P}_i - \boldsymbol{\mu}_i, \quad \hat{\boldsymbol{U}}_i = \boldsymbol{U}_i - \boldsymbol{\mu}_i, \tag{18}$$

where $\boldsymbol{\mu}_i$ is the cross-scale mean coefficient of the ith layer, which is the mean of $\boldsymbol{U}_i$ and $\boldsymbol{P}_i$. The cross-scale normalization operation is essentially a "zero-mean" operation on $\boldsymbol{U}_i$ and $\boldsymbol{P}_i$. Scaleformer enhances the primary prediction model's ability to learn multi-scale time-series information by feeding time series of different time scales into the primary prediction model.To enable the attention mechanism to focus on multi-scale time-series information, Pyraformer (Liu et al. 2021) constructs a pyramid diagram comprising layers representing different time scales. Initially, Pyraformer processes the input time series through convolution, aggregating time steps within a receptive field to generate coarser time scales. As illustrated in Fig. 11, Pyraformer utilizes the original input time series as the bottom layer of the pyramid diagram to represent the finest time scales. Subsequently, it iteratively aggregates the bottom time steps via convolution, using the resultant coarse-scale time series as the top layer of the pyramid graph. Pyraformer employs the Pyramidal Attention Module (PAM) for nodes within the pyramid graph, enabling extraction of cross-time scale features. Finally, depending on specific prediction task requirements, Pyraformer feeds the features of the top nodes into different networks to obtain the final output. Hereafter, we delve into the principles of PAM.

In PAM, each node is only concerned with its adjacent nodes. Considering the pyramid structure depicted in Fig. 11, let $n_l^{(s)} \in \mathbb{R}^D$ denote the lth node in the sth layer of the pyramid graph. The node $n_l^{(s)}$ can obtain the set $\mathbb{N}_l^{(s)}$ of its neighboring nodes in adjacent layers. This set encompasses three components: (1) neighboring A nodes within the same layer, including the $n_l^{(s)}$ node itself, denoted as $\mathbb{A}_l^{(s)}$; (2) the lower layer nodes, representing finer time scale nodes, denoted as $\mathbb{C}_l^{(s)}$; and (3) the upper layer nodes, representing the coarser time scale node, denoted as $\mathbb{P}_l^{(s)}$. This set is expressed as follows:

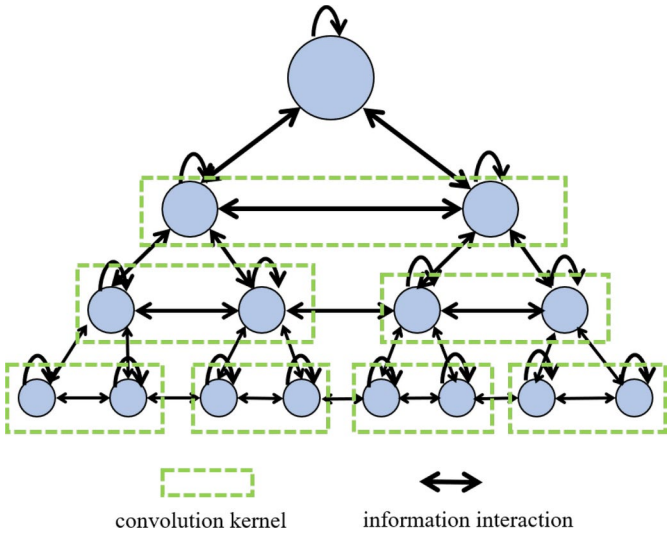

Fig. 11 Pyramid structure in Pyraformer. Different layers have different node sizes, and the node size indicates the granularity of the time scale

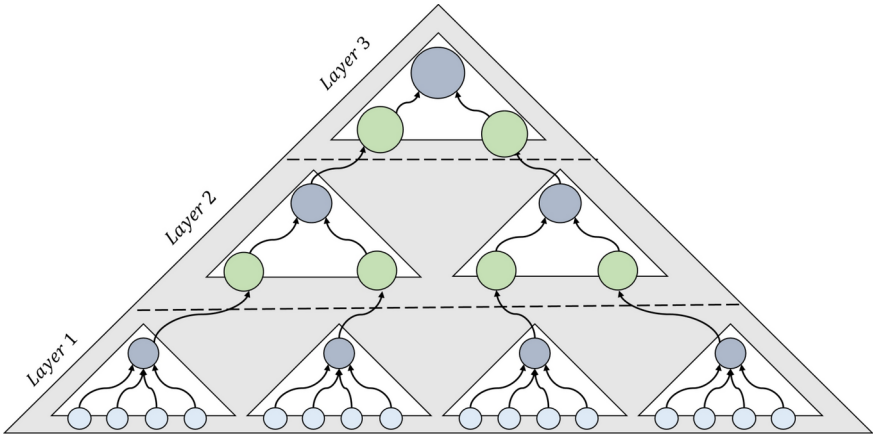

Fig. 12 Triformer. Node size indicates the granularity of the time scale

$$\mathbb{N}_l^{(s)} = \mathbb{A}_l^{(s)} \cup \mathbb{C}_l^{(s)} \cup \mathbb{P}_l^{(s)}, \tag{19}$$

$$\mathbb{A}_l^{(s)} = \{n_j^{(s)} : |j-l| \leq \frac{A-1}{2}, 1 \leq j \leq \frac{L}{C^{s-1}}\}, \tag{20}$$

$$\mathbb{C}_l^{(s)} = \{n_j^{(s-1)} : |l-1|\, C < j \leq lC\}, \tag{21}$$

$$\mathbb{P}_l^{(s)} = \{n_j^{(s+1)} : j = \left\lceil \frac{l}{C} \right\rceil\}, \tag{22}$$

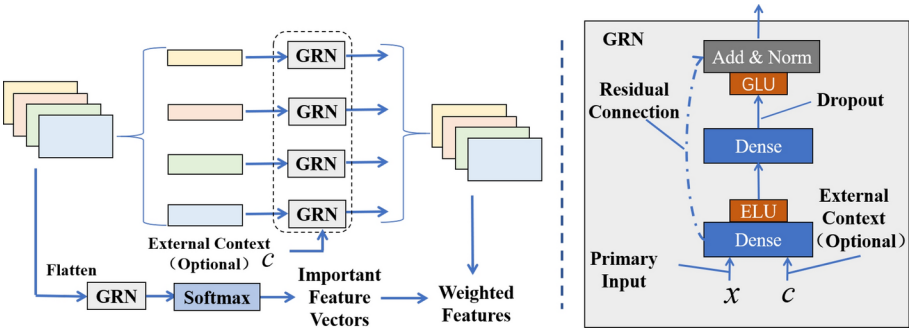

Fig. 13 Left: The VSN structure in the TFT; Right: the GRN module in VSN

Fig. 14 AliAttention in Aliformer

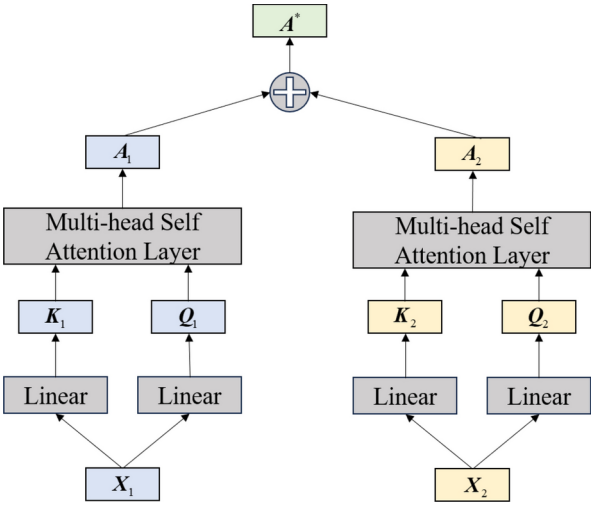

Table 2 The space complexity of different Attention modules, where L denotes the input sequence length

Models for optimizing complexity	Space complexity of attention mechanism
Transformer (Vaswani et al. 2017)	$O(L^2)$
LogTrans (Li et al. 2019)	$O(L(\log L)^2)$
Reformer (Kitaev et al. 2020)	$O(L \log L)$
Informer (Zhou et al. 2021)	$O(L \ln L)$
PatchTST (Nie et al. 2022)	$O(n^2), n = \lfloor \frac{L-p}{s} \rfloor + 2$[a]
Pyraformer (Liu et al. 2021)	$O(\sqrt[S-1]{L})$[b]
Crossformer (Zhang and Yan 2022)	$O(L)$

[a] p denotes the length of the patch and s denotes the stride

[b] S is the number of time scales

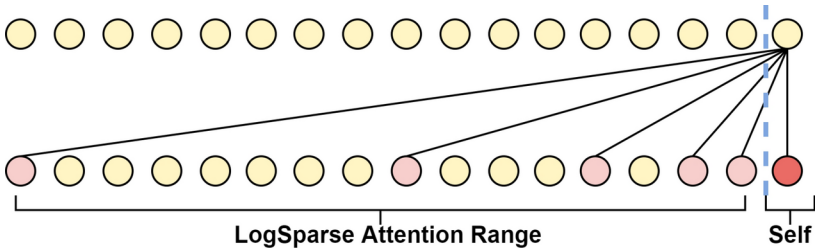

Fig. 15 LogSparse Self-Attention. The connecting line indicates the need to compute the similarity relationship

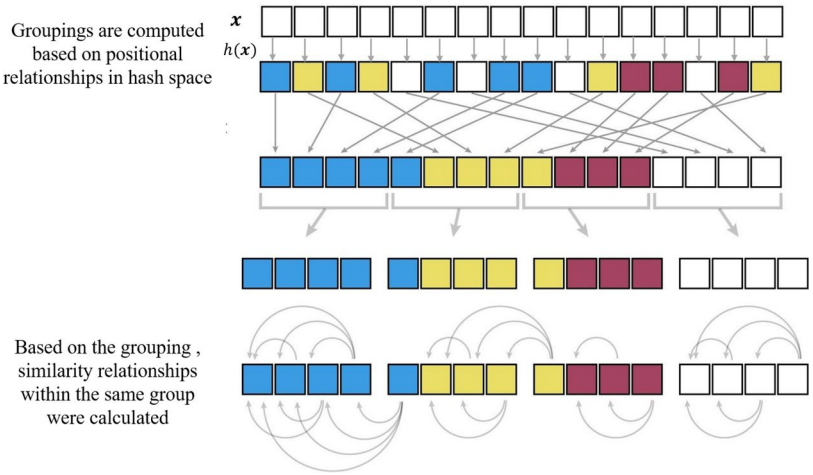

Fig. 16 LSH Attention. Different colors indicate different groups

Table 3 Commonly used time series datasets in time series prediction

Datasets	Timesteps	Variates	Granularity
ETTm1 &ETTm2[a]	69680	7	5 min
ETTh1 &ETTh2[a]	17420	7	1 h
Elctricity[b]	26304	321	1 h
Exchange[c]	7588	8	1 day
Traffic[d]	17544	862	1 h
ILI [e]	966	7	1 week

[a]https://github.com/zhouhaoyi/ETDataset

[b]https://archive.ics.uci.edu/ml/datasets/ElectricityLoadDiagrams20112014

[c]https://github.com/laiguokun/multivariate-time-series-data

[d]http://pems.dot.ca.gov

[e]https://gis.cdc.gov/grasp/fluview/fluportaldashboard.html

where L is the sequence length of the original input to the model, while A denotes a hyperparameter indicating the number of nodes selected at the same time scale. C serves as another hyperparameter designed to regulate the number of ensemble nodes. To ensure the model focuses on multi-scale time-series information, PAM lets nodes $n_l^{(s)}$ compute attention correlations within the set $\mathbb{N}_l^{(s)}$:

$$\boldsymbol{y} = \sum_{i \in \mathbb{N}_l^{(s)}} \frac{\exp\left(\boldsymbol{q}\boldsymbol{k}_i^T / \sqrt{d_K}\right) \boldsymbol{v}_i}{\sum_{i \in \mathbb{N}_l^{(s)}} \exp\left(\boldsymbol{q}\boldsymbol{k}_i^T / \sqrt{d_K}\right)}, \tag{23}$$

where $\boldsymbol{q}$ is the query vector generated by node $n_l^{(s)}$, and $\boldsymbol{k}_i$, $\boldsymbol{v}_i$ are key-value pairs generated by nodes in the set $\mathbb{N}_l^{(s)}$. Pyraformer aggregates time steps of fine time scales into coarse time scales through convolutional operations and captures similarity correlations across different time scales using the Attention mechanism.Unlike Pyraformer, Triformer (Cirstea et al. 2022) generates coarse time-scale time steps through the Attention mechanism and takes fine time-scale sequences as inputs when processing coarse time-scale sequences. As shown in Fig. 12, the Triformer consists of multiple processing layers stacked on top of each other, each of which corresponds to a different time scale. The results from the fine time-scale processing in the lower layers are fed as inputs to the coarser time-scale processing layers. The outputs from all layers are integrated into Triformer's prediction process. This mechanism enables Triformer to effectively explore multi-scale time series information. Next, we will delve into the underlying principles of each processing layer in Triformer. Triformer initially divides the input sequence into multiple local short sequences and applies the Attention mechanism within these segments. For a local short sequence $\boldsymbol{X}_p \in \mathbb{R}^{D \times p}$ in this layer, where p represents the length of the local short sequence. The output of the Attention mechanism in Triformer can be expressed as:

$$\boldsymbol{o}_p = \mathrm{Atten}(\boldsymbol{h}, \boldsymbol{X}_p, \boldsymbol{X}_p), \tag{24}$$

where $\boldsymbol{o}_p \in \mathbb{R}^D$ is the temporal feature vector at a short-term interval, corresponding to the coarser time scale. Meanwhile, $\boldsymbol{h} \in \mathbb{R}^D$ serves as the query vector, capturing the temporal information at the current scale. So $\boldsymbol{h}$ necessitates re-initialization for each processing layer accommodating distinct time scales. It is worth noting that $\boldsymbol{o}_p$ will be used as the time step in the input sequence of the next processing layer.

Non-stationary Information: Non-stationary information refers to variations in the statistical characteristics of a time series over time (Ulyanov et al. 2016; Du et al. 2021; Muandet et al. 2013; Zhong and Cambria 2023). The presence of non-stationary time series poses challenges for prediction tasks in deep learning models, as they complicate the modeling of sequences of statistically changing data during inference, typically manifested through shifts in mean and standard deviation. RevIN (Kim et al. 2021) addresses this issue by standardizing input sequences within the model, ensuring each processed sequence adheres to a consistent distribution, and subsequently reverting the model's predicted sequences to the original distribution during the output stage. Hereafter, we delve into a detailed exposition of RevIN.

RevIN is a flexible end-to-end method that can be applied to arbitrary models (Li et al. 2023; Liu et al. 2024, 2022). To mitigate non-stationary effects in input time series, it

initially normalizes each sequence within a defined lookback window. Unlike conventional normalization techniques applied during time series preprocessing, RevIN normalizes input sequences within the lookback window utilizing learnable affine parameters. The normalized sequences are then fed into the forecasting model to generate predictions. During the output stage, RevIN reverses the normalization process to restore the non-stationary characteristics of the time series. Formally, for an input time series $\boldsymbol{X}$, RevIN's processing can be represented as:

$$\boldsymbol{m} = \frac{1}{L}\sum_{j=1}^{L}\boldsymbol{X}_j, \quad \boldsymbol{v} = \frac{1}{L}\sum_{j=1}^{L}(\boldsymbol{X}_j - \boldsymbol{m})^2, \quad \hat{\boldsymbol{X}} = \boldsymbol{\gamma}\left(\frac{\boldsymbol{X} - \boldsymbol{m}}{\sqrt{\boldsymbol{v} + \varepsilon}}\right) + \boldsymbol{\beta}, \tag{25}$$

where $\boldsymbol{m}, \boldsymbol{v} \in \mathbb{R}^D$ denotes the mean and variance of the time series in the lookback window, and $\boldsymbol{\gamma}, \boldsymbol{\beta} \in \mathbb{R}^D$ denotes the affine coefficients. Then $\hat{\boldsymbol{X}}$ is fed into an arbitrary model to get the prediction $\boldsymbol{Y}$. To restore the non-stationary information of the time series, RevIN denormalizes the prediction:

$$\hat{\boldsymbol{Y}} = \sqrt{\boldsymbol{v} + \varepsilon}\left(\frac{\boldsymbol{Y} - \boldsymbol{\beta}}{\boldsymbol{\gamma}}\right) + \boldsymbol{m}. \tag{26}$$

Non-stationary information within time series poses a significant challenge to accurate forecasting, it is widely acknowledged that preprocessing methods such as those discussed in (Kim et al. 2021; Liu et al. 2022; Passalis et al. 2019) can help mitigate the non-stationary nature of the original input series. However, removing the inherent non-stationarity of sequences may lead to the problem of over-stationarization, which in turn can subject forecasting models to severe overfitting. To tackle this issue, NS-Transformer (Liu et al. 2022) integrates non-stationary information into the Attention mechanism, compensating for information lost during the normalization of the sequence.NS-Transformer adopts the standard Transformer architecture but incorporates De-stationary Attention in place of conventional Attention mechanisms. De-stationary Attention leverages statistical information, specifically mean and variance derived from instance normalization, enhancing the model's ability to capture non-stationary patterns within the original time series. Since NS-Transformer assumes that the time series have the same variance in all variables, $\boldsymbol{v}$ is simplified to the scalar v. Given the stationarized input sequence $\hat{\boldsymbol{X}}$, linear mappings yield $\hat{\boldsymbol{Q}}$, $\hat{\boldsymbol{K}}$, $\hat{\boldsymbol{V}}$, alongside statistical parameters $\boldsymbol{m}$ and v from the original sequence. The computation process of De-stationary Attention is as follows:

$$\hat{\boldsymbol{A}} = v\hat{\boldsymbol{Q}}^{\top}\hat{\boldsymbol{K}} + \mathbf{1}\boldsymbol{m}^{\top}\boldsymbol{K}, \quad \hat{\boldsymbol{O}} = \hat{\boldsymbol{V}}\mathrm{Softmax}\left(\frac{\hat{\boldsymbol{A}}}{\sqrt{D_k}}\right), \tag{27}$$

where $\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}$ are the query matrix, key matrix, and value matrix obtained by linear mapping of the original input sequence. $\boldsymbol{m}$ denotes the mean vector statistically obtained in the normalization, and v denotes the variance statistically obtained in the normalization.In order to directly use deep learning implementation Equation(27), NS-Transformer uses a multi-layer perceptron $\mathrm{MLP}(\cdot)$ to learn non-stationary information from the original input time series:

$$\log \tau = \mathrm{MLP}(v, \boldsymbol{X}), \quad \boldsymbol{\Delta} = \mathrm{MLP}(\boldsymbol{m}, \boldsymbol{X}), , \quad \hat{\boldsymbol{A}} = \tau \hat{\boldsymbol{Q}}^{\top} \hat{\boldsymbol{K}} + \mathbf{1}\boldsymbol{\Delta}^{\top}, \tag{28}$$

where $\tau \in \mathbb{R}$ denotes the variance information learned from the non-stationary information and $\boldsymbol{\Delta} \in \mathbb{R}^{L}$ denotes the mean information learned from the non-stationary information. For the prediction of the Transformer model, NS-Transformer similarly chooses to denormalize to restore the prediction's non-stationary information.

3.2 Mining correlations among variables

In the context of multivariate time series forecasting, each time series dimension represents a distinct univariate time series. In addition to considering time-step dependencies, correlations among variables also play a crucial role in predicting multivariate time series (Cheng et al. 2022; Jin et al. 2022; Cao et al. 2020; Zhang et al. 2023). For example, when predicting future temperature for a given region, relying solely on historical temperature records from the region is insufficient. Instead, including historical wind speed data from the region and historical temperature data from neighboring regions can significantly enhance the accuracy of the prediction. Therefore, in addition to capturing time-step dependencies, an alternative approach to temporal processing is to capture the correlations among variables. In section 3.1, the previously discussed forecasting models mainly concentrate on exploring dependencies among time steps, often overlooking the exploration of correlations among variables. Particularly, PatchTST (Nie et al. 2022), a model incorporating the Channel Independence (CI) strategy, entirely disregards correlations among variables. In the following, we will introduce models that explicitly extract correlations among variables.

To address variables at a more detailed level, TFT (Temporal Fusion Transformers) (Lim et al. 2021) classifies the variables of the input time series into three categories:

1. Static variables: variables remain constant over time.
2. Time-dependent variables: variables exhibit temporal changes and require prediction.
3. Future-known variables: These variables include known future information (e.g., upcoming holiday dates), other exogenous time series (e.g., historical customer foot traffic), and static metadata (e.g., store location).To handle these categorized multiple variables, TFT incorporates a Variable Selection Network (VSN) module for filtering. The structure of VSN is schematically represented in Fig. 13. For different types of input variables, the VSN guides them individually through the Gated Residual Network (GRN) module. Furthermore, the VSN plays a crucial role in regulating the representation for each variable by utilizing weighted averaging, serving as an effective mechanism for variable filtering. Specifically, the formulations of GRN are given by:

$$\boldsymbol{e} = \mathrm{ELU}(\boldsymbol{W}_{x}\boldsymbol{x} + \boldsymbol{W}_{c}\boldsymbol{c} + \boldsymbol{b}_{e}), \tag{29}$$

$$\boldsymbol{d} = \boldsymbol{W}_{d}\boldsymbol{e} + \boldsymbol{b}_{d}, \tag{30}$$

$$\mathrm{GRN}(\boldsymbol{x}, \boldsymbol{c}) = \mathrm{LayerNorm}(\boldsymbol{x} + \mathrm{GLU}(\boldsymbol{d})), \tag{31}$$

where $\boldsymbol{x} \in \mathbb{R}^{L}$ and $\boldsymbol{c} \in \mathbb{R}^{L}$ are univariate time series respectively from the primary input and the external context, $\mathrm{ELU}(\cdot)$ is the ELU activation function, $\mathrm{LayerNorm}(\cdot)$ represents

layer normalization, and $\boldsymbol{W}_x, \boldsymbol{W}_c \in \mathbb{R}^{h\times L}$, $\boldsymbol{W}_d \in \mathbb{R}^{h\times h}$ are learnable weight matrices. Additionally, $\boldsymbol{b}_e, \boldsymbol{b}_d \in \mathbb{R}^h$ are learnable biases, and h denotes the dimension of the preset hidden layer. $\mathrm{GLU}(\cdot)$ refers to the gated linear unit. For the input $\boldsymbol{d}$, the computation of $\mathrm{GLU}(\cdot)$ is as follows:

$$\mathrm{GLU}(\boldsymbol{d}) = (\boldsymbol{d}\boldsymbol{W}_1 + \boldsymbol{b}_1) \otimes \sigma(\boldsymbol{d}\boldsymbol{W}_2 + \boldsymbol{b}_2), \tag{32}$$

where $\boldsymbol{W}_1, \boldsymbol{W}_2 \in \mathbb{R}^{h\times h}$ are learnable weight matrices, and $\boldsymbol{b}_1, \boldsymbol{b}_2 \in \mathbb{R}^h$ are learnable biases. The symbol $\otimes$ represents the Hadamard product, and $\sigma(\cdot)$ denotes the sigmoid activation function.In TFT, the VSN module assigns weights to different variables, which helps suppress certain variables' representation and remove noisy inputs. The VSN only accepts an univariate sequence as input, thus its output weights cannot consider the correlation between different variables. To address this issue, Aliformer (Qi et al. 2021) leverages the Attention mechanism to capture the similarity relationships among variables, which is called AliAttention. This innovative approach enables the fusion of attention graphs generated from different variables, allowing the Attention mechanism to consider the similarity relations among diverse variables simultaneously.

As depicted in Fig. 14, for the input variables $\boldsymbol{X}_1 \in \mathbb{R}^{D_1\times L}$ and $\boldsymbol{X}_2 \in \mathbb{R}^{D_2\times L}$, AliAttention performs separate computations of attention graphs for each variable. These individual attention graphs are then combined to create a unified attention graph. The resulting attention graph is subsequently used to generate the outputs of the Attention mechanism. This process can be summarized as follows:

$$\boldsymbol{A}_1 = \boldsymbol{Q}_1^\top \boldsymbol{K}_1,\ \boldsymbol{A}_2 = \boldsymbol{Q}_2^\top \boldsymbol{K}_2,\ \boldsymbol{A}^* = \boldsymbol{A}_1 + \boldsymbol{A}_2,\ \boldsymbol{O} = \mathrm{Softmax}(\frac{\boldsymbol{A}^*}{2D_k})\boldsymbol{V}_1, \tag{33}$$

In the process, $\boldsymbol{Q}_1, \boldsymbol{K}_1, \boldsymbol{V}_1$ are derived from variable $\boldsymbol{X}_1$, while $\boldsymbol{Q}_2, \boldsymbol{K}_2$ are derived from variable $\boldsymbol{X}_2$. The resulting attention graph, denoted as $\boldsymbol{A}^* \in \mathbb{R}^{L\times L}$, is a newly obtained graph that integrates the similarity matrices of different variables. By incorporating AliAttention, the Attention mechanism can explore the interdependencies among the variables by fusing the attention graphs.To facilitate a comprehensive exploration of correlations among variables, Crossformer (Zhang and Yan 2022) introduces the concept of directly applying the Attention mechanism to the variables instead of fusing attention graphs. For this purpose, Crossformer proposes DAttention. For the input time series $\boldsymbol{X} \in \mathbb{R}^{D\times L}$, Crossformer first transposes the dimension of input series to obtain transposed $\boldsymbol{X}' \in \mathbb{R}^{L\times D}$, and then feeds $\boldsymbol{X}'$ into the Attention mechanism. DAttention, based on Equation (4) and (5), can be mathematically expressed as:

$$\boldsymbol{A} = \boldsymbol{Q}^\top \boldsymbol{K}, \quad \boldsymbol{O} = \boldsymbol{V}\mathrm{Softmax}(\frac{\boldsymbol{A}}{\sqrt{D_k}}), \tag{34}$$

where $\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V} \in \mathbb{R}^{D_k\times D}$ are all computed based on the transposed input $\boldsymbol{X}'$. Here, $\boldsymbol{Q}$ represents the query matrix, $\boldsymbol{K}$ represents the key matrix and $\boldsymbol{V}$ represents the value matrix. Importantly, the attention graph $\boldsymbol{A} \in \mathbb{R}^{D\times D}$ in DAttention captures the similarity relationships between D variables.Indeed, the distinction between AliAttention and DAttention lies in how they utilize the Attention mechanism. AliAttention focuses on capturing similar rela-

tionships of variables by fusing attention graphs derived from each variable. On the other hand, DAttention applies the Attention mechanism directly to the variables, allowing it to uncover the cross-time-step dependence of variables.

4 Long-term time series forecasting optimization

In the field of long-term time series forecasting, Transformer model (Wu et al. 2021; Zhou et al. 2021, 2022) is widely used. This is because Convolutional Neural Networks (CNNs) have limitations regarding their receptive field, and Recurrent Neural Networks (RNNs) suffer from issues like error accumulation and gradient explosion. However, the attention mechanism's computational cost grows quadratically with the series length. To address these challenges, researchers have developed various methods to optimize the attention mechanism for long-term time series prediction, as shown in Table 2.

The computational cost of the attention mechanism mainly arises from computing the similarity relationships between time steps. To optimize the Attention mechanism, two main approaches have been proposed:

(1) *Shortening the length of the input sequence of the Attention mechanism*: This approach aims to reduce the computational complexity by processing shorter subsequences instead of the entire input sequence. Various techniques, such as window-based (Zhou et al. 2021; Cirstea et al. 2022), segment-based (Nie et al. 2022; Liu et al. 2021), and hierarchical methods (Zhang and Yan 2022), have been proposed to divide the input sequence into smaller parts before applying the attention mechanism.
(2) *Sparsifying the attention mechanism*: This approach selectively computes and sparsifies the similarity relationships, reducing the required computations. Different methods like sparse Attention (Li et al. 2019), kernelized Attention (Kitaev et al. 2020), and low-rank Attention (Zhou et al. 2021) have been developed to exploit the sparsity or low-rank structure in the similarity matrix, resulting in computational savings.In the following sections, we will explore these two approaches as a guiding framework to introduce optimization methods for the Attention mechanism in long-term time series prediction.

Shorten the Length of Time Series Processed by Attention Mechanism: Informer (Zhou et al. 2021) utilizes the Transformer framework, which integrates convolution and pooling operations to shorten the length of time series gradually. This sequential aggregation of time steps occurs at each layer. As a result, the computational burden of the Transformer is significantly diminished. Therefore, the input time series of the *j*th layer in Informer can be expressed as follows:

$$\hat{\boldsymbol{X}}_j = \mathrm{MaxPool}\left(\mathrm{ELU}(\mathrm{Conv1d}(\hat{\boldsymbol{X}}_{j-1}))\right), \tag{35}$$

where $\hat{\boldsymbol{X}}_j \in \mathbb{R}^{D \times L_j}$ represents the output time series of the *j*th layer, and L_j denotes its sequence length. The input of the *j*th layer, denoted as $\hat{\boldsymbol{X}}_{j-1} \in \mathbb{R}^{D \times L_{j-1}}$, is worth noting that $L_j < L_{j-1}$.Unlike Informer, PatchTST (Nie et al. 2022) takes a different approach by dividing the original input sequence into multiple fixed-length short sequences, as depicted

in Fig. 5. This decision allows the model to transition from processing a single, lengthy sequence to managing multiple, short sequences. Additionally, PatchTST treats these short sequences as cohesive units and employs its attention mechanism to compute their similarity relationships. As a result, the space complexity of the attention mechanism in PatchTST is reduced from the original $O(L^2)$ to $O(N^2)$, where L represents the length of the input sequence and N denotes the number of units, with $N \ll L$. Consequently, PatchTST achieves significant reductions in space complexity. Other models, such as Pyraformer (Liu et al. 2021) and Triformer (Cirstea et al. 2022), employ a similar strategy to PatchTST in optimizing the space complexity of the attention mechanism. Compared to Informer, PatchTST demonstrates a greater advantage in optimizing space complexity due to its treatment of sliced short sequences during processing. Notably, this advantage becomes more pronounced when handling long-term time series, as the growth rate of N remains relatively slow compared to the progressive increase of L.

Sparse the Attention Mechanism: Sparsing the attention mechanism involves selectively computing the similarity between time steps within the mechanism while preserving the original length of the input sequence. These approaches can also effectively address the space complexity associated with the attention mechanism. Building upon this concept, LogTrans (Li et al. 2019) introduces the LogSparse Self-Attention mechanism. Illustrated in Fig. 15, for the tth time step of the input time sequence, LogSparse Self-Attention determines the index of the time step with which the similarity relationship needs to be computed as follows:

$$\boldsymbol{I}_t = \left\{ t - 2^{\lfloor \log_2 t \rfloor}, t - 2^{\lfloor \log_2 t - 1 \rfloor}, t - 2^{\lfloor \log_2 t - 2 \rfloor}, \cdots, t - 2^0, t \right\}, \tag{36}$$

where $\boldsymbol{I}_t$ represents the index of the time step in the tth time step where similarity computation is required. LogSparse Self-Attention selectively filters similarity relationships based on their temporal proximity to the current time step, prioritizing relationships among nearby time steps. Additionally, LogSparse Self-Attention is causal, meaning that the current time step disregards similar information from future time steps.Unlike LogTrans, which utilizes distance between time steps as a filtering criterion, Reformer (Kitaev et al. 2020) aims to group time steps in the feature space and compute attention within the groups, thereby achieving sparsity in the attention mechanism. The challenge of rapidly finding nearest neighbors in high-dimensional spaces can be addressed by Locality-Sensitive Hashing(LSH). A hashing scheme that assigns each time step to a hash space is called locality-sensitive if nearby vectors get the same hash with high probability and distant ones do not. Based on the LSH, Reformer proposes the LSH Attention mechanism. LSH Attention maps the input time series into a hash space, allowing attention computations only between time steps close to each other within this space.

To map time steps into the hash space, Reformer establishes the hash function as follows: $h(\boldsymbol{x}) = \arg\max\left(\left[\boldsymbol{R}^\top \boldsymbol{x}, -\boldsymbol{R}^\top \boldsymbol{x}\right]\right)$, where $\boldsymbol{R} \in \mathbb{R}^{D \times b/2}$, and b represents the number of hashes, which corresponds to the dimension of the hash space. As depicted in Fig. 16, the time steps within the input time series are mapped to the hash space using this hash function. Based on the positional relationship of the time steps in the hash space, Reformer organizes all time steps into distinct groups. Within each group, each time step computes the similarity relationship exclusively with other time steps belonging to the same group. This selective calculation significantly reduces the computation cost of the attention mechanism.

Informer (Zhou et al. 2021) also incorporates sparsity within its attention mechanism. However, unlike LogTrans and Reformer, Informer adopts a unique filtering criterion for similarity relationships based on the following assumption: within the attention mechanism, if the query vector $\boldsymbol{q}$ possesses intricate temporal information, its attention distribution with respect to all key vectors should deviate significantly from a uniform distribution. Based on this concept, Informer introduces ProbSparse Self-Attention, which selectively retains the computation results of the query according to the score of each query. The scoring function for query vector $\boldsymbol{q}$ can be expressed as follows:

$$M(\boldsymbol{q}, \boldsymbol{K}) = \ln \sum_{j=1}^{L} \exp^{\frac{\boldsymbol{q}\boldsymbol{k}_j^\top}{\sqrt{D_k}}} - \frac{1}{L} \sum_{j=1}^{L} \frac{\boldsymbol{q}\boldsymbol{k}_j^\top}{\sqrt{D_k}}, \tag{37}$$

where $\boldsymbol{K} = [\boldsymbol{k}_1, \cdots, \boldsymbol{k}_L]$ represents the key matrix containing all the key vectors in the time series, and $\boldsymbol{k}_j \in \mathbb{R}^{D_k}$ denotes the key vector corresponding to the jth time step in the time series. The scoring function primarily evaluates the query vectors $\boldsymbol{q}$ by analyzing their similarity distribution relative to the other key vectors $\boldsymbol{k}$. Ultimately, ProbSparse Self-Attention selectively retains the Top-U query vectors.LogTrans, Reformer, and Informer employ Sparse Attention mechanisms that rely on specific filtering criteria for computing similarity relationships. For example, LogTrans filters the target of attention computation based on the distance between time steps. Reformer maps the data into a hash space and filters the target of attention computation based on the distance among time steps in the hash space. Informer filters query vectors based on the distribution of similarities between the query vector and all key vectors. In contrast, Crossformer (Zhang and Yan 2022) adopts a different approach by not employing a specific filtering criterion to sparsify the attention mechanism. Instead, it introduces transition units to avoid direct similarity computations between time steps. This strategy effectively reduces the space complexity of the Attention mechanism.

In the Crossformer model, when processing the input time serok with a length of L using the attention mechanism, c transition units are introduced, where $c \ll L$. These transition units act as intermediate components for performing similarity computations between query vectors and key vectors within the attention mechanism. This approach helps avoid the space complexity of $O(L^2)$ associated with direct similarity computations between query vectors and key vectors. For a key matrix $\mathrm{K} \in \mathbb{R}^{D_k \times L}$ and a value matrix $\mathrm{V} \in \mathbb{R}^{D_k \times L}$, the Attention mechanism with transition units can be expressed as follows:

$$\boldsymbol{O}_1 = \boldsymbol{V}\mathrm{Softmax}(\frac{\boldsymbol{K}^\top \boldsymbol{R}}{\sqrt{D_k}}), \tag{38}$$

$$\boldsymbol{Q}_2 = \boldsymbol{W}_Q \boldsymbol{X}, \boldsymbol{K}_2 = \boldsymbol{W}_K \boldsymbol{O}_1, \boldsymbol{V}_2 = \boldsymbol{W}_V \boldsymbol{O}_1, \tag{39}$$

$$\boldsymbol{O}_2 = \boldsymbol{V}_2 \mathrm{Softmax}(\frac{\boldsymbol{K}_2^\top \boldsymbol{Q}_2}{\sqrt{D_k}}), \tag{40}$$

where each column of $\mathrm{R} \in \mathbb{R}^{D_k \times c}$ represents the transition unit, which are learnable parameter vectors. As evident from the provided equations, the inclusion of these transition units

allows for a reduction in the space complexity of the attention mechanism from $O(L^2)$ to a linear complexity of $O(2cL) = O(L)$.

5 Loss function

According to the optimization objective, the loss function commonly used in time series prediction models can be classified into two main types:

(1) Loss function with a single optimization objective. This type of loss function primarily focuses on optimizing the gap in value between the prediction and the ground truth. The simplicity of this category of loss functions makes it widely employed. However, it has the limitation of solely aiming to minimize the gap in value between the prediction and the ground truth. Consequently, this category of loss functions faces challenges when dealing with time series that contain intricate temporal patterns and exhibit strong fluctuations.
(2) Hybrid loss function that optimizes the multiple objectives. To address the aforementioned issue, an alternative group of models utilizes hybrid loss functions that optimize the multiple objectives. This category of loss functions aims to combine multiple loss functions to optimize different modules of the model, enabling collaborative predictions. In the following sections, we will discuss these two types of loss functions separately.

5.1 Single-objective loss function

MAE, MSE: Mean Absolute Error (MAE) and Mean Square Error (MSE) are commonly utilized loss functions in time series prediction tasks. They are employed to optimize the gap in value between the predicted values and the ground truth. MAE calculates the average of the absolute value of the prediction errors, while MSE calculates the average of the squared prediction errors. The specific formulas are provided below:

$$\mathrm{MAE}(\boldsymbol{Y}, \hat{\boldsymbol{Y}}) = \frac{1}{O} \times \frac{1}{D} \sum_{i=1}^{O} \sum_{j=1}^{D} |y_{i,j} - \hat{y}_{i,j}|, \tag{41}$$

$$\mathrm{MSE}(\boldsymbol{Y}, \hat{\boldsymbol{Y}}) = \frac{1}{O} \times \frac{1}{D} \sum_{i=1}^{O} \sum_{j=1}^{D} (y_{i,j} - \hat{y}_{i,j})^2, \tag{42}$$

where $\boldsymbol{Y} \in \mathbb{R}^{O \times D}$ represents the ground truth, $\hat{\boldsymbol{Y}} \in \mathbb{R}^{O \times D}$ represents the prediction, and O is the length of the prediction and D is the number of variables. From this equation, it becomes evident that when dealing with challenging-to-predict data points, MSE exacerbates the gap in value between the prediction and the ground truth, imposing a more substantial penalty. On the other hand, MAE lacks this characteristic and treats all prediction points uniformly. In time series forecasting tasks, using MAE as the loss function can effectively mitigate the impact of outliers during model training. This approach is particularly beneficial for datasets like electricity and wind power data. Conversely, when the chal-

lenging prediction points in the time series dataset containning significant information, as observed in Traffic datasets (Wu et al. 2021) and Exchange Rate datasets (Lai et al. 2018), the model can opt for MSE as the loss function to emphasize their importance during the training process.In essence, employing MSE or MAE implies a disregard for the direction of the error (Wang and Bovik 2009), meaning we are indifferent to whether the prediction surpasses or falls short of the actual value. However, this characteristic becomes crucial in specific time prediction scenarios. For instance, when forecasting the remaining lifespan of an engine, our prediction must underestimate the true value to prevent potential accidents.

Quantile Loss: Quantile loss is commonly used as a loss function in time series prediction tasks. Unlike MSE and MAE, quantile loss penalizes positive and negative deviations between predicted and true values differently. Given the prediction $\hat{\boldsymbol{Y}}$ and the ground truth $\boldsymbol{Y}$, the quantile loss between them can be expressed as:

$$\mathcal{L} = q * max(0, \boldsymbol{Y} - \hat{\boldsymbol{Y}}) + (1 - q) * max(0, \hat{\boldsymbol{Y}} - \boldsymbol{Y}), \tag{43}$$

where the quantile coefficient $q \in [0, 1]$ can be adjusted to meet the specific demands of the prediction task. For example, in aircraft remaining useful life prediction (Zhang et al. 2023, 2023; Wang et al. 2023), it is often desired for the model's predictions to be lower than the actual lifespan of the aircraft to minimize potential risks. Therefore, a quantile coefficient $q < 0.5$ can be used, which increases the penalty imposed by the loss function on larger predictions. This setting encourages the model to generate smaller predictions. However, a drawback of quantile loss is the requirement to specify a target quantile, which is often impractical in many prediction scenarios.In summary, MAE, MSE, and quantile loss are commonly used loss functions for point forecasts in deep learning models for time series prediction. These loss functions are effective for both univariate and multivariate predictions, as well as for single-step and multi-step forecasting. Among the three, MSE is suitable when prediction errors are expected to follow a Gaussian distribution. However, when a dataset contains numerous outliers, MSE can be heavily affected by these, potentially leading to model failure. In such cases, MAE might be a better choice as the loss function. In specific scenarios, such as Remaining Useful Life (RUL) prediction (Zhang et al. 2023, 2023; Wang et al. 2023), where biased predictions are necessary, quantile loss is often employed.

Beyond point forecasts, interval forecasts (Armstrong 2001) represent another approach in time series prediction. As the name suggests, this method provides a range of possible outcomes rather than a single point, offering insights into the model's confidence or prediction uncertainty. For interval forecasts, deep learning models typically use quantile loss as the loss function. Unlike in point forecasts, when quantile loss is applied to interval forecasts, the quantile parameter q in (43) represents a range rather than a single value. Due to space limitations, this paper does not explore interval forecasting in detail.

5.2 Hybrid loss function

The shortcomings of the aforementioned loss functions lie in their optimization objectives, which solely focus on the numerical distance between predictions and ground truth. Additionally, models based on a single optimization target are often structurally simple and cannot incorporate other structured predictive mechanisms such as SSM (State Space Model)

(Salinas et al. 2020; Lin et al. 2021) or GAN (Generative Adversarial Network) (Wu et al. 2020; Mogren 2016; Goodfellow et al. 2014). Therefore, in order to leverage these specialized structures for collaborative predictions, some models utilize a joint loss function to optimize multiple objectives introduced by different structures within the model.

Negative Log-likelihood: SSDNet (Lin et al. 2021) incorporates MAE alongside the negative log-likelihood as its loss function. This loss function allows for the simultaneous consideration of the numerical gap and distributions between the prediction and the ground truth. By tuning a parameter, SSDNet balances the influences of the MAE and the negative log-likelihood. The SSDNet model comprises the Transformer and the SSM (State Space Model) network. The SSM network is a mathematical framework that describes the relationship between latent states and observable data in a time series.

SSMs (Durbin and Koopman 2012; Hyndman 2008; Seeger et al. 2016) enable the incorporation of structural assumptions into the model, making them particularly well-suited for scenarios where the time series structure is well-understood. This approach enhances the model's interpretability and promotes efficient data utilization during the learning process. However, traditional SSMs are limited in their ability to infer shared patterns from a dataset of similar time series, as they are fitted individually to each time series.

SSDNet utilizes the Transformer to learn and estimate the parameters of the SSM network, enabling the generation of interpretable predictions through the SSM network. Therefore, SSDNet requires optimization of the SSM network's predictions and parameters. SSDNet employs a hybrid loss function that integrates MAE and negative log-likelihood. This approach serves the dual purpose of optimizing the model's prediction through MAE and refining the parameters of the SSM network via negative log-likelihood.

The negative log-likelihood loss function requires the model's output to represent the parameters of a probability distribution. The objective of this loss function is to maximize the probability of a real sample satisfying the predicted distribution. Therefore, for a real sample $\mathbf{Y} = [\mathrm{y}_1, \cdots, \mathrm{y}_O]$, and the parameters $\boldsymbol{\theta}$ representing the predicted probability distribution by the model, the optimization objective of the negative log-likelihood can be formulated as follows:

$$\boldsymbol{\theta}^* = \underset{\boldsymbol{\theta}}{argmin} \sum_{i=1}^{O} -logP(\boldsymbol{y}_i|\boldsymbol{\theta}). \tag{44}$$

Generating Adversarial Loss Functions: Inspired by the training method of Generative Adversarial Networks (GANs) (Goodfellow et al. 2014; Wu et al. 2020; Mogren 2016), some models aim to utilize adversarial training to sequences generated by the generator should closely resemble real sequences and remain indistinguishable by the discriminator. Therefore, its optimization objective can be represented as:

$$\min_{G}\max_{\mathcal{D}} V(\mathcal{D}, G) = \mathbb{E}_{x\sim p_{data}(x)}[log\mathcal{D}(x)] + \mathbb{E}_{z\sim p_z(z)}[log(1-\mathcal{D}(G(z)))], \tag{45}$$

where $\mathcal{D}$ is the discriminator of the model, G is the generator of the model, $x \sim p_{data}(x)$ denotes the data samples drawn from the true distribution, and $z \sim p_z(z)$ denotes the fake data samples generated by the generator. From Equation (45), it is evident that the optimization goal is twofold: the discriminator $\mathcal{D}$ aims to distinguish between sequences generated

by the generator and real sequences, while the generator strives to produce fake sequences that the discriminator cannot differentiate from real ones. Next, we will use the AST (Wu et al. 2020) model as an example to introduce the loss function of the generative adversarial approach.AST introduces the integration of the GAN training method into time series prediction tasks. It utilizes the Transformer model as a generator, referred to as G, to generate predictions. The predictions and the real samples are then fed into a discriminator, denoted as $\mathcal{D}$, for discrimination. The discriminator consists of multiple fully connected layers, forming a fully connected network. The GAN loss function consists of two main components: the discriminator loss and the generator loss. In AST, the generator loss is derived from the quantile loss function $P_\rho(\cdot)$, quantifying the numerical gap between the prediction and the real sample. Conversely, the discriminator loss is based on the log-likelihood loss, which evaluates the ability of the discriminator to distinguish the fake and real samples.

$$\hat{\boldsymbol{Y}} = \text{Transformer}(\boldsymbol{X}), \quad \boldsymbol{Y}_{fake} = \text{Concat}(\boldsymbol{X}, \hat{\boldsymbol{Y}}), \tag{46}$$

$$\text{Generator Loss} = P_\rho(\boldsymbol{Y}_O, \hat{\boldsymbol{Y}}) + \lambda \mathbb{E}[\log(1 - \mathcal{D}(\boldsymbol{Y}_{fake}))], \tag{47}$$

$$\text{Discriminator Loss} = \mathbb{E}[-\log \mathcal{D}(\boldsymbol{Y}_{real}) - \log(1 - \mathcal{D}(\boldsymbol{Y}_{fake}))], \tag{48}$$

$$\mathcal{L} = \text{Generator Loss} + \text{Discriminator Loss}, \tag{49}$$

where $\hat{\boldsymbol{Y}} \in \mathbb{R}^{D \times O}$ represents the prediction generated by the Transformer model. D represents the number of variables, and O represents the length of the prediction. On the other hand, $\boldsymbol{Y}_O$ refers to the corresponding ground truth for the prediction. Additionally, $\boldsymbol{Y}_{fake} \in \mathbb{R}^{D \times (L+O)}$ denotes the fake samples generated by the generator, where L represents the length of the input sequence preceding the prediction. These fake samples are generated based on the generator's predictions. $\boldsymbol{Y}_{real} \in \mathbb{R}^{D \times (L+O)}$ represents the real samples drawn from the real dataset.

6 Experiments

In this section, we conduct extensive experiments to evaluate the performance of the aforementioned methods on various real-world time series prediction scenarios.

6.1 Dataset

As shown in Table 3, our empirical results are based on datasets from diverse domains, such as energy, exchange rate, traffic, and disease. Detailed descriptions of these datasets are provided below:

(1) Energy: The ETT (Electricity Transformer Temperature) dataset (Zhou et al. 2021) consists of data collected from power transformers in a specific region of China, spanning from July 2016 to July 2018. It includes seven distinct features, such as the load and oil temperature of the power transformers. The ETTh1 and ETTh2 datasets encompass

17,420 samples with a sampling interval of 1 h, covering 7 features. On the other hand, the ETTm1 and ETTm2 datasets comprise 69,680 samples with a sampling interval of 5 min, also encompassing 7 features. The Electricity dataset (Wu et al. 2021) contains hourly electricity consumption data from 321 customers from 2012 to 2014. It consists of 321 features and a total of 26,304 samples. Due to the temporal characteristics of electricity, electricity datasets naturally exhibit strong periodicity. For instance, the electricity data shows evident monthly periodicity due to the influence of temperature during the summer and winter seasons. Additionally, the electricity dataset also exhibits pronounced daily periodicity across different time periods within a day.

(2) Exchange Rates: The Exchange dataset (Lai et al. 2018) covers daily exchange rates for eight currencies from January 1990 to October 2010. It contains eight features and a total of 7,588 samples. Unlike other time series data, the exchange rate data does not exhibit significant periodicity patterns, and its fluctuation changes are more complex. Additionally, important time series information is often contained in the extreme points of the exchange rate data, making methods like smoothing potentially result in the loss of important information.

(3) Traffic: The Traffic dataset (Wu et al. 2021) comprises hourly data from the California Department of Transportation for the years 2015-2016. It captures road occupancy, measured by various San Francisco Bay Area freeway sensors. The dataset includes 862 features and a total of 17,544 samples. Unlike electricity datasets, traffic datasets exhibit evident short-term periodic patterns. Moreover, traffic accidents frequently lead to abnormal occupancy rates on the associated roadways, resulting in a higher incidence of outliers in the traffic dataset, which can impact model performance.

(4) Illness: The ILI (Influenza-Like Illness) dataset (Wu et al. 2021) comprises weekly records of patients diagnosed with influenza illness, as reported by the U.S. Centers for Disease Control and Prevention, spanning from 2002 to 2021. This dataset includes 7 features, such as the proportion of patients with ILI and the total number of patients. It follows a sampling interval of 1 week and 966 samples. Due to the dynamics of disease transmission, the illness datasets exhibit fluctuations characterized by prominent long-term yearly and monthly periodic patterns and noteworthy short-term trends.

6.2 Model

Table 4 summarizes all the models that appear in the experimental section. For the above models, we have referred to the optimal parameter settings in their respective papers and source code, and we have applied these parameter settings in the following experiments.

6.3 Evaluation metric

The time series forecasting task is typically categorized as a regression problem. Consequently, evaluation metrics commonly used in regression tasks are employed to assess the forecasting performance of models in the time series forecasting task. Based on previous studies (Wu et al. 2021; Zhou et al. 2022; Li et al. 2019; Zhou et al. 2021), we utilize the mean absolute error (MAE) and mean square error (MSE) as fundamental metrics to evaluate the performance of models. At the same time, to facilitate a more intuitive comparison

of the prediction outcomes across various models, we integrated two supplementary evaluation metrics, MAPE, and R^2, in addition to MAE and MSE. MAPE measures the relative error between predicted and actual values, ranging from 0 to positive infinity; the closer to 0, the more accurate the predictions. R^2 evaluates how well the independent variables explain the dependent variable, with values from 0 to 1; the closer to 1, the better the model fits the data. Nonetheless, we presented Mean Absolute Percentage Error(MAPE) and R^2 solely in specific experiments due to space constraints. The definitions of all metrics are given by

$$\mathrm{MAE}(\boldsymbol{\mathcal{Y}}, \hat{\boldsymbol{\mathcal{Y}}}) = \frac{1}{N \times O \times D} \sum_{n=1}^{N} \sum_{o=1}^{O} \sum_{d=1}^{D} \left| y_{o,d}^{n} - \hat{y}_{o,d}^{n} \right|, \tag{50}$$

$$\mathrm{MSE}(\boldsymbol{\mathcal{Y}}, \hat{\boldsymbol{\mathcal{Y}}}) = \frac{1}{N \times O \times D} \sum_{n=1}^{N} \sum_{o=1}^{O} \sum_{d=1}^{D} \left(y_{o,d}^{n} - \hat{y}_{o,d}^{n} \right)^2, \tag{51}$$

$$\mathrm{MAPE}(\boldsymbol{\mathcal{Y}}, \hat{\boldsymbol{\mathcal{Y}}}) = \frac{1}{N \times O \times D} \sum_{n=1}^{N} \sum_{o=1}^{O} \sum_{d=1}^{D} \left| \frac{y_{o,d}^{n} + \epsilon - \hat{y}_{o,d}^{n}}{y_{o,d}^{n} + \epsilon} \right|, \tag{52}$$

Table 4 Summarizes all the models in the experimental section

Category	Model	Year	Category	Model	Year
Statistic model	ARIMA (Zhang 2003)	2003	Transformer-based	CNN-1D (Gudelek et al. 2017)	2017
	ETS (Hyndman and Khandakar 2008)	2008		LogTrans (Li et al. 2019)	2019
				Reformer (Kitaev et al. 2020)	2020
FC-based	Autoencoder (Lv et al. 2014)	2014		AST (Wu et al. 2020)	2020
	SAE (Lv et al. 2014)	2014		Informer (Zhou et al. 2021)	2021
	Nbeat (Oreshkin et al. 2019)	2020		Autoformer (Wu et al. 2021)	2021
	Dlinear (Yun et al. 2019)	2022		Aliformer (Qi et al. 2021)	2021
				TFT (Lim et al. 2021)	2021
RNN-based	GRU (Dey and Salem 2017)	2018		TDformer (Zhang et al. 2022)	2022
	LSTnet (Lai et al. 2018)	2018		Fedformer (Zhou et al. 2022)	2022
	DeepAR (Salinas et al. 2020)	2020		NS-Transformer (Liu et al. 2022)	2022
				Pyraformer (Liu et al. 2021)	2022
CNN-based	CNN-1D (Gudelek et al. 2017)	2017		ETSformer (Woo et al. 2022)	2022
	TCN (Bai et al. 2018)	2018		PatchTST (Nie et al. 2022)	2023
				Crossformer (Zhang and Yan 2022)	2024
State space model	SSDNet (Lin et al. 2021)	2021		Scaleformer (Shabani et al. 2022)	2024
				Triformer (Cirstea et al. 2022)	2024

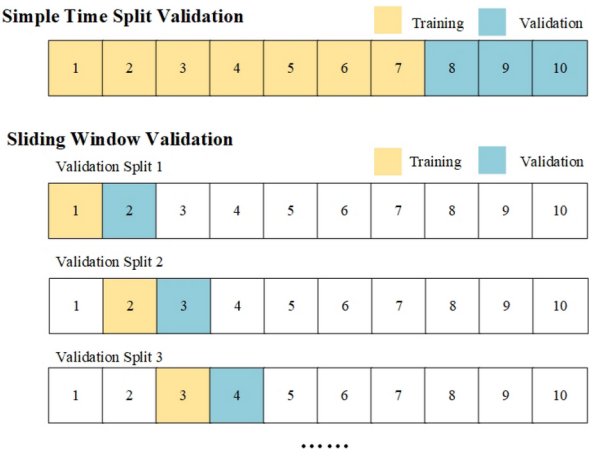

Fig. 17 Validation split of datasets

$$R^2(\boldsymbol{\mathcal{Y}}, \hat{\boldsymbol{\mathcal{Y}}}) = 1 - \frac{\sum_{n=1}^{N}\sum_{o=1}^{O}\sum_{d=1}^{D}\left(y_{o,d}^{n} - \hat{y}_{o,d}^{n}\right)^2}{\sum_{n=1}^{N}\sum_{o=1}^{O}\sum_{d=1}^{D}\left(y_{o,d}^{n} - \overline{y}\right)^2}, \tag{53}$$

where $\overline{y} = \frac{1}{N \times O \times D}\sum_{n=1}^{N}\sum_{o=1}^{O}\sum_{d=1}^{D}(y_{o,d}^{n})$, $\boldsymbol{\mathcal{Y}} = \{\boldsymbol{Y}^1, \boldsymbol{Y}^2, ..., \boldsymbol{Y}^N\}$ represents the set of true values, and $\hat{\boldsymbol{\mathcal{Y}}} = \{\hat{\boldsymbol{Y}}^1, \hat{\boldsymbol{Y}}^2, ..., \hat{\boldsymbol{Y}}^N\}$ denotes the set of predicted values generated by the model. Here, N signifies the number of elements within the set, indicating the presence of N instances. Each instance is characterized by two elements within the set with dimensions of $\mathbb{R}^{O \times D}$. The parameter ϵ is a small value close to 0, introduced to prevent division by zero scenarios. In our code implementation, we have uniformly set ϵ to 0.005 across different models to handle such cases effectively.

6.4 Main results

In this section, we perform experimental analyses on the models and methods mentioned earlier, considering various perspectives:

1. Comprehensive Prediction Accuracy Assessment: We will assess the prediction accuracy of all the mentioned models on 5 time series datasets derived from 4 distinct domains. This assessment aims to provide a comprehensive evaluation of the models' performances.
2. Investigation of Limitations: Considering the limitations of current complex models, we will investigate the underlying causes by focusing on two aspects.

   (1) Shuffling Analysis: By shuffling the input time series, we can analyze the model's ability to capture the sequential order of the data. This analysis involves comparing the model's prediction accuracy before and after shuffling.
   (2) Extending the Lookback Window Analysis: Another aspect involves extending the length of the input time series while maintaining a consistent prediction length.

Table 5 Partial results of the multivariate time series forecasting experiment

Model	Data	ETTh1				Exchange				ILI			
	Metric	MAE	MSE	MAPE	R2	MAE	MSE	MAPE	R2	MAE	MSE	MAPE	R2
Fedformer	Short	0.4566 ± 0.0063	0.4189 ± 0.0097	68.28% ± 0.0300	0.4847 ± 0.0064	0.1629 ± 0.0009	0.0509 ± 0.0002	2.22% ± 0.0000	0.7904 ± 0.0003	0.6483 ± 0.0084	0.9915 ± 0.0137	45.38% ± 0.0168	0.4547 ± 0.0068
	Regular	0.4935 ± 0.0026	0.4754 ± 0.0125	76.75% ± 0.0709	0.4100 ± 0.0036	0.2267 ± 0.0047	0.0944 ± 0.0029	3.09% ± 0.0005	0.6281 ± 0.0159	0.8024 ± 0.0734	1.3779 ± 0.1701	53.95% ± 0.0653	0.2483 ± 0.1041
	Long	0.5714 ± 0.0075	0.5979 ± 0.0092	*83.48%* ± 0.0142	0.2243 ± 0.0346	0.3917 ± 0.0029	0.2699 ± 0.0024	5.37% ± 0.0003	± — 0.0893 ± 0.0016	0.7911 ± 0.0027	1.3501 ± 0.0055	48.47% ± 0.0031	0.2630 ± 0.0019
ETSformer	Short	0.5026 ± 0.0814	0.5423 ± 0.1855	68.63% ± 0.0202	0.4134 ± 0.1594	0.1341 ± 0.0129	0.0341 ± 0.0052	1.89% ± 0.0017	0.8154 ± 0.0390	0.8059 ± 0.0602	1.3679 ± 0.1396	64.15% ± 0.0500	0.2435 ± 0.0872
	Regular	0.5414 ± 0.0881	0.6074 ± 0.2098	73.67% ± 0.0227	0.3194 ± 0.1667	0.1723 ± 0.0066	0.0550 ± 0.0025	2.40% ± 0.0007	0.7459 ± 0.0248	0.9788 ± 0.0530	1.9403 ± 0.1328	66.77% ± 0.0168	— 0.1302 ± 0.1074
	Long	0.6140 ± 0.0906	0.7113 ± 0.1954	84.07% ± 0.0116	0.1581 ± 0.1640	0.3142 ± 0.0063	0.1787 ± 0.0071	4.37% ± 0.0007	0.2277 ± 0.0151	1.0026 ± 0.0042	1.9683 ± 0.0481	59.42% ± 0.0127	— 0.2215 ± 0.0586
Crossformer	Short	0.4132 ± 0.0118	0.3788 ± 0.0273	*62.44%* ± 0.0070	0.5584 ± 0.0291	0.1357 ± 0.0031	0.0352 ± 0.0016	1.91% ± 0.0003	0.8293 ± 0.0041	0.9512 ± 0.1142	2.0541 ± 0.3662	37.92% ± 0.0408	— 0.6692 ± 0.4280
	Regular	0.4522 ± 0.0044	0.4393 ± 0.0193	**68.62%** ± 0.0148	0.4695 ± 0.0190	0.2220 ± 0.0039	0.0901 ± 0.0015	3.17% ± 0.0004	0.4857 ± 0.0600	1.0898 ± 0.0117	2.4253 ± 0.0486	43.60% ± 0.0006	— 1.2288 ± 0.0326
	Long	0.6319 ± 0.0790	0.8007 ± 0.1889	91.78% ± 0.0884	0.1175 ± 0.1432	0.4519 ± 0.0410	0.3505 ± 0.0591	6.53% ± 0.0055	-1,4567 ± 1.0243	1.1988 ± 0.0444	2.8067 ± 0.1412	45.46% ± 0.0143	— 1.4661 ± 0.2171
Dlinear	Short	**0.3979** ± 0.0055	**0.3663** ± 0.0170	66.14% ± 0.0010	**0.5768** ± 0.0166	**0.1104** ± 0.0001	**0.0256** ± 0	**1.54%** ± 0.0000	**0.8649** ± 0.0001	0.6440 ± 0.0088	0.9567 ± 0.0053	35.55% ± 0.0124	0.2927 ± 0.0168
	Regular	**0.4360** ± 0.0073	**0.4229** ± 0.0240	*71.95%* ± 0.0036	**0.4947** ± 0.0242	**0.1576** ± 0.0023	**0.0496** ± 0.0015	**2.20%** ± 0.0003	**0.7781** ± 0.0050	0.7699 ± 0.0068	1.2162 ± 0.0106	**42.02%** ± 0.0025	0.0250 ± 0.0208
	Long	**0.5052** ± 0.0115	**0.5197** ± 0.0287	**81.61%** ± 0.0084	**0.3622** ± 0.0316	**0.2983** ± 0.0036	**0.1561** ± 0.0033	**4.17%** ± 0.0005	**0.1928** ± 0.0474	0.7994 ± 0.0053	*1.2452* ± 0.0136	35.75% ± 0.0042	— 0.1719 ± 0.0209
PatchTST	Short	*0.4008* ± 0.0068	*0.3674* ± 0.0208	65.62% ± 0.0026	*0.5749* ± 0.0217	*0.1115* ± 0.0007	0.0271 ± 0.0003	*1.55%* ± 0.0000	0.8442 ± 0.0042	**0.4584** ± 0.0095	**0.5307** ± 0.0148	**31.30%** ± 0.0124	**0.6475** ± 0.0286
	Regular	*0.4401* ± 0.0097	*0.4253* ± 0.0298	72.19% ± 0.0043	*0.4875* ± 0.0301	*0.1586* ± 0.0012	0.0523 ± 0.0014	*2.22%* ± 0.0002	0.7396 ± 0.0163	**0.6180** ± 0.0317	**0.9902** ± 0.0755	*43.38%* ± 0.0167	**0.4262** ± 0.0037
	Long	*0.5215* ± 0.0113	*0.5509* ± 0.0303	84.38% ± 0.0083	*0.3153* ± 0.0373	0.3219 ± 0.0022	0.1979 ± 0.0028	4.52% ± 0.0002	*0.1393* ± 0.0139	**0.7424** ± 0.1035	**1.1373** ± 0.2004	**31.85%** ± 0.0114	*0.0047* ± 0.3247

Table 5 (continued)

Model	Data	ETTh1				Exchange				ILI			
	Metric	MAE	MSE	MAPE	R2	MAE	MSE	MAPE	R2	MAE	MSE	MAPE	R2
N-BEATS	Short	0.4579 ± 0.0006	0.4353 ± 0.0011	**61.44**% ± 0.0091	0.5110 ± 0.0002	0.1132 ± 0.0030	*0.0268* ± 0.0013	1.57% ± 0.0004	*0.8582* ± 0.0075	0.9474 ± 0.0237	1.8391 ± 0.0951	40.51% ± 0.0072	− 0.5883 ± 0.1189
	Regular	0.6128 ± 0.0013	0.7949 ± 0.0056	90.28% ± 0.0067	0.1884 ± 0.0097	0.1603 ± 0.0013	*0.0506* ± 0.0004	2.24% ± 0.0001	*0.7764* ± 0.0010	1.1330 ± 0.0274	2.5371 ± 0.0913	46.02% ± 0.0084	− 1.5213 ± 0.1167
	Long	0.6682 ± 0.0006	0.8908 ± 0.0052	99.14% ± 0.0025	0.0620 ± 0.0071	*0.3069* ± 0.0136	*0.1661* ± 0.0116	*4.31%* ± 0.0019	0.1185 ± 0.1796	1.3070 ± 0.0276	3.1641 ± 0.0984	51.67% ± 0.0115	− 2.2962 ± 0.1385
Autoformer	Short	0.5041 ± 0.0463	0.4961 ± 0.0811	68.67% ± 0.0408	0.4155 ± 0.0790	0.1786 ± 0.0167	0.0596 ± 0.0101	2.44% ± 0.0017	0.7514 ± 0.0347	0.7902 ± 0.0282	1.3502 ± 0.0904	58.48% ± 0.0404	0.2795 ± 0.0268
	Regular	0.4970 ± 0.0064	0.4796 ± 0.0179	75.32% ± 0.0054	0.4031 ± 0.0048	0.2159 ± 0.0080	0.0869 ± 0.0065	2.96% ± 0.0010	0.6575 ± 0.0273	0.8397 ± 0.0289	1.4801 ± 0.0617	60.23% ± 0.0296	0.2069 ± 0.0597
	Long	0.5766 ± 0.0095	0.6128 ± 0.0310	87.14% ± 0.0118	0.2265 ± 0.0137	0.4159 ± 0.0332	0.2997 ± 0.0441	5.74% ± 0.0043	− 0.2052 ± 0.1593	0.8438 ± 0.0332	1.5455 ± 0.0968	48.31% ± 0.0200	0.1352 ± 0.0551
NS-Transformer	Short	0.4306 ± 0.0062	0.4197 ± 0.0088	70.07% ± 0.0045	0.5293 ± 0.0084	0.1334 ± 0.0014	0.0345 ± 0.0008	1.88% ± 0.0002	0.8260 ± 0.0072	*0.5493* ± 0.0052	*0.7300* ± 0.0716	*34.15%* ± 0.0131	*0.5511* ± 0.0222
	Regular	0.4812 ± 0.0072	0.4946 ± 0.0037	77.26% ± 0.0032	0.4130 ± 0.0122	0.1931 ± 0.0088	0.0730 ± 0.0071	2.68% ± 0.0009	0.6902 ± 0.0196	*0.7099* ± 0.0783	*1.1508* ± 0.2396	46.66% ± 0.0863	*0.3363* ± 0.0760
	Long	0.6070 ± 0.0154	0.7074 ± 0.3587	84.74% ± 0.0211	0.1131 ± 0.0719	0.3627 ± 0.0162	0.2398 ± 0.0213	5.02% ± 0.0021	0.0174 ± 0.0761	*0.7579* ± 0.0277	1.3199 ± 0.1218	*35.02%* ± 0.0174	**0.2064** ± 0.0546
Model	Data	Traffic				Electricity							
	Metric	MAE	MSE	MAPE	R2	MAE	MSE	MAPE	R2				
Fedformer	Short	0.3169 ± 0.0010	0.3693 ± 0.0011	18.21% ± 0.0010	0.6774 ± 0.0014	0.2973 ± 0.0019	0.1877 ± 0.0024	17.02% ± 0.0018	0.8063 ± 0.0025				
	Regular	0.3172 ± 0.0035	0.3699 ± 0.0028	17.99% ± 0.0031	0.6727 ± 0.0064	0.3009 ± 0.0028	0.1924 ± 0.0030	16.91% ± 0.0013	0.8014 ± 0.0031				
	Long	0.3234 ± 0.0015	*0.3876* ± 0.0027	*18.34%* ± 0.0013	*0.6583* ± 0.0013	0.4646 ± 0.1300	0.3942 ± 0.1719	26.18% ± 0.0650	0.5911 ± 0.1783				

Table 5 (continued)

Model	Data	Traffic				Electricity			
	Metric	MAE	MSE	MAPE	R2	MAE	MSE	MAPE	R2
ETSformer	Short	0.4652 ± 0.0141	0.6078 ± 0.0142	27.64% ± 0.0058	0.4623 ± 0.0119	0.3185 ± 0.0005	0.2163 ± 0.0017	19.18% ± 0.0004	0.7768 ± 0.0018
	Regular	0.4721 ± 0.0083	0.4721 ± 0.0083	27.57% ± 0.0008	0.4483 ± 0.0141	0.3370 ± 0.0009	0.2430 ± 0.0010	20.35% ± 0.0004	0.7492 ± 0.0010
	Long	0.4800 ± 0.0099	0.6799 ± 0.0255	27.35% ± 0.0051	0.4059 ± 0.0224	0.3642 ± 0.0006	0.2824 ± 0.0012	21.91% ± 0.0005	0.7071 ± 0.0012
Crossformer	Short	**0.2453** ± 0.0023	**0.3085** ± 0.0030	**15.29%** ± 0.0001	0.7329 ± 0.0024	**0.2342** ± 0.0039	**0.1458** ± 0.0054	**10.88%** ± 0.0022	**0.8496** ± 0.0055
	Regular	**0.2718** ± 0.0015	**0.3452** ± 0.0016	**17.26%** ± 0.0001	**0.6998** ± 0.0013	**0.2485** ± 0.0046	**0.1647** ± 0.0080	**11.57%** ± 0.0032	**0.8301** ± 0.0082
	Long	–	–	–	–	*0.2882* ± 0.0045	**0.2022** ± 0.0071	**13.48%** ± 0.0033	**0.7907** ± 0.0073
Dlinear	Short	0.3738 ± 0.0002	0.5417 ± 0.0003	24.83% ± 0.0002	0.5231 ± 0.0003	0.2676 ± 0.0003	0.2004 ± 0.0003	14.88% ± 0.0003	0.7832 ± 0.0003
	Regular	0.3478 ± 0.0002	0.5075 ± 0.0003	22.92% ± 0.0001	0.5548 ± 0.0003	*0.2741* ± 0.0020	0.1992 ± 0.0013	14.89% ± 0.0017	0.7943 ± 0.0014
	Long	0.3292 ± 0.0002	0.4681 ± 0.0001	21.51% ± 0.0005	0.5955 ± 0.0001	0.2916 ± 0.0001	0.2095 ± 0.0002	14.98% ± 0.001	0.7831 ± 0.0002
PatchTST	Short	*0.2654* ± 0.0010	*0.3379* ± 0.0022	*17.03%* ± 0.0005	**0.7047** ± 0.0019	*0.2574* ± 0.0012	0.1860 ± 0.0008	14.39% ± 0.0011	0.8081 ± 0.0008
	Regular	*0.2755* ± 0.0013	*0.3615* ± 0.0014	*17.57%* ± 0.0005	*0.6848* ± 0.0012	0.2631 ± 0.0004	0.1891 ± 0.0002	14.05% ± 0.0004	0.8049 ± 0.0002
	Long	**0.2835** ± 0.0003	**0.3857** ± 0.0002	**18.00%** 0.0003	**0.6686** ± 0.0002	**0.2861** ± 0	0.2092 ± 0.0002	*14.20%* ± 0.0001	0.7834 ± 0.0003
N-BEATS	Short	0.3886 ± 0.0008	0.6086 ± 0.0033	24.91% ± 0.0006	0.4717 ± 0.0029	0.3760 ± 0.0007	0.2855 ± 0.0011	18.73% ± 0.0014	0.7054 ± 0.0011
	Regular	0.6862 ± 0.0002	1.0418 ± 0.0005	31.40% ± 0.0009	0.0651 ± 0.0005	0.6903 ± 0.0001	0.8176 ± 0.0022	39.21% ± 0.0016	0.1560 ± 0.0023
	Long	0.7400 ± 0.0020	1.1278 ± 0.0007	31.78% ± 0.0016	0.0012 ± 0.0008	0.7374 ± 0.0005	0.9044 ± 0.0003	41.79% ± 0.0015	0.0635 ± 0.0004

Table 5 (continued)

Model	Data	Traffic				Electricity			
	Metric	MAE	MSE	MAPE	R2	MAE	MSE	MAPE	R2
Autoformer	Short	0.3175 ± 0.0028	0.3744 ± 0.0051	17.89% ± 0.0004	0.6649 ± 0.0063	0.3005 ± 0.0044	0.1922 ± 0.0045	17.53% ± 0.0031	0.8017 ± 0.0047
	Regular	0.3419 ± 0.0088	0.4202 ± 0.0206	19.27% ± 0.0017	0.6260 ± 0.0175	0.3089 ± 0.0019	0.2016 ± 0.0012	17.44% ± 0.0002	0.7919 ± 0.0013
	Long	0.3393 ± 0.0077	0.4095 ± 0.0066	19.82% ± 0.0021	0.6389 ± 0.0062	0.3577 ± 0.0146	0.2609 ± 0.0192	20.37% ± 0.0068	0.7298 ± 0.0199
NS-Transformer	Short	0.2910 ± 0.0043	0.3491 ± 0.0071	17.34% ± 0.0011	*0.6978* ± 0.0063	0.2646 ± 0.0014	*0.1637* ± 0.0017	*13.22%* ± 0.0005	*0.8311* ± 0.0017
	Regular	0.3046 ± 0.0044	0.3718 ± 0.0068	18.44% ± 0.0013	0.6786 ± 0.0060	0.2771 ± 0.0020	*0.1784* ± 0.0019	*13.62%* ± 0.0016	*0.8159* ± 0.0019
	Long	*0.3202* ± 0.0010	0.4008 ± 0.0001	21.02% ± 0.0156	0.6587 ± 0.0003	0.3024 ± 0.0052	*0.2052* ± 0.0055	14.52% ± 0.0018	*0.7875* ± 0.0057

The best results are highlighted in bold and the second best results are highlighted with a *italic*. "–" indicates that no valid results could be obtained. 'Short,' 'Regular,' and 'Long' denote the output length O for short-term, regular-term, and long-term time series prediction. We report the average value and standard deviation of experiments

Table 6 Partial results of the univariate time series forecasting experiment

Model	Data	ETTh1				Exchange				ILI			
	Metric	MAE	MSE	MAPE	R2	MAE	MSE	MAPE	R2	MAE	MSE	MAPE	R2
Fedformer	Short	0.3325 ± 0.0027	0.1992 ± 0.0025	20.06% ± 0.0090	0.5239 ± 0.0059	0.1298 ± 0.0015	0.0281 ± 0.0007	1.45% ± 0.0000	0.8460 ± 0.0037	0.4053 ± 0.0933	*0.2466* ± 0.0751	7.63% ± 0.0030	*–0.0970* ± 0.6241
	Regular	0.3555 ± 0.0053	0.2220 ± 0.0048	21.93% ± 0.0101	0.4689 ± 0.0113	0.1930 ± 0.0058	0.0617 ± 0.0036	2.19% ± 0.0006	0.6581 ± 0.0203	0.4493 ± 0.1504	0.2987 ± 0.1756	**8.95**% ± 0.0077	− 0.9179 ± 1.5872
	Long	0.4001 ± 0.0039	0.2777 ± 0.0054	23.44% ± 0.0012	0.3039 ± 0.0168	0.3551 ± 0.0069	0.2065 ± 0.0097	3.97% ± 0.0006	− 0.1932 ± 0.0592	0.5595 ± 0.1326	0.4466 ± 0.1821	**10.67**% ± 0.0006	− 6.2454 ± 5.4025
ETSformer	Short	0.3329 ± 0.0106	0.1994 ± 0.0085	19.94% ± 0.0008	0.5233 ± 0.0217	0.1044 ± 0.0051	0.0174 ± 0.0012	1.15% ± 0.0004	0.9049 ± 0.0066	0.4790 ± 0.0629	0.3744 ± 0.0871	11.31% ± 0.0057	− 0.5793 ± 0.7979
	Regular	0.3604 ± 0.0148	0.2298 ± 0.0164	21.37% ± 0.0026	0.4503 ± 0.0392	0.1386 ± 0.0085	0.0319 ± 0.0030	1.53% ± 0.0008	0.8231 ± 0.0167	0.5411 ± 0.0583	0.4164 ± 0.1047	16.70% ± 0.0148	− 1.0299 ± 0.6397
	Long	0.4194 ± 0.0283	0.3005 ± 0.0403	23.56% ± 0.0094	0.2474 ± 0.0942	0.2559 ± 0.0299	0.1014 ± 0.0130	2.81% ± 0.0024	0.4178 ± 0.0676	0.6302 ± 0.1110	0.5230 ± 0.1656	16.04% ± 0.0222	− 5.6200 ± 4.3973
Crossformer	Short	0.3146 ± 0.0054	0.1829 ± 0.0061	19.36% ± 0.0025	0.5627 ± 0.0154	0.1229 ± 0.0116	0.0233 ± 0.0028	1.35% ± 0.0010	0.8721 ± 0.0155	1.1821 ± 0.3462	2.1848 ± 1.4362	25.17% ± 0.0389	− 8.6991 ± 7.7009
	Regular	0.3404 ± 0.0031	0.2120 ± 0.0049	20.44% ± 0.0028	0.4928 ± 0.0117	0.1811 ± 0.0159	0.0489 ± 0.0068	1.96% ± 0.0013	0.7290 ± 0.0373	1.1480 ± 0.0811	1.5548 ± 0.3470	30.23% ± 0.0012	− 6.9744 ± 2.990
	Long	0.4192 ± 0.0056	0.2960 ± 0.0057	24.86% ± 0.0040	0.2577 ± 0.0187	0.2936 ± 0.0054	0.1172 ± 0.0011	3.19% ± 0.0003	0.3332 ± 0.0115	1.1606 ± 0.0709	1.5138 ± 0.1175	28.63% ± 0.0115	− 19.8578 ± 13.3763
Dlinear	Short	**0.2987** ± 0.0069	**0.1729** ± 0.0072	**18.38**% ± 0.0035	**0.5867** ± 0.0184	**0.0927** ± 0.0002	**0.0144** ± 0.0001	**1.02**% ± 0.0000	**0.9211** ± 0.0003	0.6970 ± 0.1379	0.7121 ± 0.1996	14.43% ± 0.0027	− 2.2478 ± 1.8562
	Regular	**0.3214** ± 0.0067	**0.1952** ± 0.0082	**19.66**% ± 0.0044	**0.5331** ± 0.0196	0.1768 ± 0.0344	0.0529 ± 0.0182	**1.49**% ± 0.0003	0.6562 ± 0.0235	0.9373 ± 0.1463	1.1032 ± 0.2721	19.69% ± 0.0037	− 5.7115 ± 4.1439
	Long	*0.3712* ± 0.0089	**0.2455** ± 0.0104	**21.93**% ± 0.0062	**0.3848** ± 0.0200	**0.2465** ± 0.0077	*0.0996* 0.0085	*2.74%* ± 0.0007	**0.4506** ± 0.0519	0.9340 ± 0.1076	1.0252 ± 0.2198	21.11% ± 0.0011	− 13.7285 ± 10.1221

Table 6 (continued)

Model	Data	ETTh1				Exchange				ILI			
	Metric	MAE	MSE	MAPE	R2	MAE	MSE	MAPE	R2	MAE	MSE	MAPE	R2
PatchTST	Short	*0.3031* ± 0.0056	*0.1774* ± 0.0062	*18.61%* ± 0.0028	*0.5759* ± 0.0161	*0.0948* ± 0.0009	*0.0151* ± 0.0003	*1.05%* ± 0.0001	*0.9173* ± 0.0015	**0.2874** ± 0.0308	**0.1614** ± 0.0217	**6.54%** ± 0.0012	**0.3019** ± 0.3273
	Regular	*0.3276* ± 0.0053	*0.2021* ± 0.0071	*19.97%* ± 0.0034	*0.5163* ± 0.0171	**0.1352** ± 0.0010	**0.0315** ± 0.0003	*1.50%* ± 0.0001	**0.8251** ± 0.0016	**0.3542** ± 0.0196	**0.2233** ± 0.0433	*9.26%* ± 0.0030	**–0.3066** ± 0.7559
	Long	**0.3693** ± 0.0113	*0.2478* ± 0.0140	*21.96%* ± 0.0065	*0.3794* ± 0.0289	0.2481 ± 0.0009	0.1113 ± 0.0010	2.77% ± 0.0001	0.3609 ± 0.0056	*0.4604* ± 0.0262	*0.3440* ± 0.0645	11.38% ± 0.0014	*–4.5434* ± 3.9137
N-BEATS	Short	0.3119 ± 0.0029	0.1805 ± 0.0025	19.34% ± 0.0017	0.5686 ± 0.0073	0.0959 ± 0.0040	0.0153 ± 0.0011	1.06% ± 0.0003	0.6439 ± 0.4665	1.2647 ± 0.1778	1.8155 ± 0.3813	27.16% ± 0.0100	− 6.7432 ± 3.9041
	Regular	0.3660 ± 0.0016	0.2341 ± 0.0012	22.51% ± 0.0038	0.4400 ± 0.0028	*0.1383* ± 0.0026	*0.0321* ± 0.0006	1.53% ± 0.0002	*0.8222* ± 0.0034	1.7303 ± 0.2611	3.4102 ± 0.7110	34.33% ± 0.0078	− 19.4551 ± 12.1926
	Long	0.4114 ± 0.0003	0.2837 ± 0.0032	24.50% ± 0.0047	0.2889 ± 0.0136	*0.2468* ± 0.0294	**0.0986** ± 0.0228	**2.71%** ± 0.0024	*0.4326* ± 0.1301	1.5454 ± 0.1340	2.6395 ± 0.3146	35.11% ± 0.0074	− 35.4507 ± 23.4484
Autoformer	Short	0.3473 ± 0.0055	0.2138 ± 0.0118	21.06% ± 0.0065	0.4890 ± 0.0297	0.1537 ± 0.0343	0.0391 ± 0.0170	1.70% ± 0.0030	0.7855 ± 0.0935	0.4348 ± 0.1186	0.3105 ± 0.1343	9.29% ± 0.0065	− 0.3986 ± 0.9142
	Regular	0.3634 ± 0.0093	0.2317 ± 0.0072	21.73% ± 0.0053	0.4456 ± 0.0173	0.1727 ± 0.0127	0.05 ± 0.0076	1.91% ± 0.0012	0.7230 ± 0.0422	0.5409 ± 0.0956	0.4231 ± 0.1004	10.27% ± 0.0141	− 1.4497 ± 1.4705
	Long	0.4041 ± 0.0184	0.2819 ± 0.0253	23.70% ± 0.0052	0.2933 ± 0.0662	0.3941 ± 0.0846	0.2411 ± 0.0809	4.40% ± 0.0076	0.4032 ± 0.4633	0.6079 ± 0.1625	0.4947 ± 0.1924	*11.32%* ± 0.0030	− 6.6231 ± 5.6031
NS-Transformer	Short	0.3163 ± 0.0065	0.1821 ± 0.0034	19.89% ± 0.0085	0.5647 ± 0.0091	0.1125 ± 0.0040	0.0213 ± 0.0013	1.25% ± 0.0004	0.8834 ± 0.0071	*0.3869* ± 0.1363	0.2530 ± 0.1569	*7.08%* ± 0.0025	− 0.1363 ± 0.9093
	Regular	0.3471 ± 0.0036	0.2152 ± 0.0062	21.29% ± 0.0011	0.4850 ± 0.0149	0.1683 ± 0.0069	0.0494 ± 0.0044	1.86% ± 0.0006	0.1231 ± 0.0651	*0.4107* ± 0.0348	*0.2840* ± 0.0493	10.04% ± 0.0004	*–0.6545* ± 0.9427
	Long	0.5269 ± 0.0662	0.4747 ± 0.1039	27.92% ± 0.0255	0.1892 ± 0.2676	0.3206 ± 0.0149	0.1764 ± 0.0159	3.58% ± 0.0014	− 0.0225 ± 0.0944	**0.4165** ± 0.0617	**0.2569** ± 0.0558	12.14% ± 0.0027	**–2.4319** ± 2.1392

Table 6 (continued)

Model	Data	Traffic				Electricity			
	Metric	MAE	MSE	MAPE	R2	MAE	MSE	MAPE	R2
Fedformer	Short	0.4210 ± 0.0066	*0.2826* ± 0.0070	11.91% ± 0.0026	*0.7750* ± 0.0055	0.3732 ± 0.0081	0.2480 ± 0.0082	28.23% ± 0.0182	0.7565 ± 0.0081
	Regular	0.4261 ± 0.0134	0.2891 ± 0.0137	12.40% ± 0.0066	0.7692 ± 0.0106	0.3250 ± 0.0124	*0.1967* ± 0.0119	24.11% ± 0.0090	0.8040 ± 0.0117
	Long	0.4281 ± 0.0060	*0.2938* ± 0.0066	12.15% ± 0.0029	*0.7642* ± 0.0053	0.6163 ± 0.2896	0.6571 ± 0.5036	49.59% ± 0.2122	0.3564 ± 0.4932
ETSformer	Short	0.5593 ± 0.0064	0.4885 ± 0.0039	16.35% ± 0.0022	0.6109 ± 0.0031	0.5673 ± 0.0109	0.5584 ± 0.0255	47.01% ± 0.0114	0.4517 ± 0.0251
	Regular	0.5672 ± 0.0002	0.5067 ± 0.0002	16.30% ± 0.0005	0.5955 ± 0.0004	0.6315 ± 0.0067	0.6620 ± 0.0123	53.51% ± 0.0039	0.3505 ± 0.0121
	Long	0.5756 ± 0.0084	0.5355 ± 0.0143	16.34% ± 0.0018	0.5703 ± 0.0114	0.6709 ± 0.0029	0.7967 ± 0.0006	62.45% ± 0.0020	0.2197 ± 0.0006
Crossformer	Short	*0.3915* ± 0.0081	0.2846 ± 0.0092	14.50% ± 0.0045	0.7733 ± 0.0073	**0.3033** ± 0.0477	*0.2277* ± 0.0705	**18.85**% ± 0.0134	*0.7764* ± 0.0692
	Regular	**0.3784** ± 0.0023	**0.2790** ± 0.0014	11.94% ± 0.0010	*0.7772* ± 0.0011	**0.2796** ± 0.0138	**0.1818** ± 0.0183	**20.10**% ± 0.0222	**0.8216** ± 0.0180
	Long	–	–	–	–	*0.3350* ± 0.0225	*0.2443* ± 0.0535	*24.95%* ± 0.0313	*0.7608* ± 0.0524
Dlinear	Short	0.4687 ± 0.0042	0.4177 ± 0.0029	14.19% ± 0.0005	0.6673 ± 0.0023	0.6975 ± 0.0018	0.9390 ± 0.0022	59.08% ± 0.0010	0.0780 ± 0.0022
	Regular	0.4398 ± 0.0003	0.3811 ± 0.0006	13.19% ± 0.0003	0.6957 ± 0.0005	0.5960 ± 0.0025	0.7090 ± 0.0089	50.37% ± 0.0017	0.3044 ± 0.0087
	Long	0.4201 ± 0.0002	0.3443 ± 0.0002	12.93% ± 0.0001	0.7239 ± 0.0001	0.4864 ± 0.0005	0.4893 ± 0.0009	39.49% ± 0.0002	0.5208 ± 0.0008
PatchTST	Short	**0.3514** ± 0.0009	**0.2488** ± 0.0003	*10.33%* ± 0.0005	**0.8018** ± 0.0002	0.5249 ± 0.0025	0.6473 ± 0.0049	51.84% ± 0.0014	0.3644 ± 0.0048
	Regular	**0.3610** ± 0.0006	**0.2606** ± 0.0013	*10.51%* ± 0.0003	**0.7918** ± 0.0010	0.4437 ± 0.0036	0.4893 ± 0.0046	42.39% 0.0043	0.5199 ± 0.0045
	Long	**0.3700** ± 0.0002	**0.2722** ± 0.0002	*11.00%* ± 0.0003	**0.7815** ± 0.0001	0.3770 ± 0.0037	0.3589 ± 0.0067	31.95% ± 0.0028	0.6485 ± 0.0066

Table 6 (continued)

Model	Data	Traffic				Electricity			
	Metric	MAE	MSE	MAPE	R2	MAE	MSE	MAPE	R2
N-BEATS	Short	0.4647 ± 0.0017	0.3671 ± 0.0029	16.02% ± 0.0007	0.7076 ± 0.0024	0.5377 ± 0.0135	0.5074 ± 0.0266	46.15% ± 0.0155	0.5018 ± 0.0262
	Regular	0.8803 ± 0.0001	1.1347 ± 0.0008	25.29% ± 0.0052	0.0942 ± 0.0007	0.6250 ± 0.0019	0.6499 ± 0.0089	45.25% ± 0.0015	0.3623 ± 0.0088
	Long	0.9589 ± 0.0019	1.2246 ± 0.0029	31.83% ± 0.0019	0.0174 ± 0.0023	0.6735 ± 0.0027	0.7926 ± 0.0018	42.67% ± 0.0046	0.2237 ± 0.0018
Autoformer	Short	0.4294 ± 0.0086	0.2901 ± 0.0098	13.61% ± 0.0005	0.8665 ± 0.1012	0.4493 ± 0.0324	0.3648 ± 0.0492	39.34% ± 0.0345	0.6419 ± 0.0483
	Regular	0.4610 ± 0.0037	0.3388 ± 0.0145	13.78% ± 0.0003	0.7304 ± 0.0099	0.4126 ± 0.0071	0.2988 ± 0.0122	32.20% ± 0.0078	*0.3069* ± 0.0120
	Long	*0.4274* ± 0.0037	0.2969 ± 0.0045	12.71% ± 0.0042	0.7594 ± 0.0010	0.5767 ± 0.0688	0.5460 ± 0.1371	45.83% ± 0.0521	0.4653 ± 0.1343
NS-Transformer	Short	0.4654 ± 0.0013	0.3691 ± 0.0033	**8.84**% ± 0.0020	0.7060 ± 0.0028	*0.3133* ± 0.0038	**0.1896** ± 0.0028	*21.84%* ± 0.0042	**0.8139** ± 0.0028
	Regular	0.8804 ± 0.0050	1.1424 ± 0.0115	**9.54**% ± 0.0013	0.0926 ± 0.0012	*0.3189* ± 0.0120	0.1984 ± 0.0088	*21.72%* ± 0.0022	*0.8054* ± 0.0086
	Long	0.9661 ± 0.0052	1.2290 ± 0.0038	**10.83**% ± 0.0046	0.0121 ± 0.0019	**0.3099** ± 0.0113	**0.1743** ± 0.0103	**21.41**% ± 0.0067	**0.8293** ± 0.0101

The best results are highlighted in bold and the second best results are highlighted with a *underline*. "–" indicates that no valid results could be obtained. 'Short,' 'Regular,' and 'Long' denote the output length O for short-term, regular-term, and long-term time series prediction. We report the average value and standard deviation of experiments

Table 7 The comparison of prediction results between the model before and after shuffling the input sequence

Dataset		ETTh1		Traffic		ILI		Exchange	
Metric		MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE
DLinear	*Ori.*	0.4317	0.4089	0.3495	0.5034	0.9440	1.8397	0.1573	0.0487
	Shuf.	0.6655	0.8234	0.7535	1.1284	1.2645	2.8216	0.3443	0.2153
	Drop	54.16%	101.37%	115.59%	124.16%	33.95%	53.37%	118.88%	342.09%
PatchTST	*Ori.*	0.4070	0.3865	0.3329	0.4540	0.7055	1.1853	0.1609	0.0520
	Shuf.	0.7860	1.0518	0.7664	1.1403	1.1530	2.3930	0.6244	0.6201
	Drop	*93.12%*	*172.13%*	130.22%	151.17%	*63.43%*	*101.89%*	*288.07%*	*1092.50%*
Crossformer	*Ori.*	0.4454	0.4160	0.7634	1.1349	1.0672	2.3385	0.2575	0.1118
	Shuf.	0.7744	1.0274	0.7610	1.1493	1.2951	3.2890	0.6540	0.5988
	Drop	73.84%	146.97%	— 0.31%	1.27%	21.35%	28.90%	153.98%	435.60%
NS-Transformer	*Ori.*	0.6121	0.7382	0.3403	0.4252	0.8402	1.7844	0.1695	0.0565
	Shuf.	0.7796	1.0340	0.7617	1.1400	1.1948	2.7962	0.6183	0.6120
	Drop	27.36%	40.07%	123.83%	168.11%	29.68%	56.70%	264.78%	983.19%
ETSformer	*Ori.*	0.5246	0.5397	0.4722	0.6042	0.9896	1.9351	0.1817	0.0596
	Shuf.	0.6653	0.8575	0.7753	1.1925	1.1065	2.1160	0.3195	0.1867
	Drop	26.82%	58.88%	64.19%	97.37%	11.81%	9.35%	75.84%	213.26%
LSTnet	*Ori.*	0.6960	0.9192	0.7115	0.9279	1.2103	2.7753	0.8062	0.8898
	Shuf.	0.7548	1.0156	0.7654	1.1435	1.1605	2.6412	0.8217	0.9303
	Drop	8.45%	10.49%	7.58%	23.24%	— 4.11%	— 4.83%	1.92%	4.55%
N-BEATS	*Ori.*	0.6098	0.7883	0.6878	1.0420	1.2166	2.7416	0.1618	0.0514
	Shuf.	0.6479	0.8235	0.7507	1.1260	1.2702	3.2442	0.3484	0.2103
	Drop	6.25%	4.47%	9.15%	8.06%	4.41%	18.33%	115.33%	309.14%
TDformer	*Ori.*	0.6742	0.7662	0.3524	0.4342	1.4841	3.6448	1.5010	3.2275
	Shuf.	0.7689	1.0767	0.7591	1.1573	1.4038	3.7931	1.2450	2.1344
	Drop	14.05%	40.52%	115.41%	166.54%	— 5.41%	4.07%	— 17.06%	— 33.87%
Autoformer	*Ori.*	0.5389	0.5443	0.3190	0.3814	0.8414	1.5320	0.2162	0.0885
	Shuf.	0.8101	1.1000	0.8059	1.2024	1.1584	2.2000	0.5779	0.5850
	Drop	50.32%	102.09%	152.63%	215.26%	37.68%	43.60%	167.30%	561.02%
Fedformer	*Ori.*	0.4980	0.4695	0.3168	0.3695	0.8095	1.3598	0.2245	0.0951
	Shuf.	0.8130	1.1080	0.8036	1.1964	1.2082	2.4158	0.5740	0.5861
	Drop	63.25%	136.00%	*153.66%*	*223.79%*	49.25%	77.66%	155.68%	516.30%

Ori. denotes the original prediction accuracy, *Shuf.* denotes the prediction accuracy after the shuffling, and *Drop* is calculated as follows: $\mathbf{Drop} = \frac{(\mathbf{Shuf.} - \mathbf{Ori.})}{\mathbf{Ori.}} \times 100\%$. The maximum results of the decline rate are highlighted in *bold*

Through this analysis, we can examine the model's susceptibility to overfitting and the level of interference caused by noise.

3. Trend and Season Information Analysis: To evaluate the model's capacity to capture trend and season information, experiments will be conducted on an artificial dataset explicitly incorporating trend and season terms. This analysis assesses how well the models can capture such temporal patterns.
4. Attention Module Evaluation for Transformer-based Models: For the Transformer-based models, our primary focus will be on assessing the performance of different

Table 8 The prediction results for each model across various datasets, considering different lengths of the lookback window. The best results are highlighted in *bold*

Dataset		ETTh1		Exchange	
Metric		MAE	MSE	MAE	MSE
DLinear	48	0.4389	0.4367	0.1582	0.0500
	96	0.4317	0.4089	**0.1572**	**0.0487**
	192	**0.4301**	**0.3994**	0.1585	0.0488
PatchTST	48	0.4434	0.4459	0.6148	0.6097
	96	0.4323	0.4058	0.6079	0.5925
	192	**0.4289**	**0.3965**	**0.5881**	**0.5371**
Crossformer	48	0.4464	0.4338	0.5892	0.5196
	96	0.4454	**0.4157**	0.5743	**0.5051**
	192	**0.4452**	0.4175	**0.5705**	0.5170
NS-Transformer	48	0.5164	0.5420	0.6182	0.6200
	96	**0.4990**	**0.5165**	0.6193	0.6100
	192	0.5115	0.5360	**0.6023**	**0.5655**
Fedformer	48	**0.4962**	**0.4656**	**0.1946**	**0.0731**
	96	0.5082	0.4748	0.2255	0.0937
	192	0.5170	0.4834	0.2782	0.1343
Autoformer	48	**0.5388**	**0.5289**	**0.1997**	**0.0767**
	96	0.5439	0.5480	0.2298	0.0978
	192	0.5521	0.5536	0.2290	0.0909
N-BEATS	48	0.6110	0.7959	**0.1598**	0.0517
	96	**0.6099**	0.7874	0.1633	0.0522
	192	0.6130	**0.7872**	0.1632	**0.0513**
ETSformer	48	**0.5061**	**0.5318**	**0.1795**	**0.0581**
	96	0.5257	0.5393	0.1837	0.0602
	192	0.5482	0.5604	0.1842	0.0611
LSTnet	48	**0.6989**	**0.9221**	0.8237	0.9241
	96	0.6991	0.9270	0.8073	0.8907
	192	0.7097	0.9385	**0.7675**	**0.8195**
TDformer	48	**0.6514**	**0.7323**	**1.3290**	**2.5359**
	96	0.6705	0.7506	1.4197	2.9719
	192	0.6845	0.7826	1.3507	2.7163

Attention modules utilized in the models. To achieve this, a comparative experiment has been designed to evaluate the prediction accuracy and complexity associated with various Attention modules.These analyses aim to gain insights into the strengths, limitations, performances, and characteristics of the models and methods mentioned earlier.

6.4.1 Evaluation of model prediction accuracy

In order to comprehensively evaluate the prediction accuracy of each model, this section presents experimental results on both multivariate and univariate time series prediction tasks.

As shown in Fig. 17, We employ two different validation set splitting methods: (1) simple time split validation and (2) sliding window validation. In simple time split validation (Wu et al. 2021; Zhou et al. 2021; Kitaev et al. 2020; Li et al. 2019), the training, validation, and test sets are sequentially divided based on time. Sliding window validation (Liu and

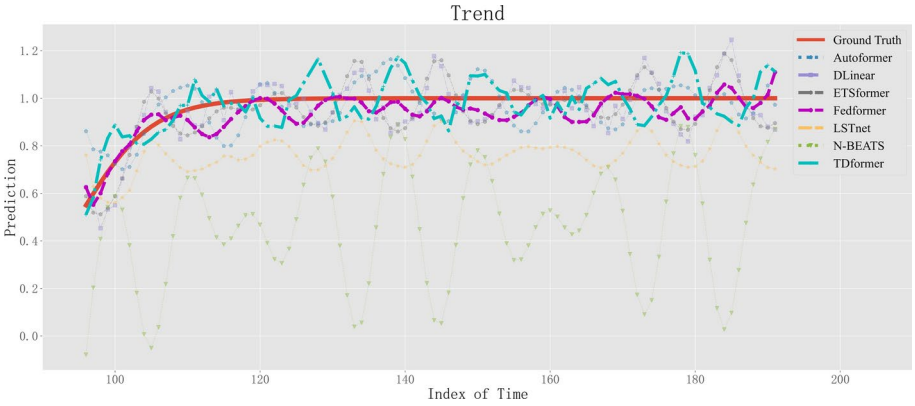

Fig. 18 The prediction results for the trend term. The horizontal axis represents the time step index, and the vertical axis represents the model's prediction result. Ground Truth represents the true value corresponding to the predicted segment, and we highlight the better-performing model results and Ground Truth

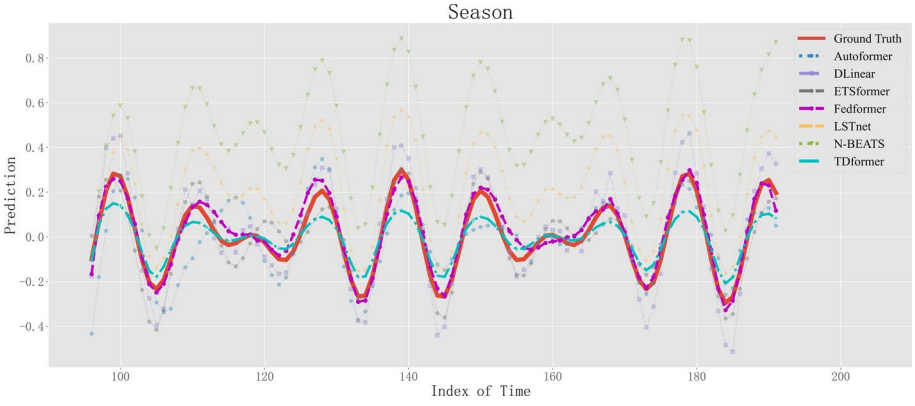

Fig. 19 The prediction results for the season term. The horizontal axis represents the time step index, and the vertical axis represents the model's prediction result. Ground Truth represents the true value corresponding to the predicted segment, and we highlight the better-performing model results and Ground Truth

Table 9 The prediction of the model for the trend and season term in the artificial dataset. The best results are highlighted in **bold** and the second best results are highlighted with a *underline*

Data	Original Sequence		Trend Term		Season Term	
Metric	MAE	MSE	MAE	MSE	MAE	MSE
DLinear	**0.0145**	**0.0003**	0.1456	0.0351	0.1500	0.0351
Fedformer	0.0612	0.0057	**0.1074**	**0.0181**	**0.0823**	**0.0115**
Autoformer	0.1246	0.0276	0.1602	0.0433	0.1861	0.0570
N-BEATS	*0.0269*	*0.0014*	0.6359	0.4996	0.9032	0.8704
ETSformer	0.1705	0.0447	0.1501	0.0351	0.1162	0.0206
LSTnet	0.0481	0.0058	0.4210	0.1962	0.4159	0.1921
TDformer	0.0718	0.0106	*0.1109*	*0.0193*	*0.1122*	*0.0202*

Table 10 Comparison of prediction results among different Attention modules with equivalent space complexity. We highlight in **bold** the results that show improvement compared to the vanilla Attention module

Dataset		ETTh1		Exchange	
Metric		MAE	MSE	MAE	MSE
Auto-Correction	96	**0.5160**	**0.5331**	0.4882	0.3837
LogSparse Attention	96	**0.5285**	**0.5349**	**0.4012**	**0.2340**
Patch Attention	96	**0.4357**	**0.4015**	**0.1594**	**0.0501**
Pyraformer Attention	96	**0.5110**	**0.5157**	0.5194	0.4060
Dttention	96	0.5648	0.6112	0.4739	0.3314
LSH Attention	96	0.5805	0.6701	0.5352	0.4877
ProSparse Attention	96	0.6591	0.8632	0.4840	0.3497
FA	96	0.7975	1.1465	1.1525	1.7961
Attention	96	0.5310	0.5741	0.4605	0.3402

Table 11 Time series prediction results of models based on Attention. The best results for each prediction length are highlighted in *bold*

Dataset		ETTh1	
Metric		MAE	MSE
PatchTST	48	**0.3932**	**0.3506**
	96	**0.4309**	**0.4027**
	336	**0.5126**	**0.5277**
Crossformer	48	0.5566	0.6142
	96	0.4454	0.4160
	336	0.5380	0.5800
Transformer	48	0.4745	0.4491
	96	0.5174	0.5291
	336	0.6636	0.8045
Pyraformer	48	0.4799	0.4682
	96	0.5358	0.5705
	336	0.6907	0.8723
Reformer	48	0.4986	0.4984
	96	0.5646	0.6062
	336	0.7690	1.0775
Logtrans	48	0.5800	0.6109
	96	0.5846	0.5922
	336	0.7202	0.9112
Informer	48	0.5099	0.5303
	96	0.6583	0.8626
	336	0.8301	1.3028

Table 12 The maximum memory occupation in MB of models during training

Metric	Maximum Memory Occupation (MB)		
O	48	96	336
Logtrans	1512	1554	1799
Informer	1762	1846	2075
Reformer	1639	1717	2110
PatchTST	1559	1586	1602
Crossformer	2489	2830	3748
Transformer	1738	1821	2038
Pyraformer	1659	1671	1745

Yang 2022; Malenica et al. 2021; Mintarsih et al. 2023), on the other hand, uses a fixed-size window that continuously slides over time for training and validation. Compared to simple time split validation, sliding window validation traverses the entire time series, altering the training and validation sets as it progresses. This method is particularly suited for slightly stationary or concept-drifting time series, especially in financial datasets.

We initially apply simple time split validation to all datasets to assess the predictive accuracy of the models. Unless otherwise specified, the batch size for all experiments was set to 16. To ensure robustness and reliability, all experimental results presented in this study are averaged over five trials. Consistent with previous work (Wu et al. 2021; Zhou et al. 2022; Woo et al. 2022; Zhou et al. 2021), we fixed the input length of the ILI dataset at $L = 36$ and set the output length O to 12, 24, 48 for short-term, regular-term, and long-term time series predictions, respectively. For the remaining datasets, we fixed the input length at $L = 96$ and set the output length O to 48, 96, 336 for short-term, regular-term, and long-term predictions, respectively. All hyperparameters for the deep learning models in the experiments were set according to the optimal configurations provided in their respective papers or code. For univariate time series prediction, we used the traditional ARIMA model as the baseline, while for multivariate time series prediction, we employed the VAR (Vector Autoregressive) model as the baseline.

The complete experimental results of the multivariate time series prediction task can be found in Table 13 in Appendix A. From the table, it is observed that among all the mentioned models, models utilizing the method of time series decomposition (e.g., Fedformer, ETSformer) and Transformer-based approaches (e.g., Crossformer, PatchTST) exhibit significantly superior prediction accuracy compared to the remaining models. Therefore, in Table 5, we only present the experimental results of these two types of methods. It is worth noting that, to further evaluate the performance of these methods on slightly stationary time series, in Table 5, we applied sliding window validation with a window size set to one-tenth of the dataset length for the financial exchange dataset.

In the multivariate time-series forecasting experiment, we observe several noteworthy findings. DLinear (Yun et al. 2019), which directly extracts temporal features using linear layers. demonstrates commendable prediction accuracy across the vast majority of datasets. Specifically, in Table 5, we observe that DLinear outperforms the suboptimal model PatchTST in terms of MAE and Fedformer in terms of MAPE for the ETTh1 dataset. It achieves improvements of 3.13% and 2.24% for prediction length of 336, respectively. Furthermore, it is worth noting that DLinear also boosted performance by 14.87% on R^2 for prediction lengths of 336 compared with the suboptimal model PatchTST. On the other hand, Crossformer (Zhang and Yan 2022) primarily focuses on capturing the correlation among time-series variables, making it particularly effective for datasets characterized by intricate variables. For instance, in the Electricity dataset, which comprises 321 variables, surpassing other datasets in terms of the number of variables, Crossformer exhibits a distinct accuracy advantage over the suboptimal model NS-Transformer. It achieves improvements of 10.93% and 17.70% in MSE and MAPE for prediction lengths of 48, respectively. However, it is worth noting that Crossformer's computational cost is considerably high, which may limit its feasibility for long-term time series prediction tasks due to resource constraints.

PatchTST (Nie et al. 2022), on the other hand, demonstrates enhanced suitability for datasets characterized by pronounced long-term periodicity and trends, such as exchange

rate and illness datasets. By dividing the input time series into fixed-length patches and emphasizing time-step dependencies between the patches, PatchTST is capable of uncovering long-term temporal relationships and achieves notable improvements in MSE and R^2 compared to other models. For example, in Table 5, PatchTST shows improvements of 27.30% and 17.49% in MSE and R^2, respectively, for a prediction length of 48 on the ILI dataset compared to the suboptimal model NS-Transformer. Similarly, on the Traffic dataset, PatchTST exhibits significant accuracy advantages over the suboptimal model NS-Transformer, achieving improvements of 11.46% in MAE for a prediction length of 336.

The complete results of the univariate time series prediction experiment can be found in Table 14 in Appendix A. The findings in Table 14, consistent with those in the multivariate time series forecasting experiment, indicate that models based on time-series decomposition methods and Transformer-based models exhibit superior prediction accuracy for univariate time series forecasting compared to other models. Therefore, in Table 6, we present the results of these two categories separately. Similarly, in Table 6, we applied sliding window validation with a window size set to one-tenth of the dataset length for the financial exchange dataset.

In the univariate time series prediction task, both NS-Transformer (Liu et al. 2022) and DLinear model demonstrate superior performance across various datasets. For instance, in the long-term prediction task on the ILI dataset, NS-Transformer outperforms the suboptimal model PatchTST with improvements of 9.54%, 25.32%, 46.47% in MAE, MSE and R^2, respectively. Comparing the results of multivariate and univariate time series prediction experiments, it is evident that NS-Transformer exhibits inferior performance in multivariate scenarios while demonstrating better performance in univariate scenarios. The main reason is that NS-Transformer incorporates non-stationary information from variables into the attention mechanism and adjusts the output of the attention mechanism based on the non-stationary information. However, in multivariate prediction tasks, an excessive amount of non-stationary information negatively impacts the output results of the attention mechanism.

Upon analyzing the empirical results of both multivariate and univariate time series prediction, we observe that DLinear consistently demonstrates superior accuracy compared to the majority of models. Interestingly, despite using a simpler linear layer for capturing time-step dependencies, DLinear outperforms numerous sophisticated models. This finding suggests that existing complex models may have limited capability in effectively capturing and utilizing time-step information, potentially leading to overfitting and noise interference. Additionally, compared to other models, DLinear makes more effective use of the sequential order of the input data, which is likely a key factor in its success.

6.4.2 Evaluation of information mining capabilities for complex models

Based on the analysis of results from both multivariate and univariate time series forecasting experiments, we argue that current complex models are prone to issues such as overfitting and still demonstrate deficiencies in capturing essential time-step dependencies. As a result, in this section, we will conduct an input shuffling experiment and extend the look-back window experiment to explore further and clarify the limitations of complex models.

Input Shuffling Experiment: In time series forecasting, the sequential order of time series often carries significant sequential information. A reliable model should be able to effec-

tively capture and understand the sequential order information. As a result, different sequential orders of input time series should lead to distinct predictions.

To evaluate the model's ability to capture sequential order information across time steps, we shuffle the input time series of the model. By comparing the model's prediction accuracy before and after shuffling, we can analyze the model's capacity to capture the sequential information of the time series.

The input length for the ILI dataset was set to $L = 36$, and the prediction length O was set to 24. Furthermore, for the other datasets, the input length was set to $L = 96$, and the prediction length was set to $O = 96$. All empirical results presented in this study were obtained by averaging the outputs of five randomized experiments. The prediction accuracy of the model is positively correlated with its capability to extract sequential information. Based on the analysis in Sect. 6.4.1, it is evident that DLinear and PatchTST exhibit significant accuracy advantages over various existing complex models (such as Autoformer, TDformer, ETSformer, etc.) across all datasets despite maintaining a simpler model structure. Table 7 shows that PatchTST exhibits the most notable decline in prediction accuracy after shuffling the input sequence across the ETTh1, ILI, and Exchange datasets. Similarly, DLinear also experiences a substantial decline in model accuracy following order shuffling in these datasets. On the other hand, complex models like Autoformer, TDformer, and others demonstrate minimal impact on their prediction accuracy or even show slight improvements after input shuffling. For instance, TDformer demonstrates a remarkable 33.87% improvement in MSE on the Exchange dataset after shuffling, whereas LSTnet exhibits a 4.83% enhancement in MSE on the ILI dataset after shuffling.

Based on the experimental results and the preceding analysis, we can conclude a positive correlation between the prediction accuracy of a model and its ability to capture the sequential order of the time series. Moreover, the existing complex models exhibit limited proficiency in effectively capturing the sequential order of the time series, which explains their comparatively lower prediction accuracy when compared to simpler models in both multivariate and univariate time series forecasting experiments. Apart from their weaker capability to capture the time series sequential information, we also suggest that the complex models face challenges such as overfitting and noise interference. Therefore, our subsequent experiment investigates whether these issues affect these models.

Extending the Lookback Window Experiment: To investigate the potential overfitting and noise interference of complex models in time series prediction, we conduct extending lookback window experiments. The size of the lookback window significantly influences the accuracy of time series prediction, as it determines the amount of historical information incorporated by the model. Generally, superior models achieve more accurate predictions using an extended lookback window. However, in the case of models affected by overfitting and noise interference, an extended lookback window introduces more noise. Due to overfitting, the model cannot effectively incorporate additional historical information, resulting in decreased prediction accuracy with an extended lookback window. Building on this concept, this section presents the extending lookback window experiment.

In our experiment, we set input time series lengths to 48, 96, 192 while keeping the output prediction length fixed at 96 for the time series. We then examine the changes in prediction accuracy for each model as the lookback window is extended. It is important to note that all results presented in this study are obtained by averaging the outputs of five randomized experiments.

The experimental results are depicted in Table 8. Complex models, including ETSformer, TDformer, and Autoformer, achieve their highest prediction accuracies only with the shortest input time series. In contrast, their accuracies gradually decline as the input time series lengthens. We believe this is primarily due to their limited ability to handle lengthy input time series, leading to longer sequences introducing more noise and exacerbating overfitting. However, PatchTST is the only model with optimal prediction accuracy for the longest input on both datasets. Additionally, on the ETTh1 dataset, DLinear demonstrates the highest prediction accuracy for the longest input time series.

Based on the analysis of the findings from the input shuffling experiment and extending the lookback window experiment, we propose that the insufficient prediction capabilities of current complex models can be primarily attributed to two key factors: (1) their limited ability to utilize the sequential information inherent in the time series effectively, and (2) their susceptibility to overfitting and noise interference.

6.4.3 Assessment of trend and season information mining capability

Based on the results of multivariate and univariate time series prediction experiments, it is apparent that models incorporating the method of time series decomposition tend to exhibit superior prediction accuracy. For models that employ time series decomposition methods, the trend term and the season term are the temporal terms they most aspire to extract. To further explore the performance of the aforementioned models in time series decomposition, trend, and seasonal term extraction, and overall prediction capabilities, we will train and test these models on an artificial dataset with the explicit trend and seasonal terms. Using this dataset, we can clearly observe the differences between the model outputs for the trend and seasonal terms and their corresponding true values. Next, we will provide an overview of the artificial dataset, including the process of generating the trend and seasonal terms.

Artificial Dataset: The artificial dataset consists of two terms: the trend term and the season term. Specifically, the trend term at the tth time step in each instance follows the equation $b(t) = \frac{1}{e^{\beta_0(t-\beta_1)}}$, while the season term at the tth time step in each instance follows the equation $s(t) = A_1 \cos(2\pi f_1 t) + A_2 \cos(2\pi f_2 t)$. The parameters used for generating the dataset are as follows: $\beta_0 = -0.2$, $\beta_1 = L$, $A_1 = A_2 = 0.15$, $f_1 = \frac{1}{10}$, and $f_2 = \frac{1}{13}$. In this experiment, the artificial dataset contains 16,847 instances. During the training phase, the instances in the training set are represented as $\boldsymbol{x_i} = [b(1) + s(1+i), \ldots, b(L+O) + s(L+O+i)] + \xi_i$, where ξ_i represents Gaussian-distributed noise with a mean of 0 and a standard deviation of 0.05.

In this experiment, we set both the input and output lengths of the time series to 96. The dataset was partitioned into training, validation, and test sets following a ratio of 6:2:2, utilizing a batch size of 16. The reported outcomes are derived from the mean results of five distinct and independent experiments.

Table 9 presents the predictions of different models incorporating time-series decomposition on the artificial dataset. Figure 18 shows the models' prediction accuracy for the trend terms. Among the models evaluated for trend term prediction on the artificial dataset, Fedformer, TDformer, and DLinear demonstrate superior performance. Fedformer is the best, which shows improvements of 3.14% and 6.18% in MAE and MSE for trend term prediction compared to TDformer, and 26.24% and 48.43% compared to DLinear. Fedformer, TDformer, and DLinear all use simple trend information extraction methods, such as mean

filtering and linear layers. This indicates that simple techniques can effectively extract trend information, while complex methods like N-BEATS and ETSformer may suffer from a certain degree of overfitting in accurately predicting trend terms.

The predictions for the season term are depicted in Fig. 19, and Table 9 provides an overview of the season term predictions for each model. Based on the aforementioned findings, it becomes apparent that the top-performing models for season-term prediction are Fedformer, TDformer, and ETSformer. Notably, all of these models employ frequency domain-based approaches to process season terms. Among the three methods, Fedformer has the strongest ability to extract seasonal terms. Compared to TDformer, Fedformer improves the MAE and MSE for season-term prediction by 26.65% and 43.07%, respectively. Similarly, compared to ETSformer, Fedformer shows improvements of 29.17% in MAE and 44.17% in MSE for season-term prediction.

Based on the aforementioned results of season term prediction, it becomes evident that models that utilize frequency domain-based methods outperform other models that rely on time domain-based methods. Therefore, it can be concluded that processing the Fourier components in the frequency domain is an effective approach for capturing the inherent periodic information present in the time series.

By integrating the predictions of both trend and season terms from the models, we notice a positive correlation between their predictive capabilities. Specifically, a more accurate prediction of season terms aligns with a better prediction of trend terms across the majority of models. We propose that this phenomenon can be attributed to most models' iterative decomposition of trend and season terms. The prediction accuracy of one term directly influences the subsequent decomposition of the remaining terms.

6.4.4 Assessing the efficacy of attention modules

Drawing upon the outcomes of multivariate and univariate time series prediction experiments, it becomes evident that most Transformer-based models demonstrate superior prediction performance compared to other models. The main difference between Transformer-based models lies in their internal Attention modules. Therefore, in the following we will analyze these Attention modules to explore their contributions to the models' overall predictive capabilities and evaluate their complexities.

To examine the impact of different Attention modules on the model's prediction capability, we employ the Transformer as the baseline framework in our experiments. Subsequently, we replace the Attention modules with the test objects to facilitate a comparative analysis of their effects on prediction accuracy. Additionally, we fine-tune the hidden layer dimensions of the test objects to ensure consistent space complexity. This adjustment is necessary because LogSparse Attention, LSH Attention, and ProSparse Attention have been optimized in terms of space complexity compared to the vanilla Attention modules. To ensure a fair comparison, we aligned the space complexity of the test subjects to a similar scale.

We keep a consistent batch size of 8 and set the model with an input time series length $L = 96$ and a predicted time series length $O = 96$. The rest of the experimental configurations align with those utilized in the multivariate time series prediction experiments described in Sect. 6.4.1.

Table 10 demonstrates that Patch Attention in PatchTST significantly improves the prediction capability of the Attention mechanism. When compared with the vanilla Attention module, Patch Attention exhibits substantial accuracy enhancements on the ETTh1 dataset, with improvements of 17.95% and 30.06% in MAE and MSE, respectively. Additionally, Patch Attention achieves an impressive 85.27% improvement in MSE on the Exchange dataset. These results indicate that aggregating short-term time steps collectively significantly enhances the capacity of the attention mechanism to capture temporal information within a time series.

Furthermore, our experiments in Sect. 6.4.1 reveal the notable performance of Fedformer and Crossformer. However, it is worth noting that their attention modules, FA and DAttention, exhibit subpar performance in this specific experiment. We believe that the primary factor contributing to this outcome is the absence of space complexity limitations on their attention modules during the experiments conducted in Sect. 6.4.1. However, the space complexity of FA and DAttention significantly surpasses that of the vanilla Attention module. To maintain consistency in space complexity across all experimental subjects, the dimensions of the hidden layer in FA and DAttention are compressed to a smaller scale in this experiment, resulting in their subpar prediction accuracies.

According to the introduction in Sect. 4, numerous models based on the Attention mechanism have optimized their complexity to meet the demands of long-term time series forecasting tasks. Consequently, we delve into the complexity of these optimized models and examine their corresponding prediction performance.

We uniformly set the batch size to 8. All optimized models are trained on the ETTh1 dataset with an input time series length of $L = 96$ and predicted time series lengths of $O = \{48, 96, 336\}$. Following the original works (Nie et al. 2022; Zhang and Yan 2022; Liu et al. 2021; Kitaev et al. 2020; Li et al. 2019; Zhou et al. 2021), the remaining hyperparameters of these models are set to their optimal values.

Next, we will analyze the performance of these models from two perspectives: time complexity and space complexity. Firstly, regarding time complexity, we observe that Reformer demonstrates superior prediction performance while maintaining a low time complexity. This observation is illustrated in Fig. 20 and summarized in Table 11. Specifically, Reformer exhibits the shortest test time, reducing 10.14% compared to LogTrans, the model with the second-best test time. Additionally, on the ETTh1 dataset, Reformer showcases significant improvements in MAE by 14.03% and MSE by 18.42% compared to LogTrans. One of the distinctive features of Reformer is incorporating the Locality-Sensitive Hashing (LSH) Attention mechanism. This mechanism partitions the time steps into groups using a hash function, limiting the computation of similarity relationships to time steps within the same group. As a result, the complexity of the Attention mechanism is effectively reduced.

When considering space complexity, our analysis reveals that models utilizing the patch slicing method, such as PatchTST and Pyraformer, demonstrate distinct advantages while maintaining high prediction accuracy. In particular, these models demonstrate even more pronounced benefits in space complexity when dealing with long-term time series. In Fig. 21 and Table 12, we observe that for a prediction length of 336, PatchTST exhibits a memory occupation that is 10.93% lower than that of LogTrans. Additionally, PatchTST showcases significant improvements in MAE by 28.83% and in MSE by 42.09% compared

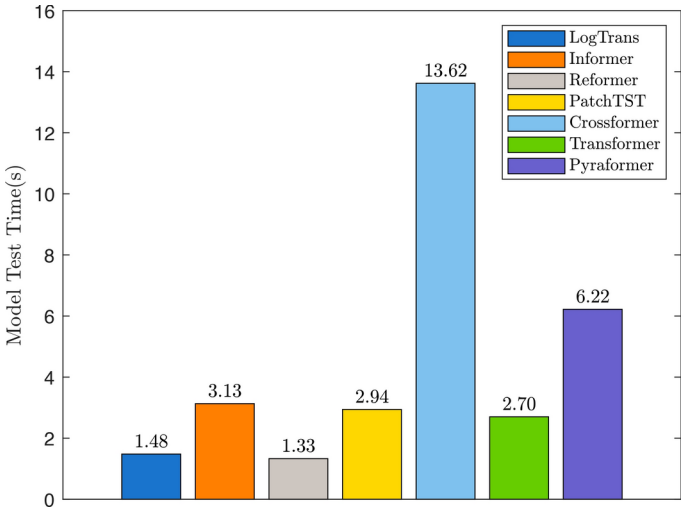

Fig. 20 Inference time. The vertical axis represents the inference time of different models for the whole test dataset. We report the mean inference time for each model across three scenarios with prediction lengths of 48, 96, 336

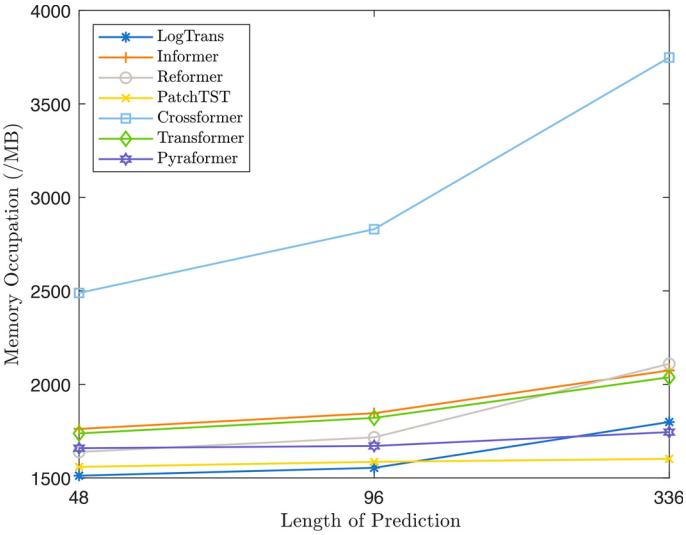

Fig. 21 Maximum memory occupation during training. The horizontal axis represents the prediction length, and the vertical axis represents the maximum memory occupation

to LogTrans on the ETTh1 dataset. Similarly, for a prediction length of 336, Pyraformer exhibits a memory occupation that is 3.00% lower than LogTrans. Moreover, Pyraformer surpasses LogTrans in prediction accuracy with a 4.10% improvement in MAE and a 4.27% improvement in MSE on the ETTh1 dataset.

Through an analysis of the prediction ability and complexity of the Attention module, we have observed that the patch-slicing approach effectively enhances the module's prediction capability. Additionally, this approach significantly reduces the complexity of the Attention module by processing shorter lengths of time series data. The benefits of complexity optimization through patch slicing become more apparent as the length of the input time series increases. While other sparse methods may also optimize the module's complexity, they often result in information loss, potentially impacting the prediction performance to some extent.

7 Conclusion and future directions

In this paper, we comprehensively examine the principal models used for time series forecasting during the 10-year period from 2014 to 2024. Our focus is on analyzing the underlying processing logic employed by these models. Specifically, we investigate the correlation relationships between time steps and the interdependencies among time series variables. We then present approaches explicitly tailored for long-term time series forecasting tasks. Furthermore, we provide a concise overview of commonly employed loss functions in time series forecasting. In the section dedicated to comparative experiments, we conduct thorough investigations into the performance of the principal models for both univariate and multivariate time series prediction tasks. We meticulously explore and analyze the challenges of overfitting and noise interference in existing complex models. Throughout our investigation, we identify the distinct advantages of employing simple methods for trend term prediction and employing methods based on frequency domain for season term prediction. Moreover, we validate the effectiveness of the patch-slicing method applied to the Attention mechanism by assessing its performance in terms of prediction accuracy and complexity. Drawing on a comprehensive analysis of existing research findings, we propose potential research directions within the field of time series prediction. These directions encompass the following areas:

Mining Dependencies among Time Steps: Existing models have utilized time series decomposition methods to extract temporal information such as trends and seasonal patterns, but these models still face limitations in effectively mining this information. Exploring seasonal information in the frequency domain remains a promising direction for future research. Additionally, capturing time-scale information, non-stationary patterns, and other relevant time series information is becoming increasingly important.

Mining Relationships among Time-Series Variables: In datasets related to traffic or weather, there is an inherent correlation among time-series signals from different sensors. Therefore, exploring the correlation information between time-series variables is crucial for such datasets. However, existing models have paid less attention to extracting relationships among time-series variables. Effectively utilizing these relationships remains an important research direction.

Complexity Optimization of Models for Long-term Time-Series Prediction: As the length of the predicted sequence increases, the computational cost of models becomes more burdensome. This issue is particularly prominent in Transformer-based models. Although some algorithms have explored complexity optimization for the Attention mechanism, they often face the challenge of information loss. Therefore, optimizing the computational cost of the

Attention mechanism without sacrificing prediction accuracy remains a key area for further exploration and research.

Investigation of Overfitting in Complex Models for Time-Series Prediction: Relevant experiments in this paper indicate that existing complex models often suffer from overfitting and noise interference, leading to prediction performance that is sometimes inferior to simpler models. Therefore, mitigating overfitting and reducing the impact of noise and outliers remain important research directions.

Exploring Flexible Prediction Models: Current time series prediction models are often constrained by the fixed length of input and output sequences, meaning they can only be used for predetermined prediction lengths. If the prediction length or position changes, these models must be redesigned and retrained. There is a critical need to develop time series prediction models that can adapt to flexible prediction scenarios, accommodating changes in prediction length and position without requiring extensive redesign or retraining.

Addressing the non-stationarity of time series: Although deep learning-based models do not require input sequences to be stationary, the non-stationarity of time series, particularly the differences in statistical characteristics between the training and test sets, can often impact the model's predictive accuracy. Therefore, exploring and modeling changes in the statistical properties of time series is a crucial research direction that can significantly enhance predictive accuracy. Research in this area is still in its early stages.

Probabilistic Forecasting: Probabilistic forecasting (Gneiting et al. 2007; Salinas et al. 2020), which involves estimating the future probability distribution of a time series based on its past behavior, is a key area for optimizing predictive models. This approach not only improves the robustness and interpretability of models but also equips them with capabilities for risk assessment and management, which are essential in many real-world applications. Currently, deep learning models for probabilistic forecasting face several challenges, including the development of more effective loss functions, improved evaluation metrics, and novel model architectures tailored for random processes and probability estimation.

Additional information and experimental results

In this section, we provide additional information and experimental results that support the main body of the paper. Section A.1 presents the complete results of the multivariate and univariate time series forecasting experiments.

Results of multivariate and univariate time series forecasting experiments

Table 13 Multivariate time series forecasting results

Model	Data	ETTh1		Exchange		ILI		Traffic		Electricity	
	Metric	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE
Model	Data	ETTh1		Exchange		ILI		Traffic		Electricity	
	Metric	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE
ARIMA	Short	1.2874	2.3800	1.1574	3.5088	1.0836	2.5038	0.7340	0.8655	1.0963	1.8875
	Regular	1.4327	2.8096	1.3296	3.4915	1.0283	2.3477	0.7089	0.8121	0.9924	1.5787
	Long	1.0144	1.4551	1.1372	2.2342	0.9696	1.8138	0.7212	0.9080	0.9707	1.5114
CNN_1D	Short	0.5794	0.6096	0.7574	0.7717	1.1983	2.7567	0.7619	1.1303	0.3990	0.3258
	Regular	0.6664	0.8049	0.8277	0.9209	1.2479	2.9318	0.7632	1.1347	0.5737	0.5905
	Long	0.8382	1.1594	0.8659	1.0250	1.3800	3.3879	0.7487	1.0399	–	–
TCN	Short	0.6655	0.8862	0.1834	0.0617	1.1992	2.9021	0.5308	0.7229	0.3129	0.2719
	Regular	0.4940	0.4853	0.2512	0.1067	1.3409	3.3631	0.4181	0.5460	0.3670	0.3415
	Long	0.6410	0.6933	0.3238	0.1769	1.2214	2.8402	0.5130	0.7605	0.3648	0.2775
DeepAR	Short	0.9030	1.3482	0.8777	1.1188	1.2294	2.8305	0.8119	1.1921	0.8560	1.0811
	Regular	0.9240	1.3994	0.8975	1.2054	1.2870	3.0435	0.9628	1.6597	0.8409	1.0337
	Long	1.1843	2.2798	1.1901	2.2022	1.3712	3.3984	0.8752	1.3683	–	–
GRU	Short	0.7803	1.0360	0.7621	0.0884	1.1508	2.4973	1.2360	2.0568	0.7838	0.8927
	Regular	0.7493	0.9711	0.7812	0.8324	1.1799	2.0048	1.2360	2.0568	0.7829	0.8914
	Long	0.7687	1.0085	0.7398	0.7430	1.2922	3.0311	0.7532	1.0687	0.7838	0.8884
LSTnet	Short	0.8130	1.1303	0.3699	0.2598	1.1298	2.0822	0.6480	0.8468	0.6181	0.6353
	Regular	0.7601	0.9960	0.4955	0.4042	1.2103	2.7753	0.6793	0.9148	0.6986	0.7821
	Long	0.7644	0.9953	0.7643	0.8241	1.2546	2.7236	–	–	–	–
Trans-former	Short	0.4745	0.4491	0.2807	0.1393	1.0202	2.1587	0.3148	0.3415	0.3969	0.3283
	Regular	0.5174	0.5291	0.3404	0.1804	1.1004	2.3836	0.3896	0.5322	0.3996	0.3080
	Long	0.6636	0.8045	0.9066	1.2028	1.2277	2.8575	0.7488	1.0401	0.7871	1.0231
LogTrans	Short	0.5362	0.5482	0.3736	0.2137	0.7762	1.5743	0.5296	0.7651	0.3795	0.2930
	Regular	0.6392	0.7138	0.3949	0.2508	0.7972	1.6156	0.3617	0.4842	0.4052	0.3248
	Long	0.8206	1.1379	0.7025	0.7366	0.8755	1.7925	0.3490	0.4619	0.4057	0.3242
Reformer	Short	0.4986	0.4984	0.6313	0.5838	1.1229	2.4342	0.3855	0.4504	0.4408	0.3855
	Regular	0.5646	0.6062	0.8194	0.9184	1.2586	2.9970	0.7644	1.1385	0.5078	0.4575
	Long	0.7690	1.0775	0.9664	1.2704	1.4413	3.7349	0.7488	1.0402	0.8196	1.0802
Informer	Short	0.5023	0.5197	0.3635	0.2122	0.8486	1.6803	0.3533	0.3963	0.4709	0.4322
	Regular	0.6482	0.8527	0.4379	0.3113	0.9692	2.1427	0.5002	0.7643	0.4840	0.4424
	Long	0.6687	0.8390	0.6804	0.6786	1.0854	2.5005	0.7548	1.1833	0.7731	1.0135
Auto-former	Short	0.5041	0.4961	0.1812	0.0605	0.7902	1.3502	0.3175	0.3744	0.3335	0.1922
	Regular	0.4970	0.4796	0.2177	0.0886	0.8397	1.4801	0.3419	0.4202	0.3089	0.2016
	Long	0.5766	0.6128	0.3822	0.2579	0.8438	1,5455	0.3393	0.4095	0.3577	0.2609
Fedformer	Short	0.4566	0.4189	0.1815	0.0607	0.6483	0.9915	0.3169	0.3693	0.2973	0.1877
	Regular	0.4935	0.4754	0.2174	0.0877	0.8024	1.3779	0.3172	0.3699	0.3009	0.1924
	Long	0.5714	0.5979	0.3694	0.2429	0.7911	1.3501	0.3234	*0.3876*	0.4646	0.3942
ETS-former	Short	0.5026	0.5423	0.1159	0.0274	0.8059	1.3679	0.4652	0.6078	0.3185	0.2163
	Regular	0.5414	0.6074	0.1693	*0.0557*	0.9788	1.9403	0.4721	0.6234	0.3370	0.2430
	Long	0.6140	0.7113	*0.3211*	0.1937	1.0026	1.9683	0.4800	0.6799	0.3642	0.2824
TDformer	Short	0.5810	0.6334	0.5819	0.5126	1.1809	2.6896	0.3602	0.3676	0.4169	0.3459
	Regular	0.5963	0.6530	0.7822	0.8633	1.2581	2.9438	0.3677	0.5183	0.4174	0.3463
	Long	0.7826	1.0615	0.8391	1.0206	1.3461	3.3141	0.3816	0.5569	0.4575	0.4160
AST	Short	0.4410	0.4457	0.5003	0.3945	1.0302	2.3503	0.3106	0.3933	0.3696	0.2816
	Regular	0.4998	0.5324	0.6505	0.6381	1.1992	2.9159	0.3180	0.4133	0.3706	0.2844
	Long	0.6059	0.7076	0.8950	1.1737	1.2093	2.9306	0.3572	0.4899	0.3892	0.3113

Table 13 (continued)

Model	Data	ETTh1		Exchange		ILI		Traffic		Electricity	
	Metric	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE
Aliformer	Short	0.7668	1.0024	0.5696	0.5009	1.1775	2.6960	0.7502	1.0473	0.8223	0.9717
	Regular	0.8467	1.2102	0.8112	0.8993	1.2329	2.8906	0.7632	1.1348	0.8250	0.9845
	Long	0.7632	0.9963	1.0791	1.5914	1.4442	3.7724	0.7487	1.0398	0.8221	0.9674
NS-Transformer	Short	0.4306	0.4197	0.1199	0.0285	*0.5493*	*0.7300*	0.2910	0.3491	0.2646	*0.1637*
	Regular	0.4812	0.4946	0.1695	0.0565	*0.7099*	*1.1508*	0.3046	0.3718	0.2771	*0.1784*
	Long	0.6070	0.7074	0.3403	0.2142	*0.7579*	1.3199	*0.3202*	0.4008	0.3024	*0.2052*
Crossformer	Short	0.4132	0.3788	0.2005	0.0723	0.9512	2.0541	**0.2453**	**0.3085**	**0.2342**	**0.1458**
	Regular	0.4522	0.4393	0.2575	0.1118	1.0898	2.4253	**0.2718**	**0.3452**	**0.2485**	**0.1647**
	Long	0.6319	0.8007	0.5546	0.5053	1.1988	2.8067	–	–	*0.2882*	**0.2022**
DLinear	Short	**0.3979**	**0.3663**	**0.1109**	**0.0259**	0.6440	0.9567	0.3738	0.5417	0.2676	0.2004
	Regular	**0.4360**	**0.4229**	0.1775	0.0570	0.7699	1.2162	0.3478	0.5075	0.2741	0.1992
	Long	**0.5052**	**0.5197**	**0.3117**	**0.1648**	0.7994	*1.2452*	0.3292	0.4681	0.2916	0.2095
Scaleformer	Short	0.5480	0.6227	0.1858	0.0635	0.6766	1.0450	0.3054	0.3572	0.3390	0.2359
	Regular	0.4643	0.4324	0.1761	0.0600	0.7967	1.4051	0.3159	0.3723	0.3515	0.2607
	Long	0.6311	0.7666	0.3905	0.2646	0.8967	1.6122	0.3387	0.4056	0.3749	0.2839
Pyraformer	Short	0.4799	0.4682	0.3796	0.2333	0.8096	1.6223	0.4478	0.5959	0.3821	0.2972
	Regular	0.5358	0.5705	0.5940	0.5645	1.0740	1.9305	0.4984	0.6638	0.4037	0.3323
	Long	0.6907	0.8723	0.7570	0.8616	1.0720	2.3328	–	–	0.4242	0.3640
PatchTST	Short	*0.4008*	*0.3674*	*0.1147*	*0.0283*	**0.4584**	**0.5307**	*0.2654*	*0.3379*	*0.2574*	0.1860
	Regular	*0.4401*	*0.4253*	**0.1609**	**0.0520**	**0.6180**	**0.9902**	*0.2755*	*0.3615*	*0.2631*	0.1891
	Long	*0.5215*	*0.5509*	0.3308	0.2087	**0.7424**	**1.1373**	**0.2835**	**0.3857**	**0.2861**	0.2092
Triformer	Short	0.7668	1.0024	0.1678	0.0510	1.0915	2.4694	–	–	–	–
	Regular	0.4679	0.4429	0.1759	0.0560	1.1890	2.7092	–	–	–	–
	Long	0.5562	0.5877	0.5116	0.4676	1.2457	2.9084	–	–	–	–
TFT	Short	0.4594	0.4663	0.1995	0.0694	1.3075	3.3835	–	–	0.3568	0.2721
	Regular	0.5515	0.5552	0.2317	0.0928	1.2891	3.1048	–	–	0.4493	0.4010
	Long	0.5301	0.5571	0.6355	0.6263	1.5055	4.1189	–	–	–	–
SSDNet	Short	0.8618	1.2990	0.4704	0.3884	1.3896	3.3878	–	–	–	–
	Regular	0.7075	0.8740	0.3986	0.2668	1.4744	3.7407	–	–	–	–
	Long	–	–	0.6555	0.6487	1.7026	4.7365	–	–	–	–
Autoencoder	Short	0.7606	0.9954	0.1854	0.0633	1.2128	2.8800	0.7585	1.1517	0.8219	0.9705
	Regular	0.5279	0.5809	0.3018	0.1569	1.2891	3.2390	0.7601	1.1585	0.8221	0.9707
	Long	0.7645	0.9986	1.0101	1.4107	1.4179	3.7566	0.7488	1.0398	0.8227	0.9695
SAE	Short	0.4210	0.3778	0.8057	0.8961	1.1582	2.5776	0.3417	0.4537	0.3831	0.2984
	Regular	0.7381	0.9107	1.0898	1.6129	1.2364	2.9496	0.3521	0.4877	0.4411	0.4289
	Long	0.7640	1.0732	1.1855	1.9782	1.3723	3.5140	0.7485	1.0401	0.8189	1.0621
N-BEATS	Short	0.4579	0.4353	0.1151	0.0269	0.9474	1.8391	0.3886	0.6086	0.3760	0.2855
	Regular	0.6128	0.7949	*0.1629*	0.0581	1.1330	2.5371	0.6862	1.0418	0.6903	0.8176
	Long	0.6682	0.8908	0.3290	*0.1797*	1.3070	3.1641	0.7400	1.1278	0.7374	0.9044

The best results are highlighted in *bold* and the second best results are highlighted with a *underline*.. "–" indicates out of memory. 'Short', 'Regular', 'Long' denotes the output length O for short-term, regular-term and long-term time series prediction

Table 14 Univariate time series forecasting results

Model	Data	ETTh1		Exchange		ILI		Traffic		Electricity	
	Metric	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE
ARIMA	Short	0.5341	0.2904	0.8298	0.6908	2.8446	8.1104	1.5089	2.9879	0.8334	0.9120
	Regular	0.6769	0.4613	1.1036	1.2307	2.6658	7.1389	1.3781	2.4447	0.7523	0.7447
	Long	0.7517	0.5809	1.2046	1.4630	2.0703	4.7428	1.2276	2.1735	0.7134	0.6477
CNN-1D	Short	0.5670	0.4782	0.3974	0.2305	0.9066	1.1410	0.8560	1.0486	0.4783	0.3992
	Regular	0.5670	0.4779	0.3071	0.1470	1.6404	3.1585	0.8193	0.9720	0.5042	0.4420
	Long	0.5506	0.4552	0.6736	0.7026	1.3437	3.3111	0.8555	1.0478	0.5618	0.5150
TCN	Short	0.3322	0.2075	0.3275	0.1818	1.4640	2.6350	0.4934	0.4646	0.8229	1.1922
	Regular	0.3619	0.2383	0.4640	0.3343	1.5244	2.8274	0.6185	0.6433	0.7375	0.9382
	Long	0.3683	0.2410	0.6464	0.6255	1.5310	2.7988	0.4309	0.3664	0.8169	1.0796
DeepAR	Short	0.4840	0.3610	0.9770	1.3248	1.4786	3.6056	1.2122	2.2182	1.1570	1.1570
	Regular	0.6246	0.5837	0.9870	1.3568	1.4792	2.7966	1.2481	2.3466	1.1761	2.2091
	Long	0.7266	0.8006	1.0411	1.5027	1.8888	4.1431	–	–	–	–
GRU	Short	0.7675	0.7355	0.8287	1.0144	1.3457	2.3184	1.0187	1.4046	0.8059	1.0043
	Regular	0.8253	1.0101	0.8253	1.0101	1.3554	2.3010	1.0186	1.4047	0.8048	1.0039
	Long	0.8268	1.0316	0.8268	1.0316	1.3623	2.2733	1.0218	1.4094	0.8020	1.0042
LSTnet	Short	0.6540	0.5656	0.5887	0.4949	1.0897	1.5923	0.7020	0.7594	0.6927	0.6999
	Regular	0.6546	0.5675	0.6168	0.5340	1.2049	1.8884	0.7516	0.8135	0.7510	0.8845
	Long	0.6601	0.5914	0.9144	1.2287	1.2914	2.0774	–	–	–	–
Trans-former	Short	0.3489	0.2109	0.3423	0.1711	0.7755	0.8995	0.7969	0.9045	0.3996	0.2803
	Regular	0.3871	0.2612	0.3652	0.2087	0.8634	1.0702	0.7809	0.8834	0.4730	0.3934
	Long	0.5154	0.4288	0.5294	0.3849	0.9816	1.2789	0.8414	1.0310	0.4647	0.3873
LogTrans	Short	0.3415	0.1833	0.3250	0.1686	0.8294	0.8792	1.1643	1.8174	0.4389	0.3623
	Regular	0.3571	0.2064	0.3857	0.2251	0.8593	0.9287	1.0159	1.4864	0.4432	0.3545
	Long	0.4776	0.3544	0.4997	0.3649	0.8999	0.9355	0.8352	1.0665	0.4696	0.4115
Reformer	Short	0.4624	0.3293	0.5477	0.4349	0.9098	1.2002	0.8658	1.0754	0.5157	0.4667
	Regular	0.5138	0.4088	0.5974	0.5264	1.1512	1.8010	0.9858	1.3908	0.5350	0.4957
	Long	0.5397	0.4395	0.7098	0.7446	1.3585	2.2942	0.9435	1.3019	0.5228	0.4851
Informer	Short	0.3855	0.2463	0.5156	0.4046	0.9464	1.2762	1.2931	2.3961	0.5526	0.5625
	Regular	0.4711	0.3706	0.6412	0.5808	1.0865	1.5350	1.0285	1.5467	0.5740	0.5632
	Long	0.5258	0.4454	0.7829	0.8706	1.2137	1.7989	0.8971	1.2184	0.7572	0.9080
Auto-former	Short	0.3437	0.2138	0.1946	0.0638	0.4348	0.3105	0.4294	0.2901	0.4493	0.3648
	Regular	0.3634	0.2317	0.2645	0.1154	0.5409	0.4231	0.4619	0.3388	0.4126	0.2988
	Long	0.4041	0.2819	0.4783	0.3768	0.6079	0.4947	0.4274	0.2969	0.5767	0.5460
Fedformer	Short	0.3325	0.1992	0.2194	0.0798	0.4053	*0.2466*	0.4210	0.2826	0.3732	0.2480
	Regular	0.3555	0.2220	0.2682	0.1212	0.4493	0.2987	0.4261	0.2891	0.3250	*0.1967*
	Long	0.4001	0.2777	0.4739	0.3685	0.5595	0.4466	0.4281	0.2938	0.6163	0.6571
ETS-former	Short	0.3329	0.1994	0.3423	0.1711	0.4790	0.3744	0.5593	0.4885	0.5673	0.5584
	Regular	0.3604	0.2298	0.2178	0.0735	0.5411	0.4164	0.5672	0.5067	0.6315	0.6620
	Long	0.4194	0.3005	*0.3925*	*0.2265*	0.6302	0.5230	0.5756	0.5355	0.6709	0.7967
TDformer	Short	0.4272	0.3113	0.9326	1.2894	2.0571	4.9538	0.4244	0.3633	0.4163	0.3130
	Regular	0.4779	0.3727	1.1635	1.7472	1.1753	2.6595	0.4572	0.4127	0.4716	0.3981
	Long	0.4715	0.3780	1.2804	2.1319	2.1227	5.1123	0.4590	0.4144	0.4954	0.4201
AST	Short	0.5517	0.6820	0.4682	0.3521	1.1750	3.0143	*0.3203*	0.4072	0.3703	0.2837
	Regular	0.5598	0.6656	0.8395	0.9553	1.1257	2.5269	*0.3291*	0.4288	0.3786	0.2925
	Long	0.6219	0.7784	0.9843	1.3628	1.3748	3.8622	*0.3398*	0.4497	0.3952	0.3203

Table 14 (continued)

Model	Data	ETTh1		Exchange		ILI		Traffic		Electricity	
	Metric	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE
Aliformer	Short	1.0248	1.2065	0.4364	0.2761	1.4713	2.6747	0.9729	1.1016	0.8180	1.0663
	Regular	0.8248	1.1298	0.5353	0.3708	1.5087	2.7804	0.9811	1.1930	0.8166	1.0659
	Long	0.9166	0.9959	1.3951	2.3686	1.4399	3.7476	0.9827	1.1928	0.8190	1.0657
NS-Transformer	Short	0.3163	0.1821	*0.1541*	*0.0404*	*0.3869*	0.2530	0.4654	0.3691	*0.3133*	**0.1869**
	Regular	0.3471	0.2152	0.2235	0.0911	*0.4107*	*0.2840*	0.8804	1.1424	*0.3189*	0.1984
	Long	0.5269	0.4747	0.4828	0.3880	**0.4165**	**0.2569**	0.9661	1.2290	**0.3099**	**0.1743**
Crossformer	Short	0.3146	0.1829	0.2728	0.1187	1.1821	2.1848	0.3915	0.2846	**0.3033**	*0.2277*
	Regular	0.3404	0.2120	0.3820	0.2287	1.1480	1.5548	0.3784	0.2790	**0.2796**	**0.1818**
	Long	0.4192	0.2960	0.8162	1.0097	1.1606	1.5138	–	–	*0.3350*	*0.2443*
Dlinear	Short	**0.2987**	*0.1729*	**0.1413**	**0.0339**	0.6970	0.7121	0.4687	0.4177	0.6975	0.9390
	Regular	*0.3214*	*0.1952*	**0.1953**	**0.0657**	0.9373	1.1032	0.4398	0.3811	0.5960	0.7090
	Long	*0.3712*	**0.2455**	**0.3847**	**0.2209**	0.9340	1.0252	0.4201	0.3443	0.4864	0.4893
Scaleformer	Short	*0.3003*	**0.1679**	0.1674	0.0471	0.4511	0.3643	**0.3051**	**0.1912**	0.4147	0.3031
	Regular	**0.3131**	**0.1734**	0.2249	0.0862	0.5383	0.4665	**0.3163**	**0.1979**	0.4337	0.3384
	Long	0.4234	0.3099	0.4641	0.3559	0.5628	0.4290	**0.2908**	**0.1861**	0.4706	0.4004
Pyraformer	Short	0.4021	0.2615	0.3448	0.1914	1.1677	1.7479	0.4151	0.3509	0.4523	0.3564
	Regular	0.5803	0.5474	0.3467	0.1873	1.2846	2.2077	0.4145	0.3491	0.4717	0.3892
	Long	0.5070	0.4283	0.6196	0.5879	1.3637	2.2317	0.4308	0.3706	0.5145	0.4510
PatchTST	Short	0.3031	0.1774	0.1566	0.0425	**0.2874**	**0.1614**	0.3514	*0.2488*	0.5249	0.6473
	Regular	0.3276	0.2021	*0.2124*	*0.0789*	**0.3542**	**0.2233**	0.3610	*0.2606*	0.4437	0.4893
	Long	**0.3693**	*0.2478*	0.4163	0.2917	*0.4604*	*0.3440*	0.3700	*0.2722*	0.3770	0.3589
Triformer	Short	0.3149	0.1866	0.2410	0.0935	1.7262	3.4845	–	–	–	–
	Regular	0.3334	0.2071	0.3398	0.1900	1.7423	3.4663	–	–	–	–
	Long	0.4041	0.2843	0.6297	0.7034	1.4855	2.5357	–	–	–	–
TFT	Short	0.3852	0.2125	0.5234	0.4296	1.2492	2.9732	–	–	0.3568	0.2721
	Regular	0.3975	0.2325	0.6230	0.5707	1.2936	3.1257	–	–	0.4369	0.3828
	Long	0.6813	0.7711	0.5307	0.4244	1.3590	3.3232	–	–	–	–
SSDNet	Short	0.3994	0.2447	0.5471	0.4941	1.2719	2.0677	–	–	–	–
	Regular	0.4760	0.3493	0.6099	0.5869	1.3293	2.1916	–	–	–	–
	Long	–	–	0.8132	1.0029	1.5207	2.7740	–	–	–	–
Autoencoder	Short	0.3441	0.2095	0.4243	0.2946	1.2054	1.9340	0.7613	1.1320	0.8156	1.0888
	Regular	0.3615	0.2281	0.5458	0.4641	1.3731	2.3999	0.9632	1.1615	0.8194	1.0719
	Long	0.5640	0.4772	0.7607	0.8692	1.4058	3.6954	0.9643	1.1710	0.8179	1.0794
SAE	Short	0.4854	0.3539	0.3752	0.2119	1.1519	2.4958	0.3895	0.3295	0.4836	0.4035
	Regular	0.5297	0.4225	0.4577	0.3130	1.6535	3.2258	0.4312	0.3777	0.5179	0.4563
	Long	0.7807	1.0845	0.7391	0.8023	1.8888	4.1431	0.4527	0.4081	0.5660	0.5481
N-BEATS	Short	0.3119	0.1805	0.2950	0.1354	1.2647	1.8155	0.4647	0.3671	0.5377	0.5074
	Regular	0.3660	0.2341	0.3245	0.1733	1.7303	3.4102	0.8803	1.1347	0.6250	0.6499
	Long	0.4114	0.2837	0.4720	0.3880	1.5454	2.6395	0.9589	1.2246	0.6735	0.7926

The best results are highlighted in *bold* and the second best results are highlighted with a *underline*. "–" indicates out of memory. 'Short', 'Regular', 'Long' denotes the output length O for short-term, regular-term and long-term time series prediction

Funding This work was supported by the National Natural Science Foundation of China (Grant No. 62206178 and 72301180) and the Stable Support Plan for Higher Education Institutions in Shenzhen (Project No. 20231121221536001). The authors have no relevant financial or non-financial interests to disclose.

References

Achiam J, Adler S, Agarwal S, Ahmad L, Akkaya I, Aleman FL, Almeida D, Altenschmidt J, Altman S, Anadkat S, et al (2023) Gpt-4 technical report. arXiv preprint arXiv:2303.08774

Al-Tahan H, Mohsenzadeh Y (2021) Clar: Contrastive learning of auditory representations. In: International conference on artificial intelligence and statistics, pp. 2530–2538. PMLR

Amalou I, Mouhni N, Abdali A (2022) Multivariate time series prediction by RNN architectures for energy consumption forecasting. Energy Rep 8:1084–1091

Ariyo AA, Adewumi AO, Ayo CK (2014) Stock price prediction using the arima model. In: 2014 UKSim-AMSS 16th international conference on computer modelling and simulation, pp. 106–112. IEEE

Armstrong JS (2001) Principles of forecasting: a handbook for researchers and practitioners, vol 30. Springer, Berlin

Asadi R, Regan AC (2020) A spatio-temporal decomposition based deep neural network for time series forecasting. Appl Soft Comput 87:105963

Bai S, Kolter JZ, Koltun V (2018) An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. arXiv preprint arXiv:1803.01271

Bell WR, Hillmer SC (1984) Issues involved with the seasonal adjustment of economic time series. J Bus Econ Stat 2(4):291–320

Binkowski M, Marti G, Donnat P (2018) Autoregressive convolutional neural networks for asynchronous time series. In: International conference on machine learning, pp. 580–589. PMLR

Cai L, Janowicz K, Mai G, Yan B, Zhu R (2020) Traffic transformer: capturing the continuity and periodicity of time series for traffic forecasting. Trans GIS 24(3):736–755

Cao D, Wang Y, Duan J, Zhang C, Zhu X, Huang C, Tong Y, Xu B, Bai J, Tong J (2020) Spectral temporal graph neural network for multivariate time-series forecasting. Adv Neural Inf Process Syst 33:17766–17778

Celeghini E, Gadella M, Olmo MA (2021) Hermite functions and Fourier series. Symmetry 13(5):853

Chakraborty K, Mehrotra K, Mohan CK, Ranka S (1992) Forecasting the behavior of multivariate time series using neural networks. Neural Netw 5(6):961–970

Challu C, Olivares KG, Oreshkin BN, Ramirez FG, Canseco MM, Dubrawski A (2023) Nhits: Neural hierarchical interpolation for time series forecasting. In: Proceedings of the AAAI conference on artificial intelligence, vol. 37, pp. 6989–6997

Chen Z, Ma M, Li T, Wang H, Li C (2023) Long sequence time-series forecasting with deep learning: a survey. Inf Fus 97:101819

Chen Y, Huang J, Xu H, Guo J, Su L (2023) Road traffic flow prediction based on dynamic spatiotemporal graph attention network. Sci Rep 13(1):14729

Cheng D, Yang F, Xiang S, Liu J (2022) Financial time series forecasting with multi-modality graph neural network. Pattern Recogn 121:108218

Cirstea R-G, Guo C, Yang B, Kieu T, Dong X, Pan S (2022) Triformer: Triangular, variable-specific attentions for long sequence multivariate time series forecasting–full version. arXiv preprint arXiv:2204.13767

Cleveland RB, Cleveland WS, McRae JE, Terpenning I (1990) STL: a seasonal-trend decomposition. J Off Stat 6(1):3–73

Contreras J, Espinola R, Nogales FJ, Conejo AJ (2003) Arima models to predict next-day electricity prices. IEEE Trans Power Syst 18(3):1014–1020

Cui Z (2016) Multi-scale convolutional neural networks for time series classification. arXiv preprint arXiv:1603.06995
Dagum EB (2010) Time series modeling and decomposition. Statistica (Bologna) 70(4):433–457
De Livera AM, Hyndman RJ, Snyder RD (2011) Forecasting time series with complex seasonal patterns using exponential smoothing. J Am Stat Assoc 106(496):1513–1527
Devlin J, Chang M-W, Lee K, Toutanova K (2018) Bert: Pre-training of deep bidirectional transformers for language understanding. arXiv preprint arXiv:1810.04805
Dey R, Salem FM (2017) Gate-variants of gated recurrent unit (gru) neural networks. In: 2017 IEEE 60th international midwest symposium on circuits and systems (MWSCAS), pp. 1597–1600. IEEE
Dinh TN, Thirunavukkarasu GS, Seyedmahmoudian M, Mekhilef S, Stojcevski A (2023) Predicting commercial building energy consumption using a multivariate multilayered long-short term memory time-series model. Appl Sci 13(13):7775
Dosovitskiy A, Beyer L, Kolesnikov A, Weissenborn D, Zhai X, Unterthiner T, Dehghani M, Minderer M, Heigold G, Gelly S et al. (2020) An image is worth 16x16 words: Transformers for image recognition at scale. arXiv preprint arXiv:2010.11929
Du Y, Wang J, Feng W, Pan S, Qin T, Xu R, Wang C (2021) Adarnn: Adaptive learning and forecasting of time series. In: Proceedings of the 30th ACM international conference on information and knowledge management, pp. 402–411
Durbin J, Koopman SJ (2012) Time series analysis by state space methods, vol 38. Oxford University Press, Oxford
Edition F, Papoulis A, Pillai SU (2002) Probability, random variables, and stochastic processes. New York, McGraw-Hill
Ensafi Y, Amin SH, Zhang G, Shah B (2022) Time-series forecasting of seasonal items sales using machine learning-a comparative analysis. Int J Inf Manag Data Insights 2(1):100058
Fan C, Wang J, Gang W, Li S (2019) Assessment of deep recurrent neural network-based strategies for short-term building energy predictions. Appl Energy 236:700–710
Fischer T, Krauss C (2018) Deep learning with long short-term memory networks for financial market predictions. Eur J Oper Res 270(2):654–669
Gál V, Hámori J, Roska T, Bálya D, Borostyánkői Z, Brendel M, Lotz K, Négyessy L, Orzó L, Petrás I (2004) Receptive field atlas and related CNN models. Int J Bifurc Chaos 14(02):551–584
Gneiting T, Balabdaoui F, Raftery AE (2007) Probabilistic forecasts, calibration and sharpness. J R Stat Soc Ser B Stat Methodol 69(2):243–268
Goodfellow I, Pouget-Abadie J, Mirza M, Xu B, Warde-Farley D, Ozair S, Courville A, Bengio Y (2014) Generative adversarial nets. Adv Neural Inf Process Syst. **27**
Gudelek MU, Boluk SA, Ozbayoglu AM (2017) A deep learning based stock trading model with 2-D CNN trend detection. In: 2017 IEEE symposium series on computational intelligence (SSCI), pp. 1–8. IEEE
Guo J, Lin P, Zhang L, Pan Y, Xiao Z (2023) Dynamic adaptive encoder-decoder deep learning networks for multivariate time series forecasting of building energy consumption. Appl Energy 350:121803
Hajirahimi Z, Khashei M (2023) Hybridization of hybrid structures for time series forecasting: a review. Artif Intell Rev 56(2):1201–1261
Hatami N, Gavet Y, Debayle J (2018) Classification of time-series images using deep convolutional neural networks. In: Tenth international conference on machine vision (ICMV 2017), vol. 10696, pp. 242–249. SPIE
Hewamalage H, Bergmeir C, Bandara K (2021) Recurrent neural networks for time series forecasting: current status and future directions. Int J Forecast 37(1):388–427
He K, Zhang X, Ren S, Sun J (2016) Deep residual learning for image recognition. In: Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 770–778
Hsieh WW (2004) Nonlinear multivariate and time series analysis by neural network methods. Rev Geophys. https://doi.org/10.1029/2002RG000112
Hu S, Xiong C (2023) High-dimensional population inflow time series forecasting via an interpretable hierarchical transformer. Transp Res Part C Emerg Technol 146:103962
Hyndman R (2008) Forecasting with Exponential Smoothing: The State Space Approach. Springer, Berlin
Hyndman RJ, Athanasopoulos G (2018) Forecasting: principles and practice. OTexts
Hyndman RJ, Khandakar Y (2008) Automatic time series forecasting: the forecast package for R. J Stat Softw 27:1–22
Hyndman R, Koehler AB, Ord JK, Snyder RD (2008) Forecasting with exponential smoothing: the state space approach. Springer, USA
Iwata T, Kumagai A (2020) Few-shot learning for time-series forecasting. arXiv preprint arXiv:2009.14379
Jaiswal A, Babu AR, Zadeh MZ, Banerjee D, Makedon F (2020) A survey on contrastive self-supervised learning. Technologies 9(1):2

Jiang S, Yu Z-G, Anh VV, Zhou Y (2021) Long-and short-term time series forecasting of air quality by a multi-scale framework. Environ Pollut 271:116381
Jin M, Zheng Y, Li Y-F, Chen S, Yang B, Pan S (2022) Multivariate time series forecasting with dynamic graph neural odes. IEEE Trans Knowl Data Eng 35:9168
Jin M, Wen Q, Liang Y, Zhang C, Xue S, Wang X, Zhang J, Wang Y, Chen H, Li X et al. (2023) Large models for time series and spatio-temporal data: a survey and outlook. arXiv preprint arXiv:2310.10196
Kim T, Kim J, Tae Y, Park C, Choi J-H, Choo J (2021) Reversible instance normalization for accurate time-series forecasting against distribution shift. In: International conference on learning representations
Kitaev N, Kaiser Ł, Levskaya A (2020) Reformer: The efficient transformer. arXiv preprint arXiv:2001.04451
Krizhevsky A, Sutskever I, Hinton GE (2017) Imagenet classification with deep convolutional neural networks. Commun ACM 60(6):84–90
Lacasa L, Nicosia V, Latora V (2015) Network structure of multivariate time series. Sci Rep 5(1):15508
Lai G, Chang W-C, Yang Y, Liu H (2018) Modeling long-and short-term temporal patterns with deep neural networks. In: The 41st international ACM SIGIR conference on research & development in information retrieval, pp. 95–104
Le P, Zuidema W (2016) Quantifying the vanishing gradient and long distance dependency problem in recursive neural networks and recursive lstms. arXiv preprint arXiv:1603.00423
Lee S, Hong J, Liu L, Choi W (2024) TS-Fastformer: fast transformer for time-series forecasting. ACM Trans Intell SystTechnol 15(2):1–20
Li S, Jin X, Xuan Y, Zhou X, Chen W, Wang Y-X, Yan X (2019) Enhancing the locality and breaking the memory bottleneck of transformer on time series forecasting. Adv Neural Inf Process Syst. **32**
Liang Y, Liu Y, Wang X, Zhao Z (2024) Exploring large language models for human mobility prediction under public events. Comput Environ Urban Syst 112:102153
Liao R, Xiong Y, Fetaya E, Zhang L, Yoon K, Pitkow X, Urtasun R, Zemel R (2018) Reviving and improving recurrent back-propagation. In: International conference on machine learning, pp. 3082–3091. PMLR
Lim B, Arık SÖ, Loeff N, Pfister T (2021) Temporal fusion transformers for interpretable multi-horizon time series forecasting. Int J Forecast 37(4):1748–1764
Li Z, Qi S, Li Y, Xu Z (2023) Revisiting long-term time series forecasting: An investigation on linear mapping. arXiv preprint arXiv:2305.10721
Lin Y, Koprinska I, Rana M (2021) SsdNet: State space decomposition neural network for time series forecasting. In: 2021 IEEE International conference on data mining (ICDM), pp. 370–378. IEEE
Lin H, Gharehbaghi A, Zhang Q, Band SS, Pai HT, Chau K-W, Mosavi A (2022) Time series-based groundwater level forecasting using gated recurrent unit deep neural networks. Eng Appl Comput Fluid Mech 16(1):1655–1672
Liu Z, Yang X (2022) Cross validation for uncertain autoregressive model. Commun Stat Simul Comput 51(8):4715–4726
Liu C-L, Hsaio W-H, Tu Y-C (2018) Time series classification with multivariate convolutional neural network. IEEE Trans Industr Electron 66(6):4788–4797
Liu Z, Lin Y, Cao Y, Hu H, Wei Y, Zhang Z, Lin S, Guo B (2021) Swin transformer: Hierarchical vision transformer using shifted windows. In: Proceedings of the IEEE/CVF international conference on computer vision, pp. 10012–10022
Liu S, Yu H, Liao C, Li J, Li W, Liu AX, Dustdar S (2021) Pyraformer: Low-complexity pyramidal attention for long-range time series modeling and forecasting. In: International conference on learning representations
Liu Y, Wu H, Wang J, Long M (2022) Non-stationary transformers: exploring the stationarity in time series forecasting. Adv Neural Inf Process Syst 35:9881–9893
Liu Y, Hu T, Zhang H, Wu H, Wang S, Ma L, Long M (2023) itransformer: Inverted transformers are effective for time series forecasting. arXiv preprint arXiv:2310.06625
Liu Z, Cheng M, Li Z, Huang Z, Liu Q, Xie Y, Chen E (2024) Adaptive normalization for non-stationary time series forecasting: A temporal slice perspective. Adv Neural Inf Process Syst. **36**
Lu W, Li J, Li Y, Sun A, Wang J (2020) A CNN-LSTM-based model to forecast stock prices. Complexity 2020:1–10
Luo W, Li Y, Urtasun R, Zemel R (2016) Understanding the effective receptive field in deep convolutional neural networks. Adv Neural Inf Process Syst. **29**
Lv Y, Duan Y, Kang W, Li Z, Wang F-Y (2014) Traffic flow prediction with big data: a deep learning approach. IEEE Trans Intell Transp Syst 16(2):865–873
Lyu H, Sha N, Qin S, Yan M, Xie Y, Wang R (2019) Advances in neural information processing systems. Adv Neural Inf Process Syst. **32**
Ma M, Xie P, Teng F, Wang B, Ji S, Zhang J, Li T (2023) Histgnn: Hierarchical spatio-temporal graph neural network for weather forecasting. Inf Sci 648:119580

Maggiolo M, Spanakis G (2019) Autoregressive convolutional recurrent neural network for univariate and multivariate time series prediction. arXiv preprint arXiv:1903.02540
Malenica I, Phillips RV, Pirracchio R, Chambaz A, Hubbard A, Laan MJ (2021) Personalized online machine learning. arXiv preprint arXiv:2109.10452
Mao J, Middleton SE, Niranjan M (2023) Prompt position really matters in few-shot and zero-shot NLU tasks. arXiv preprint arXiv:2305.14493
Markova M (2022) Convolutional neural networks for forex time series forecasting. In: AIP conference proceedings, vol. 2459. AIP Publishing
Mathieu M, Henaff M, LeCun Y (2013) Fast training of convolutional networks through ffts. arXiv preprint arXiv:1312.5851
Miller JA, Aldosari M, Saeed F, Barna NH, Rana S, Arpinar IB, Liu N (2024) A survey of deep learning and foundation models for time series forecasting. arXiv preprint arXiv:2401.13912
Mintarsih F, Rasyidi MA, Nurjannah W, Khairani D, Sukmana HT(2023) Lstm variants comparison for exchange rate idr/usd forecasting with rolling window cross validation. In: 2023 Eighth international conference on informatics and computing (ICIC), pp. 1–4. IEEE
Mogren O (2016) Continuous recurrent neural networks with adversarial training. arXiv preprint arXiv:1611.09904
Muandet K, Balduzzi D, Schölkopf B (2013) Domain generalization via invariant feature representation. In: International conference on machine learning, pp. 10–18. PMLR
Mung PS, Phyu S (2023) Time series weather data forecasting using deep learning. In: 2023 IEEE conference on computer applications (ICCA), pp. 254–259. IEEE
Murray FT, Ringwood JV, Austin PC (2000) Integration of multi-time-scale models in time series forecasting. Int J Syst Sci 31(10):1249–1260
Nie Y, Nguyen NH, Sinthong P, Kalagnanam J (2022) A time series is worth 64 words: Long-term forecasting with transformers. arXiv preprint arXiv:2211.14730
Niu Z, Zhong G, Yu H (2021) A review on the attention mechanism of deep learning. Neurocomputing 452:48–62
Noh S-H (2021) Analysis of gradient vanishing of RNNS and performance comparison. Information 12(11):442
Orang O, Silva PC, Guimarães FG (2023) Time series forecasting using fuzzy cognitive maps: a survey. Artif Intell Rev 56(8):7733–7794
Oreshkin BN, Carpov D, Chapados N, Bengio Y (2019) N-beats: Neural basis expansion analysis for interpretable time series forecasting. arXiv preprint arXiv:1905.10437
Parzen E (1961) An approach to time series analysis. Ann Math Stat 32(4):951–989
Pascanu R, Mikolov T, Bengio Y (2013) On the difficulty of training recurrent neural networks. In: international conference on machine learning, pp. 1310–1318. PMLR
Passalis N, Tefas A, Kanniainen J, Gabbouj M, Iosifidis A (2019) Deep adaptive input normalization for time series forecasting. IEEE Trans Neural Netw Learn Syst 31(9):3760–3765
Pavlov-Kagadejev M, Jovanovic L, Bacanin N, Deveci M, Zivkovic M, Tuba M, Strumberger I, Pedrycz W (2024) Optimizing long-short-term memory models via metaheuristics for decomposition aided wind energy generation forecasting. Artif Intell Rev 57(3):45
Pöppelbaum J, Chadha GS, Schwung A (2022) Contrastive learning based self-supervised time-series analysis. Appl Soft Comput 117:108397
Qi M, Zhang GP (2008) Trend time-series modeling and forecasting with neural networks. IEEE Trans Neural Netw 19(5):808–816
Qi X, Hou K, Liu T, Yu Z, Hu S, Ou W (2021) From known to unknown: Knowledge-guided transformer for time-series sales forecasting in alibaba. arXiv preprint arXiv:2109.08381
Rawat AS, Chen J, Yu FXX, Suresh AT, Kumar S (2019) Sampled Softmax with random Fourier features. Adv Neural Inf Process Syst. **32**
Salinas D, Flunkert V, Gasthaus J, Januschowski T (2020) Deepar: probabilistic forecasting with autoregressive recurrent networks. Int J Forecast 36(3):1181–1191
Seeger MW, Salinas D, Flunkert V (2016) Bayesian intermittent demand forecasting for large inventories. Adv Neural Inf Process Syst. **29**
Series ST forecasting seasonal time series
Shabani, A., Abdi, A., Meng, L., Sylvain, T.: Scaleformer: iterative multi-scale refining transformers for time series forecasting. arXiv preprint arXiv:2206.04038 (2022)
Shelatkar T, Tondale S, Yadav S, Ahir S (2020) Web traffic time series forecasting using ARIMA and LSTM RNN. In: ITM Web of Conferences, Vol 32, p. 03017. EDP Sciences

Shi X, Chen Z, Wang H, Yeung D-Y, Wong W-K, Woo W-c (2015) Convolutional lstm network: a machine learning approach for precipitation nowcasting. Adv Neural Inf Process Syst. **28**
Shin T, Razeghi Y, Logan IV RL, Wallace E, Singh S (2020) Autoprompt: Eliciting knowledge from language models with automatically generated prompts. arXiv preprint arXiv:2010.15980
Soltani S (2002) On the use of the wavelet decomposition for time series prediction. Neurocomputing 48(1–4):267–277
Son NN, Van Cuong N (2023) Neuro-evolutionary for time series forecasting and its application in hourly energy consumption prediction. Neural Comput Appl 35(29):21697–21707
Szegedy C, Liu W, Jia Y, Sermanet P, Reed S, Anguelov D, Erhan D, Vanhoucke V, Rabinovich A (2015) Going deeper with convolutions. In: Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 1–9
Tang W, Long G, Liu L, Zhou T, Jiang J, Blumenstein M (2020) Rethinking 1d-cnn for time series classification: A stronger baseline. arXiv preprint arXiv:2002.10061, 1–7
Tang Y, Yu F, Pedrycz W, Yang X, Wang J, Liu S (2021) Building trend fuzzy granulation-based LSTM recurrent neural network for long-term time-series forecasting. IEEE Trans Fuzzy Syst 30(6):1599–1613
Taylor SJ, Letham B (2018) Forecasting at scale. Am Stat 72(1):37–45
Ulyanov D, Vedaldi A, Lempitsky V (2016) Instance normalization: The missing ingredient for fast stylization. arXiv preprint arXiv:1607.08022
Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, Kaiser Ł, Polosukhin I (2017) Attention is all you need. Adv Neural Inf Process Syst. 30
Venkateshwari P, Veeraiah V, Talukdar V, Gupta DN, Anand R, Gupta A (2023) Smart city technical planning based on time series forecasting of iot data. In: 2023 international conference on sustainable emerging innovations in engineering and technology (ICSEIET), pp. 646–651. IEEE
Verbesselt J, Hyndman R, Newnham G, Culvenor D (2010) Detecting trend and seasonal changes in satellite image time series. Remote Sens Environ 114(1):106–115
Wang Z, Bovik AC (2009) Mean squared error: love it or leave it? A new look at signal fidelity measures. IEEE Signal Process Mag 26(1):98–117
Wang X, Cai Z, Luo Y, Wen Z, Ying S (2022) Long time series deep forecasting with multiscale feature extraction and seq2seq attention mechanism. Neural Process Lett 54(4):3443–3466
Wang Z, Liu N, Chen C, Guo Y (2023) Adaptive self-attention LSTM for RUL prediction of lithium-ion batteries. Inf Sci 635:398–413
Wang S, Fan Y, Jin S, Takyi-Aninakwa P, Fernandez C (2023) Improved anti-noise adaptive long short-term memory neural network modeling for the robust remaining useful life prediction of lithium-ion batteries. Reliab Eng Syst Safety 230:108920
Weerakody PB, Wong KW, Wang G, Ela W (2021) A review of irregular time series data handling with gated recurrent neural networks. Neurocomputing 441:161–178
Wen Q, Sun L, Yang F, Song X, Gao J, Wang X, Xu H (2020) Time series data augmentation for deep learning: a survey. arXiv preprint arXiv:2002.12478
Wen Q, Zhou T, Zhang C, Chen W, Ma Z, Yan J, Sun L (2022) Transformers in time series: a survey. arXiv preprint arXiv:2202.07125
West M (1997) Time series decomposition. Biometrika 84(2):489–494
Weston J, Chopra S, Bordes A (2014) Memory networks. arXiv preprint arXiv:1410.3916
Woo G, Liu C, Sahoo D, Kumar A, Hoi S (2022) Etsformer: Exponential smoothing transformers for time-series forecasting. arXiv preprint arXiv:2202.01381
Woodward WA, Gray HL (1993) Global warming and the problem of testing for trend in time series data. J Clim 6(5):953–962
Wu S, Xiao X, Ding Q, Zhao P, Wei Y, Huang J (2020) Adversarial sparse transformer for time series forecasting. Adv Neural Inf Process Syst 33:17105–17115
Wu H, Xu J, Wang J, Long M (2021) Autoformer: decomposition transformers with auto-correlation for long-term series forecasting. Adv Neural Inf Process Syst 34:22419–22430
Yazdanbakhsh O, Dick S (2019) Multivariate time series classification using dilated convolutional neural network. arXiv preprint arXiv:1905.01697
Yin J, Rao W, Yuan M, Zeng J, Zhao K, Zhang C, Li J, Zhao Q (2019) Experimental study of multivariate time series forecasting models. In: Proceedings of the 28th ACM international conference on information and knowledge management, pp. 2833–2839
Young, J., Chen, J., Huang, F., Peng, J.: Dateformer: Time-modeling transformer for longer-term series forecasting. arXiv preprint arXiv:2207.05397 (2022)

Yu F, Koltun V (2015) Multi-scale context aggregation by dilated convolutions. arXiv preprint arXiv:1511.07122
Yun C, Bhojanapalli S, Rawat AS, Reddi SJ, Kumar S (2019) Are transformers universal approximators of sequence-to-sequence functions? arXiv preprint arXiv:1912.10077
Zhai Y, Lv Z, Zhao J, Wang W (2023) Knowledge discovery and variable scale evaluation for long series data. Artif Intell Rev 56(4):3157–3180
Zhang GP (2003) Time series forecasting using a hybrid ARIMA and neural network model. Neurocomputing 50:159–175
Zhang Y, Yan J (2022) Crossformer: Transformer utilizing cross-dimension dependency for multivariate time series forecasting. In: The eleventh international conference on learning representations
Zhang X, Jin X, Gopalswamy K, Gupta G, Park Y, Shi X, Wang H, Maddix DC, Wang Y (2022) First de-trend then attend: Rethinking attention for time-series forecasting. arXiv preprint arXiv:2212.08151
Zhang X, Li Q, Liang D (2023) An adaptive spatio-temporal neural network for PM2.5 concentration forecasting. Artif Intell Rev 56(12):14483–14510
Zhang J, Li X, Tian J, Luo H, Yin S (2023) An integrated multi-head dual sparse self-attention network for remaining useful life prediction. Reliab Eng Syst Safety 233:109096
Zhang K, Zhou F, Wu L, Xie N, He Z (2024) Semantic understanding and prompt engineering for large-scale traffic data imputation. Inf Fus 102:102038
Zhao B, Lu H, Chen S, Liu J, Wu D (2017) Convolutional neural networks for time series classification. J Syst Eng Electron 28(1):162–169
Zhao WX, Zhou K, Li J, Tang T, Wang X, Hou Y, Min Y, Zhang B, Zhang J, Dong Z, et al (2023) A survey of large language models. Preprint at arXiv:2303.18223
Zheng S, Ristovski K, Farahat A, Gupta C (2017) Long short-term memory network for remaining useful life estimation. In: 2017 IEEE international conference on prognostics and health management (ICPHM), pp. 88–95. IEEE
Zhong X, Cambria E (2023) Time expression recognition and normalization: a survey. Artif Intell Rev 56(9):9115–9140
Zhou H, Zhang S, Peng J, Zhang S, Li J, Xiong H, Zhang W (2021) Informer: Beyond efficient transformer for long sequence time-series forecasting. In: Proceedings of the AAAI conference on artificial intelligence, vol. 35, pp. 11106–11115
Zhou T, Ma Z, Wen Q, Wang X, Sun L, Jin R (2022) Fedformer: Frequency enhanced decomposed transformer for long-term series forecasting. In: International conference on machine learning, pp. 27268–27286. PMLR

Authors and Affiliations

Xiaobao Song[1,2] · Liwei Deng[2] · Hao Wang[1,2] · Yaoan Zhang[2] · Yuxin He[3] · Wenming Cao[1,2]

✉ Hao Wang
haowang@szu.edu.cn

Xiaobao Song
2020284089@email.szu.edu.cn

Liwei Deng
2021280356@email.szu.edu.cn

Yaoan Zhang
2021280103@email.szu.edu.cn

Yuxin He
heyuxin@sztu.edu.cn

Wenming Cao
wmcao@szu.edu.cn

[1] Guangdong Provincial Key Laboratory of Intelligent Information Processing, Shenzhen University, Shenzhen, China

[2] Guangdong Multimedia Information Service Engineering Technology Research Cente, Shenzhen University, Shenzhen, China

[3] College of Urban Transportation and Logistics, Shenzhen Technology University, Shenzhen, China